Simulating Spectral Sampling of the MICA Files with Enfys: InAs MWIR

August 31, 2025

1 Abstract

Here we investigate the expected ability of a new remote-sensing stand-off infrared spectrometer, *Enfys*, for the ExoMars Rosalind Franklin Rover, to resolve diagnostic features of the reflectance spectra of key minerals previously identified on the surface of Mars, using the Spectral Angle as a metric.

In this notebook we simulate Enfys performance using an InAs Mid-Wave Infrared (1.6 - 3.1 µm, 'MWIR') sensor, that is subject to thermal noise from the Enfys structure.

2 Introduction

As discussed in previous notebooks, Enfys is an Infra-Red point-spectrometer for the ExoMars Rosalind Franklin rover.

In this notebook, and the others accompanying it, we explore how we expect Enfys to sample Minerals Identified through CRISM Analysis, as defined by the 'MICA' files.

In this notebook, we consider the expected performance of Enfys using an InAs Mid-Wave Infrared sensor for the wavelength range of 1.6 - 3.1 µm. Included in the modelling is the noise expected for the sensor, as provided by the EXM-EN-MTX-ABU-0001_Enfys_Science_Traceability_Matrix_3-0 document (contact Enfys P.I. M. Gunn).

2.1 Problem

The optical design of Enfys is functionally different to the Acousto-Optic Tunable Filter design of it's predecessor ISEM, so it is important that the design is demonstrated to fulfil the performance requirements of the instrument, and that expectations of performance can be set prior to laboratory tests of the completed instrument.

At time of writing, the spectrometer is in the development stage. This notebook describes how the Spectral Parameters Toolkit[1] can be used to convert the developing expectations of the spectral range and resolution and radiometric sensitivity of the instrument into predictions of its ability to discriminate mineral species and phases on the Mars surface at the rover landing site.

In these notebooks we consider two candidate MWIR sensors, that trade-off spectral range against Signal-to-Noise Ratio.

[1] Stabbins, R., & Grindrod, P. (2024). The Spectral Parameters Toolkit (sptk): Release v0.1 (Version v0.1) [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.10694286

Candidate A is the InAs sensor, simulated here, that is sensitive up to ~3.1 µm, with a nominal SNR range of 100 - 600.

Candidate B is a modified InGaAs sensor, sensitive to ~2.5 µm, with a nominal SNR range of ~2000 - 60,000.

3 Method

To investigate the expected performance we use high-resolution laboratory measured spectra of a range of minerals expected at the Mars surface, and computationally simulate the sampling of the spectra with the expected spectral response of *Enfys*. We will perform repeat sampling with synthetically added noise to investigate the spectral and radiometric limits of the instrument.

We will produce a number of visualisations to investigate the predicted instrument performance, and demonstrate the limits of the instrument discriminatory ability.

We will quantify the ability of the instrument to resolve the defined minerals of interest by computing the Spectral Angle between a noisy sampled spectrum of a given mineral and all other 'clean' (resampled to Enfys spectral channels, but without noise added). We will compare the noisy sample Spectral Angle to a noise-free baseline Spectral Angle, to understand the performance limits and the impact of the simulated instrument noise.

3.1 Spectral Resampling

We take a simple approach to resampling, as follows.

We neglect reflectance calibration uncertainty, and any systematic uncertainty when merging the data from the two separate photodiodes, and assume that the linear-variable filter (LVF) movement is perfectly calibrated such that a measurement at centre-wavelength λ_{cwl} can be requested with arbitrary precision.

We assume an observation of λ_{cwl} has a Cauchy transmission profile. We use the latest expectations of the spectral resolving power of the short-wave infrared (SWIR) and mid-wave infrared (MWIR) LVFs, that we have linearly interpolated from ~30 measurements from 0.9 - 2.6 µm to arbitrary values of λ in the SWIR and MWIR ranges, to give the vector $\Gamma[\lambda]$. The Cauchy transmission profile for a given λ_{cwl} has a full-width-at-half-maximum of $\Delta\lambda = \lambda_{cwl}/\Gamma[\lambda_{cwl}]$.

This data has been digitsed from the plot reported in the `Enfys schedule review 20250807.ppt`, by Matt Gunn.

[92]:
```
import pandas as pd

enfys_swir_respow = pd.read_csv('../data/instruments/enfys_swir_respow.csv',␣
 ↪index_col=0)
enfys_mwir_respow = pd.read_csv('../data/instruments/enfys_mwir_respow.csv',␣
 ↪index_col=0)
g=enfys_swir_respow.plot(label='SWIR', marker='o')
enfys_mwir_respow.plot(marker='o',label='MWIR', xlabel='Wavelength (nm)',␣
 ↪ylabel=r'$\lambda/\Delta\lambda$ Resolving Power',ax=g)
g.legend(['SWIR', 'MWIR'])
```

```
title = g.set_title('Enfys SWIR & MWIR Spectral Resolving Power␣
 ↪$\Gamma[\lambda]$')
```

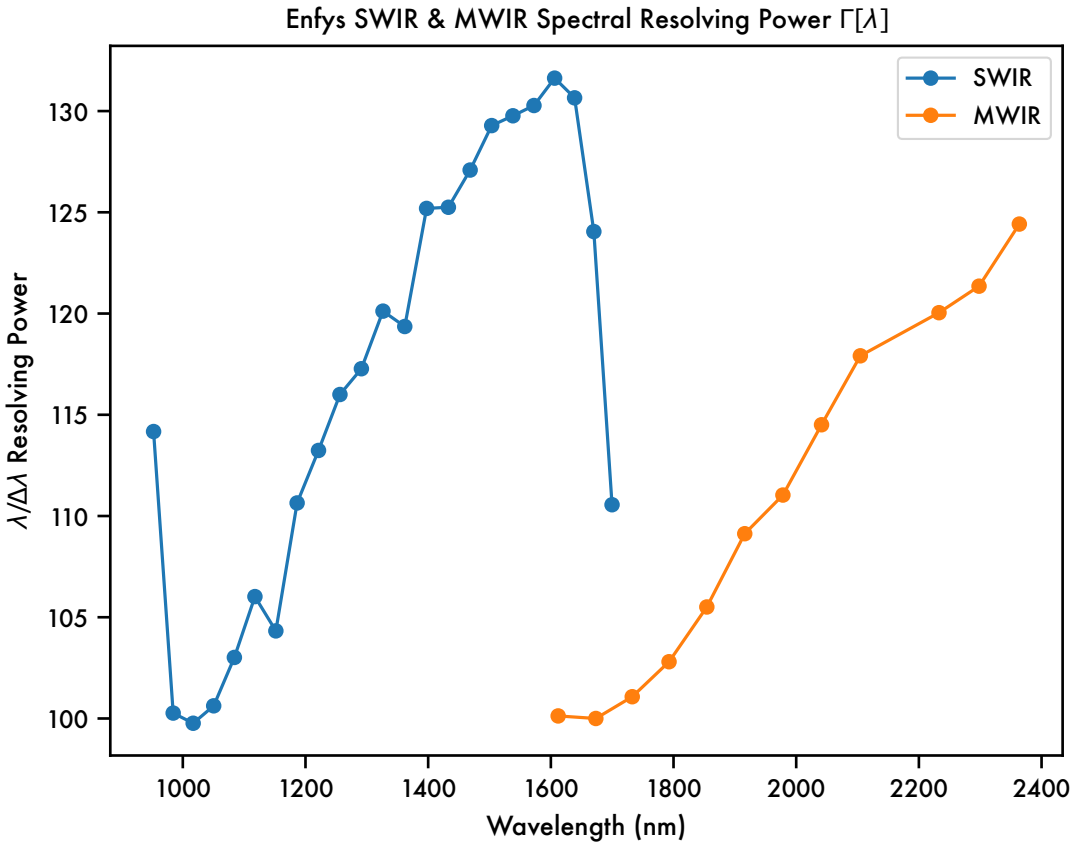


Figure 1 Enfys SWIR & MWIR Spectral Resolving Power $\Gamma[\lambda]$, extracted from `Enfys schedule review 20250807.ppt`, M. Gunn.

The Cauchy transmission profile $T(\lambda|\lambda_{cwl})$ is given by

$$T(\lambda|\lambda_{cwl}) = \frac{\gamma^2}{(\lambda - \lambda_{cwl}^2) + \gamma^2}$$

where $\gamma = \Delta\lambda/2 = (\lambda_{cwl}/\Gamma[\lambda_{cwl})/2$.

The shape of the Cauchy profile is illustrated here.

[93]:
```
from sptk.instrument import Instrument
import sptk.config as cfg
from matplotlib import pyplot as plt

exmpl_cauchy = Instrument.build_cauchy_filter(1600, 1600/100)
plt.plot(cfg.WVLS, exmpl_cauchy.squeeze())
```

```
plt.xlim(1500, 1700)
plt.xlabel('Wavelength (nm)')
plt.ylabel('Relative Transmittance')
title = plt.title('Example Cauchy Filter at 1600 nm with Spectral Resolving␣
 ↪Power $\Gamma$ = 100')
```

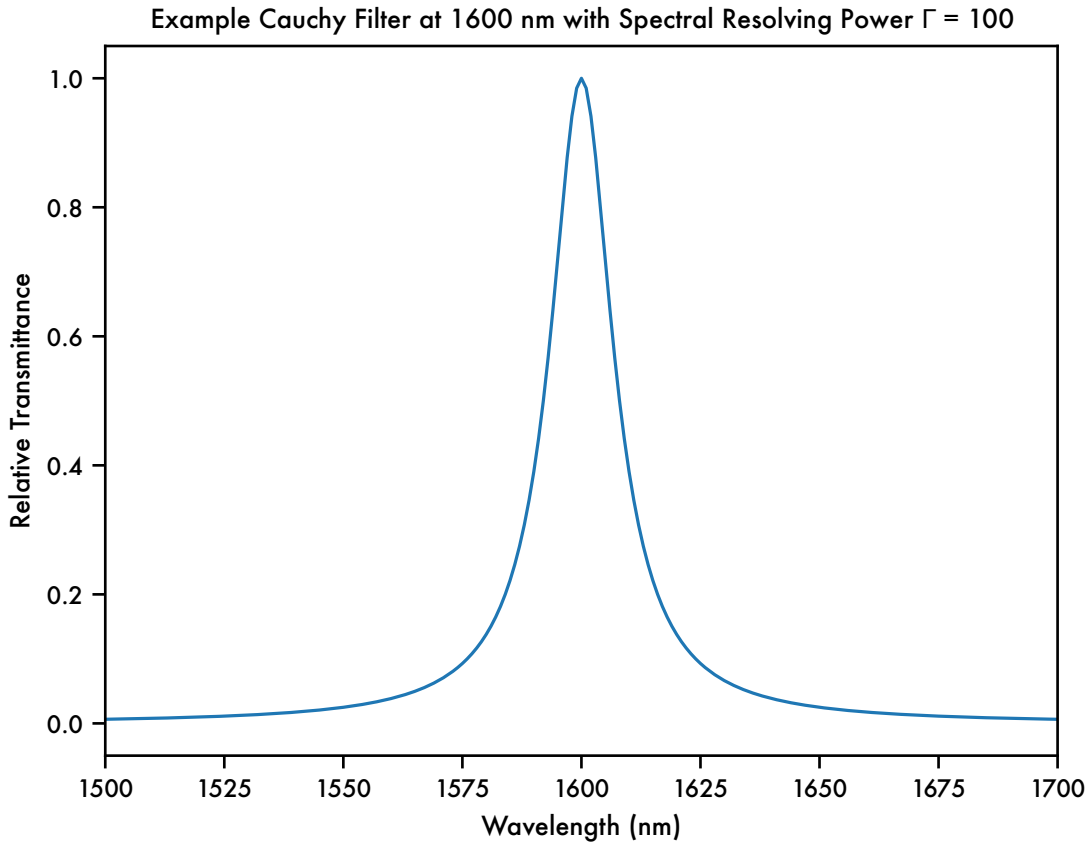


Figure 2 Example Cauchy Filter transmission profile, at 1600 nm with Spectral Resolving Power $\Gamma = 100$.

We assume that a high-resolution laboratory spectrum approximates the continuous function $R(\lambda)$, such that the calibrated reflectance at λ_{cwl} has the expected value $R[\lambda_{cwl}]$ of:

$$R[\lambda_{cwl}] = \frac{\int R(\lambda)T(\lambda|\lambda_{cwl})d\lambda}{\int T(\lambda|\lambda_{cwl})d\lambda}$$

We use this equation to compute the Enfys-sampled reflectance spectra.

3.2 Adding Noise

To simulate noise we add ε to a reflectance measurement, an additive noise term drawn from the Gaussian distribution with standard deviation σ, that we define relative to the specified Signal-to-

Noise Ratio (SNR),

$$\sigma = \frac{S}{\text{SNR}}$$

where S is the signal.

The supplied Signal-to-Noise Ratio vector has been defined under the assumption of a 30% albedo spectrally uniform reflector.

The expected spectral SNR of the SWIR and MWIR sensors has been measured and processed by P.I. M. Gunn, and reported in the `EXM-EN-MTX-ABU-0001_Enfys_Science_Traceability_Matrix_3-0` spreadsheet.

We read in the SNR file, and get the SNR data for the InAs MWIR sensor and the InGaAs SWIR sensor,

[94]:
```
enfys_snr = pd.read_csv('../data/instruments/enfys_snr.csv', index_col=0)
inas_mwir_snr = enfys_snr['InAs MWIR SNR']
ingaas_swir_snr = enfys_snr['InGaAs SWIR SNR']

g = ingaas_swir_snr.plot(grid=True, label='Nominal InGaAs SWIR SNR')
inas_mwir_snr.plot(logy=True, grid=True, label='Nominal InAs MWIR SNR')
g.set_xlabel('Wavelength (nm)')
g.set_ylabel('SNR')
g.legend()
title = g.set_title('Spectral SNR of InGaAs SWIR and InAs MWIR Sensors')
```

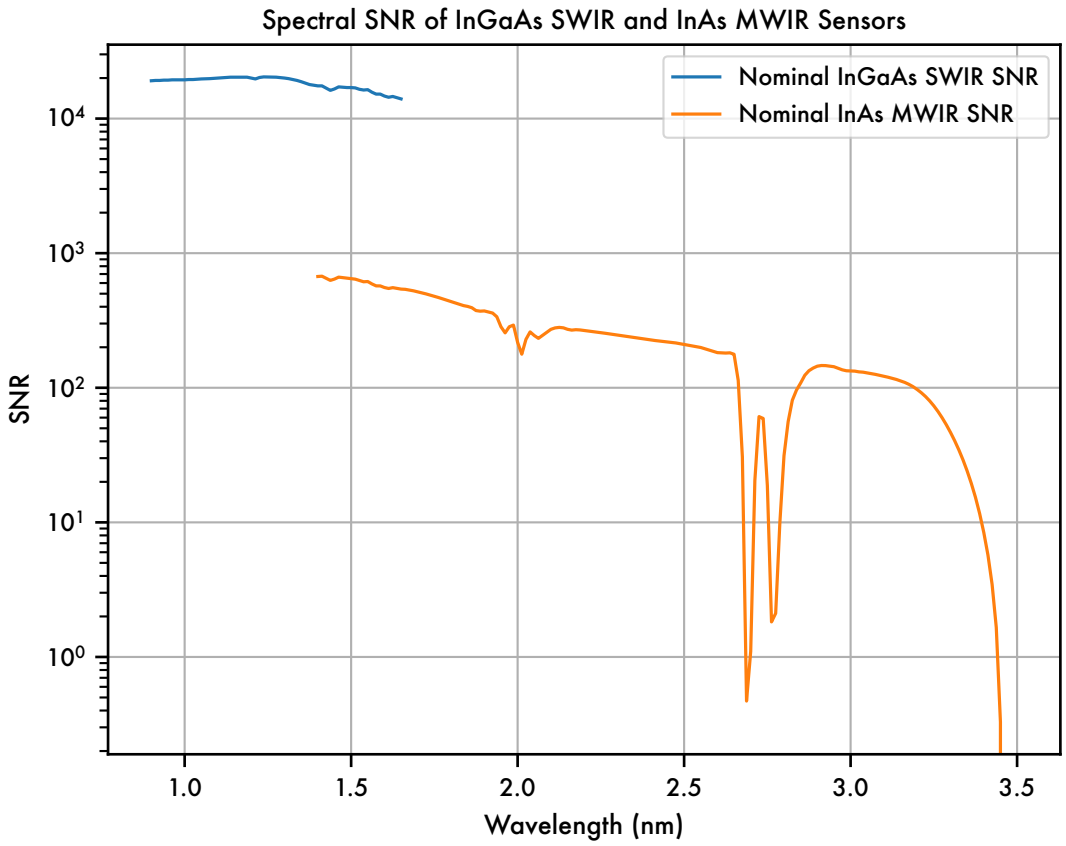


Figure 3 Spectral Signal-to-Noise Ratio for InGaAs SWIR and InAs MWIR Sensors, extracted from `EXM-EN-MTX-ABU-0001_Enfys_Science_Traceability_Matrix_3-0` spreadsheet, M. Gunn, and the 1/10th SNR performance simualted in this notebook.

We assume that this SNR vector is modulated by the reflectance of the mineral of interest, such that σ as a function of the spectral Signal-to-Noise Ratio SNR$[\lambda]$ is:

$$\sigma[\lambda] = \frac{R[\lambda]}{\text{SNR}[\lambda]}$$

For each observation $R[\lambda_{cwl}]$ we draw n-repeat noisy samples $\tilde{R}_i[\lambda_{cwl}]$, such that

$$\tilde{R}_i[\lambda_{cwl}] = R[\lambda_{cwl}] + \varepsilon_i[\lambda_{cwl}]$$

with

$$\varepsilon_i[\lambda_{cwl}] \sim \mathcal{N}\left(0, \sigma[\lambda_{cwl}]\right)$$

This gives an indication of the expected noise of a calibrated *Enfys* acquired spectrum.

3.3 Analysis

First we produce plots of the resampled spectra, and single instances of noisy spectra, and visually inspect these to consider the ability of this Enfys configuration to resolve diagnostic spectral features against the noise.

3.3.1 Spectral Angle

We then take a single noisy sample and evaluate the Spectral Angle between the noisy sample and each noise-free entry of the MICA files spectral database that we are sampling. We repeat this for each noisy sample, to give the mean Spectral Angle and it's uncertainty, to determine the ability to dicriminate between similar mineral reflectance spectra.

The Spectral Angle is defined[2] between the noisy spectrum $\tilde{R}_i[\lambda]$ of material i in the set of MICA file materials ($\mathcal{M}$) and the clean spectrum $R_j[\lambda]$, where $j \in \mathcal{M}$.

The Spectral Angle θ is:

$$\theta(i,j) = \arccos\left(\frac{\sum_\lambda \tilde{R}_i[\lambda] R_j[\lambda]}{\left(\sum_\lambda \tilde{R}_i[\lambda]^2\right)^{1/2}\left(\sum_\lambda R_j[\lambda]^2\right)^{1/2}}\right)$$

This closed form expression can be formulated in a way such that θ can be computed for all $\mathcal{M}$ for a given material i in parallel, as well as multiple noisy repeat samples of i.

3.4 Notebook and Environment Setup

Here we include the required code blocks for setting up this notebook for this study.

[95]:
```
%load_ext autoreload
%autoreload 2
%config InlineBackend.figure_format = 'svg' # note - requires Inkscape to be␣
 ↪installed, and to be accessible from the terminal
# on macos, add inkscape to the path with `sudo ln -s /Applications/Inkscape.
 ↪app/Contents/MacOS/inkscape /usr/local/bin/inkscape`
%matplotlib inline
```

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

We will be using the Spectral Parameters Toolkit. First we import the required modules.

[96]:
```
import sptk.config as cfg
from sptk.material_collection import MaterialCollection
from sptk.instrument import Instrument
from sptk.observation import Observation
from sptk.spectral_library_analyser import SpectralLibraryAnalyser as sla
```

[2] F.A. Kruse, A.B. Lefkoff, J.W. Boardman, K.B. Heidebrecht, A.T. Shapiro, P.J. Barloon, A.F.H. Goetz, The spectral image processing system (SIPS)—interactive visualization and analysis of imaging spectrometer data, Remote Sensing of Environment, Volume 44, Issues 2–3, 1993, Pages 145-163, ISSN 0034-4257, https://doi.org/10.1016/0034-4257(93)90013-N

We set our simulation resolution range to extend the range of Enfys, of 0.9 - 3.1 µm, such that the long tails of the Cauchy distribution of the linear-variable-filter profiles can be accommodated during sampling. We leave our spectral resolution at 1 nm for all λ.

[97]:
```
cfg.update_sample_res('wvl_min', 800)
cfg.update_sample_res('wvl_max', 3300)
```

Finally we set the project name.

[98]:
```
PROJECT_NAME = 'enfys_mica_inas_mwir'
```

4 Data

For this study we are using a spectral library of minerals identified through CRISM analysis (the MICA files)[3]. Here we use the laboratory spectra that have been identified that best represent the type localities presented in the MICA files. These can be accessed here, and are sourced from the RELAB and USGS public spectral libraries.

A comprehensive description of the mineral groups can be found in the [4] text.

Here we have omitted the CO2 and H2O ice entries, as tabulated laboratory data is not publicly available.

We have grouped the spectral library into categories as follows, and included this categorisation in the sptk distribution, accessed from the `sptk.material_sets` module.

[99]:
```
from sptk.material_sets import MICA_SET
```

We use the `sptk.material_collection` module to parse the spectral library:

[100]:
```
matcol = MaterialCollection(
        MICA_SET,
        'MICA_new_dirtree',
        PROJECT_NAME,
        load_existing=False,
        balance_classes=False,
        random_bias_seed=None,
        allow_out_of_bounds=True,
        plot_profiles=False,
        export_df=True)
```

```
Building new enfys_mica_inas_mwir MaterialCollection DF
```

[3] Viviano-Beck, C. E., Seelos, F. P., Murchie, S. L., Kahn, E. G., Seelos, K. D., Taylor, H. W., Taylor, K., Ehlmann, B. L., Wiseman, S. M., Mustard, J. F., & Morgan, M. F. (2014). Revised CRISM spectral parameters and summary products based on the currently detected mineral diversity on Mars. Journal of Geophysical Research: Planets, 119(6), 1403–1431. https://doi.org/10.1002/2014JE004627

[4] Viviano-Beck, C. E., Seelos, F. P., Murchie, S. L., Kahn, E. G., Seelos, K. D., Taylor, H. W., Taylor, K., Ehlmann, B. L., Wiseman, S. M., Mustard, J. F., & Morgan, M. F. (2014). Revised CRISM spectral parameters and summary products based on the currently detected mineral diversity on Mars. Journal of Geophysical Research: Planets, 119(6), 1403–1431. https://doi.org/10.1002/2014JE004627

```
---------------------------------
Category
  species|subgroup|group|
     data_id status
---------------------------------
Iron Oxides & Primary Silicates
  -clinopyroxene|pyroxenes|inosilicates|
     -NBPP22.csv loaded
  -orthopyroxene|pyroxenes|inosilicates|
     -LAPP47B.csv loaded
  -fayalite|olivines|nesosilicates|
     -LAPO05.csv loaded
  -forsterite|olivines|nesosilicates|
     -LASC02.csv loaded
  -hematite|hematites|oxides|
     -GDS27.csv loaded
  -plagioclase|feldspars|tectosilicates|
     -NCLS04.csv loaded
---------------------------------
Sulfates
  -alunite|hydrous_sulphates|sulphates|
     -F1CC08B.csv loaded
  -bassanite|hydrous_sulphates|sulphates|
     -GDS145.csv loaded
  -gypsum|hydrous_sulphates|sulphates|
     -LASF41A.csv loaded
  -jarosite|hydrous_sulphates|sulphates|
     -LASF21A.csv loaded
  -kieserite|hydrous_sulphates|sulphates|
     -F1CC15.csv loaded
  -mg-sulphate|hydrous_sulphates|sulphates|
     -799F366.csv loaded
---------------------------------
Phyllosilicates
  -chlorite|chlorites|phyllosilicates|
     -LACL14.csv loaded
  -kaolinite|kaolinites|phyllosilicates|
     -LAKA04.csv loaded
  -illite|micas|phyllosilicates|
     -LAIL02.csv loaded
  -margarite|micas|phyllosilicates|
     -GDS106.csv loaded
  -vermiculite|micas|phyllosilicates|
     -c1cy29.csv loaded
  -serpentine|serpentines|phyllosilicates|
     -LALZ01.csv loaded
  -montmorillonite|smectites|phyllosilicates|
     -LAMO02.csv loaded
```

```
  -nontronite|smectites|phyllosilicates|
     -NCJB26.csv loaded
  -saponite|smectites|phyllosilicates|
     -LASA58.csv loaded
  -talc|talcs|phyllosilicates|
     -GDS23.csv loaded
---------------------------------
Carbonates
  -calcite|calcites|carbonates|
     -CAGR01.csv loaded
  -magnesite|calcites|carbonates|
     -F1CC06B.csv loaded
---------------------------------
Hydrated Silicates & Halides
  -halite|chlorides|halides|
     -HS433.csv loaded
  -prehnite|prehnites|inosilicates|
     -LAZE03.csv loaded
  -siliceous-sinter|hydrated-silicas|mineraloids|
     -bkr1jb329c.csv loaded
  -epidote|epidotes|sorosilicates|
     -GDS26.csv loaded
  -analcime|zeolites|tectosilicates|
     -GDS1.csv loaded
Loading 29 of entries complete.
Exporting the Material Collection to CSV and Pickle formats…
```

We can use the `material_collection` plotting routines to visualise the library.

[101]:
```
ax = matcol.plot_profiles(stacked=True, scope='categories', groupby='Species')
```

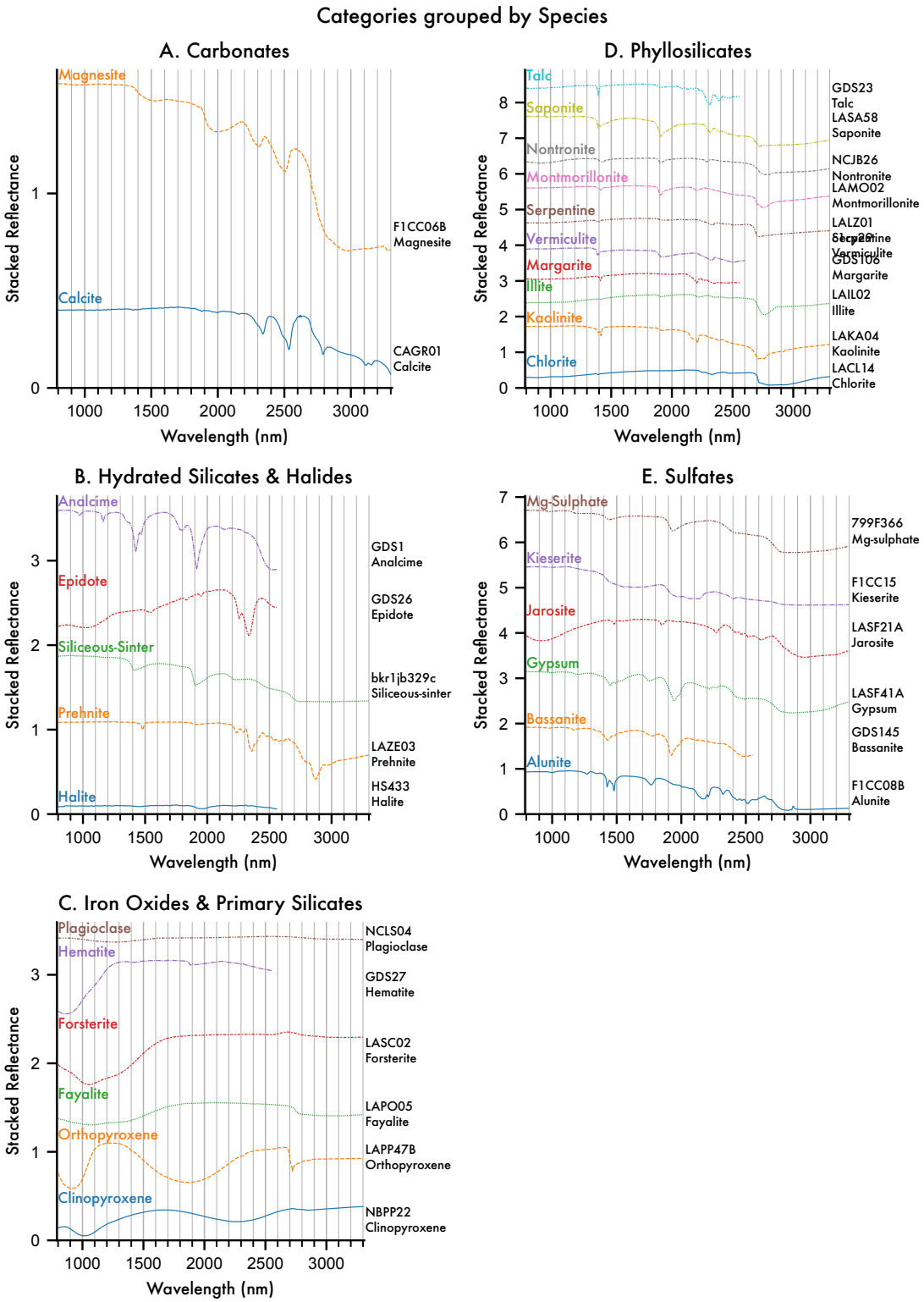


Figure 4 High resolution MICA Files spectral library, arranged by category and grouped by mineral

species.

We can see that not all of the entries cover the complete 0.9 - 3.1 µm range of *Enfys*, so in future work we should look to replace these MICA files laboratory type spectra with spectra that all span the entire range.

We can show band locations more clearly after continuum removal. We can use the `SpectralLibraryAnalyser.remove_continuum()` function to produce a duplicate of the `MaterialCollection` but with the data continuum-removed.

[102]:
```
from sptk.spectral_library_analyser import SpectralLibraryAnalyser as sla

matcol_sla = sla(matcol)
matcol_cr = matcol_sla.remove_continuum()
```

We can plot the profiles to illustrate the continuum removal.

[103]:
```
axes = matcol_cr.plot_profiles(stacked=True, scope='categories',␣
↪groupby='Subgroup')
```

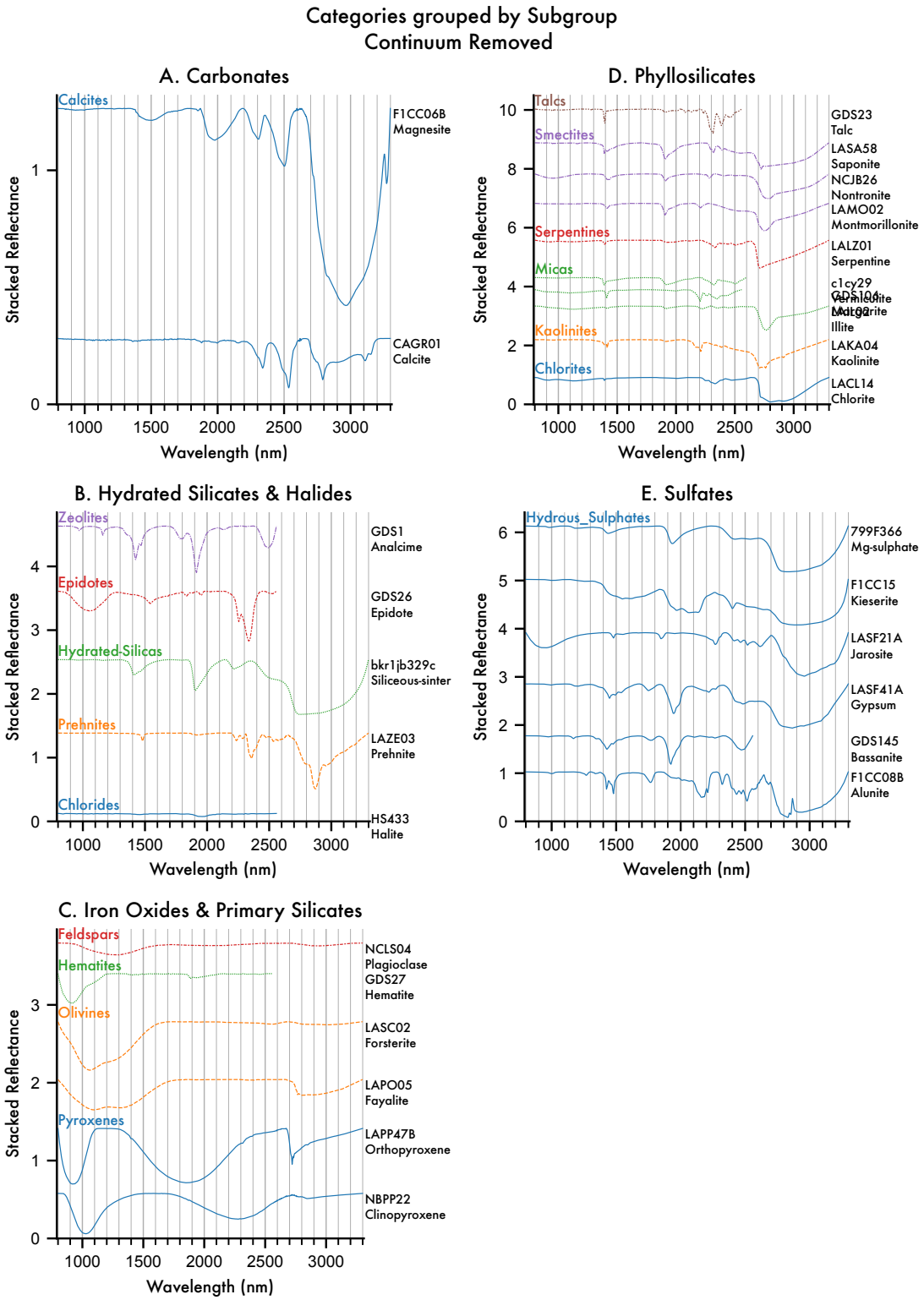


Figure 5 Continuum-removed high resolution MICA Files spectral library, grouped by Category.

We can illustrate the band locations more clearly with the 'waterfall' visualisation:

[104]:
```
axes = matcol_cr.plot_profiles(waterfall=True, scope='all', groupby='Category')
```

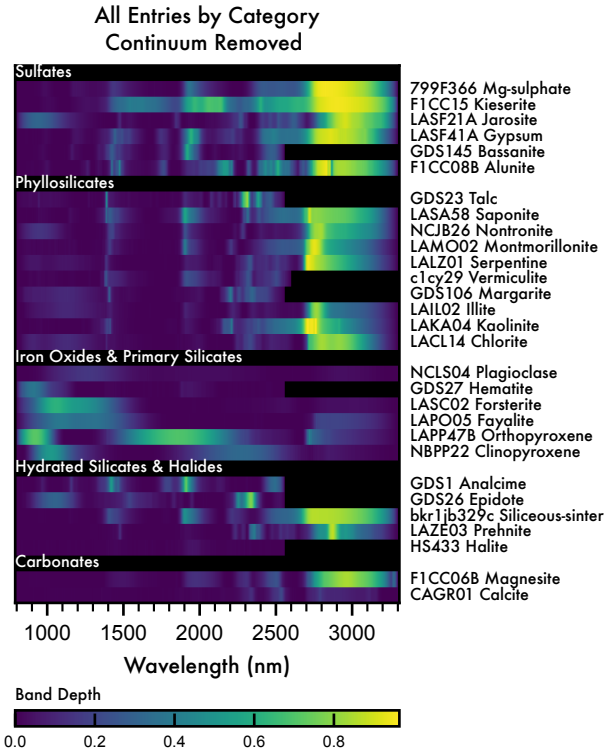


Figure 6 Continuum-removed waterfall plot of high resolution MICA Files spectral library, grouped by Category.

We can see that the waterfall plot represents the band depth information of the `MaterialCollection` in a compact format. Scanning vertically allows for quick identification of common band locations, their depths, widths, and asymmetries.

4.1 Simulating the Enfys Spectral Response

To simulate Enfys, rather than providing a file with the expected spectral response of each position of the Linear-Variable Filter (LVF, above), we specify the spectral range and resolution and sampling regime, and generate a set of central-wavelengths and bandwidths. We do this with the `InstrumentBuilder` class of the `sptk.instrument` module.

[105]:
```
from sptk.instrument import InstrumentBuilder
```

4.1.1 Signal-to-Noise Ratio

Previously we loaded the spectral SNR, as extracted from the current version of the Enfys STM. Here we merge the SWIR and MWIR SNR datasets, and prepare the data for passing to the

`sptk.Instrument` constructor.

We have adapted the InstrumentBuilder function to accept a pandas Series mapping wavelength to SNR. The InstrumentBuilder then performs interpolation to the simulated wavelength range.

We splice the SWIR and MWIR data together, using the higher value SNR where the wavelength ranges overlap:

[106]:
```
# concatanate the inas_mwir_snr and the ingaas_swir, using the highest SNR for␣
↪overlapping wavelength values
enfys_inas_mwir_snr = ingaas_swir_snr.combine_first(inas_mwir_snr)
```

We also convert the wavelengths from µm to nm:

[107]:
```
enfys_inas_mwir_snr.index = enfys_inas_mwir_snr.index * 1000
```

for computational convenience, we should set values of SNR<=3 to NaN, to interpret these as 'bad bands'

[108]:
```
# where SNR <=1 to NaN, to interpret these as 'bad bands'
import numpy as np
enfys_inas_mwir_snr = enfys_inas_mwir_snr.where(enfys_inas_mwir_snr > 3,␣
↪other=np.nan)
```

[109]:
```
enfys_inas_mwir_snr.index.name = 'wavelength'
```

The continuous spectral SNR for the complete instrument is:

[110]:
```
g = enfys_inas_mwir_snr.plot(logy=True)
# set the x and y axes
g.set_xlabel('Wavelength (nm)')
g.set_ylabel('SNR')
title = g.set_title('Enfys InGaAs SWIR & InAs MWIR Merged Sensor SNR')
```

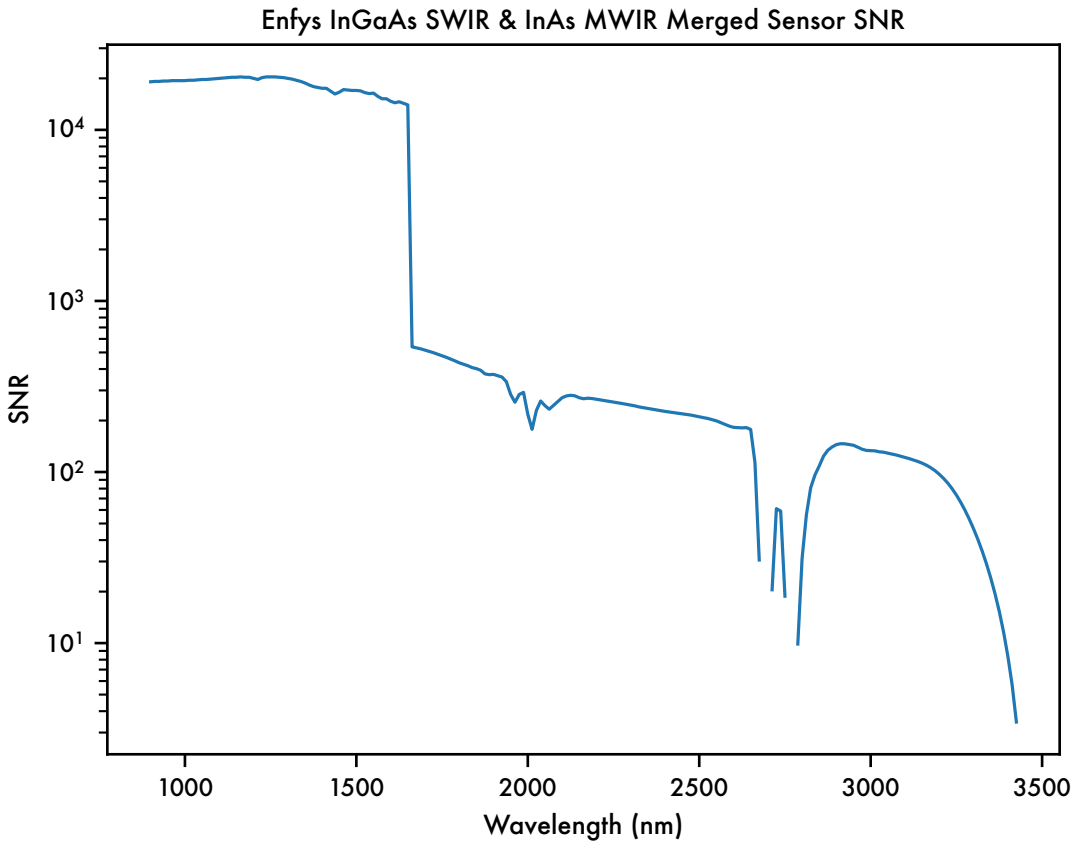


Figure 7 SWIR & MWIR merged spectral Signal-to-Noise Ratio input data, extracted from the current Enfys Science Traceability Matrix, maintained by M. Gunn.

4.1.2 Spectral Resolving Power

Previously we loaded in the latest data for modelling the Spectral Resolving Power for the SWIR and MWIR LVFs. Here we merge the dataset, and prepare it to be passed to the `sptk.Instrument` module.

We merge the datasets, using the SWIR values where the sensor range overlaps.

[111]:
```
# get the max wavelength where ingaas_swir_snr is not nan
ingaas_swir_snr = ingaas_swir_snr[~np.isnan(ingaas_swir_snr)]
max_swir_wavelength = ingaas_swir_snr.index[-1] * 1000 # convert from µm to nm
max_swir_wavelength
# get the enfys_swir_respow data where the wavelength is less than the␣
 ↪max_swir_wavelength
enfys_swir_respow = enfys_swir_respow[enfys_swir_respow.index <␣
 ↪max_swir_wavelength]
enfys_swir_respow
```

```
# get the enfys_mwir_respow where the wavelength is greater than the␣
 ↪max_swir_wavelength
enfys_mwir_respow = enfys_mwir_respow[enfys_mwir_respow.index >␣
 ↪max_swir_wavelength]
enfys_mwir_respow
# now merge the datasets
enfys_respow = pd.concat([enfys_swir_respow, enfys_mwir_respow])
```

The complete instrument Spectral Resolving Power is:

[112]:
```
g=enfys_respow.plot(marker='o',xlabel='Wavelength (nm)', ylabel=r'$\lambda/
 ↪\Delta\lambda$ Spectral Resolving Power')
g.legend(['SWIR & MWIR'])
title = g.set_title('Enfys SWIR & MWIR merged Spectral Resolving Power␣
 ↪$\Gamma[\lambda]$')
```

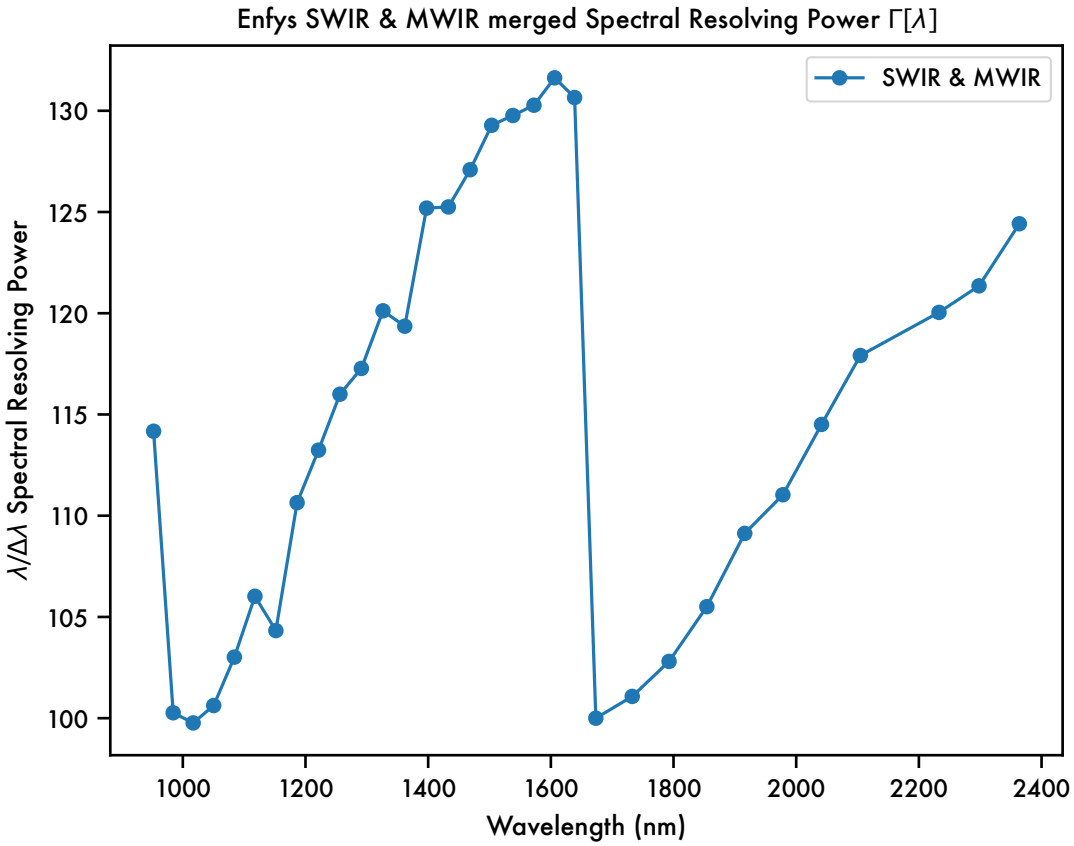


Figure 8 SWIR and MWIR merged Spectral Resolving Power data, as extracted from the `Enfys schedule review 20250807.ppt`, M. Gunn.

We pass this data to the InstrumentBuilder function.

4.1.3 Instrument Build

Now we will pass this to the InstrumentBuilder, defining the spectral range as $\lambda \in [900, 3100]$ nm.

[113]:
```
enfys_builder = InstrumentBuilder(
    instrument_name='enfys_inas_mwir',
    instrument_type='lvf',
    sampling='nyquist',
    resolution = enfys_respow,
    spectral_range=[900, 3101],
    snr = enfys_inas_mwir_snr)
```

```
Exporting instrument to ../data/instruments/enfys_inas_mwir.csv…
```

[114]:
```
from sptk.instrument import Instrument
```

Now we build the transmission profiles for each LVF position, according to the list of CWLs and FWHMs generated above. These are accessed by reading the `.csv` file produced by the `InstrumentBuilder` procedure.

When generating the profiles, we turn the CWL and FWHM into a profile by assuming a distribution function. For Enfys we use the Cauchy distribution, under the guidance of the Enfys build team, which behaves similarly to a Gaussian, but with wider 'wings'. It is parameterised by the full-width-at-half-maximum, as the distribution strictly has no finite variance. We have implemented a `build_cauchy_filter` method for the `Instrument` class.

[115]:
```
enfys = Instrument(
            name = 'enfys_inas_mwir', # filename of the instrument information,␣
 ↪held in /software/data/instruments/
            project_name = matcol.project_name, # the project directory name,␣
 ↪hosting outputs in the /spectral_parameter_studies/ directory
            shape = 'cauchy',
            load_existing=False, # if True, load existing instrument data from␣
 ↪the project directory
            plot_profiles=False, # if true, plot the transmission profiles␣
 ↪during construction
            export_df=True) # if True, export the instrument transmission␣
 ↪profiles to the project directory
```

```
Building Instrument…
Building new DataFrame for 'enfys_inas_mwir'…
Exporting the Instrument to CSV and Pickle formats…
```

[116]:
```
fig ,ax = enfys.plot_instrument_characteristics()
```

```
Plotting Instrument Transmission…
```

```
/opt/homebrew/Caskroom/miniconda/base/envs/enfys_sptk/lib/python3.10/site-
packages/seaborn/_oldcore.py:200: FutureWarning: elementwise comparison failed;
returning scalar instead, but in the future will perform elementwise comparison
  if palette in QUAL_PALETTES:
```

```
Plotting Maximum Signal-to-Noise Ratio…
Plotting FWHM…
Plotting Spectral Resolution…
Plots exported to ../projects/enfys_mica_inas_mwir/instrument
```

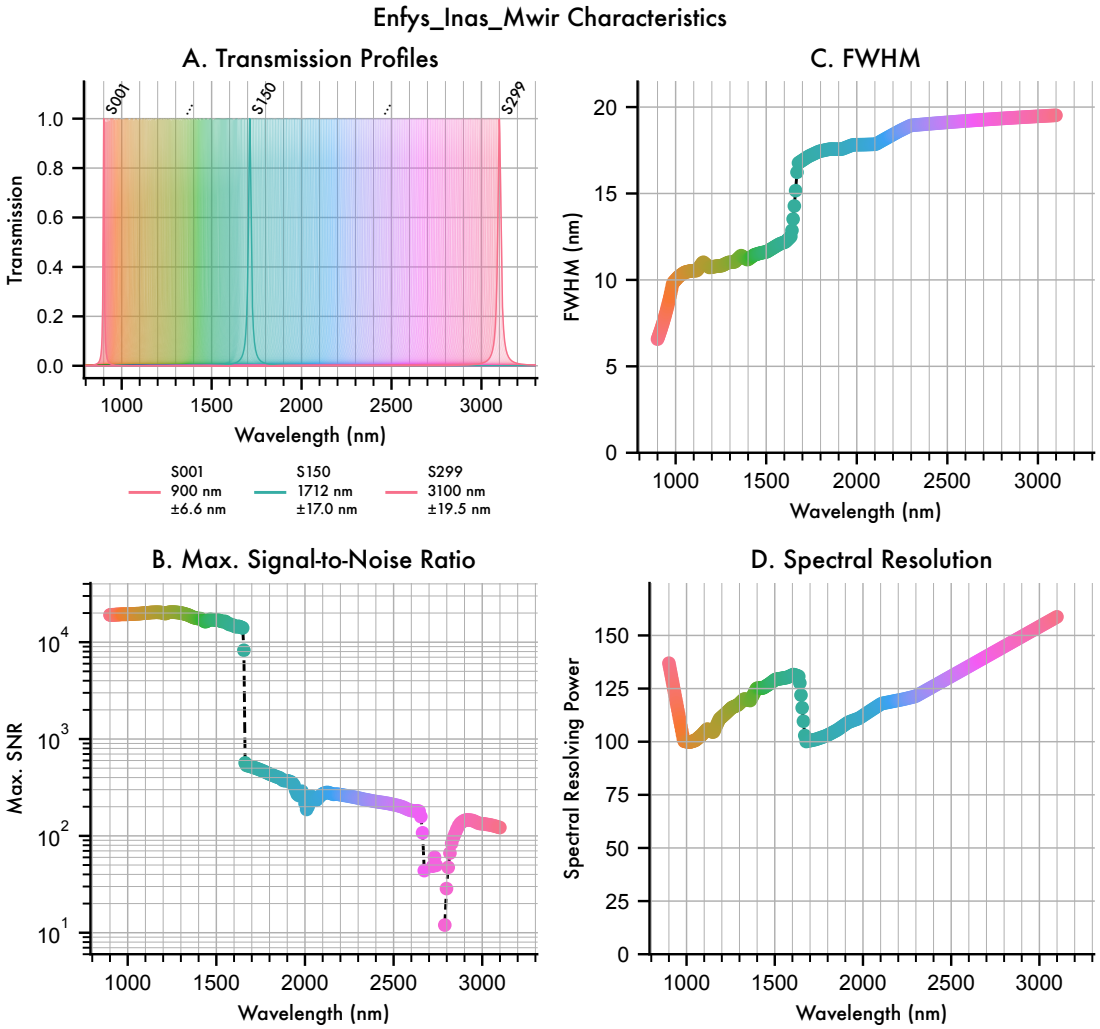


Figure 9 Simulated spectral response and resolution of *Enfys*, based on preliminary expectations of the spectral and radiometric response.

The data shown in figure 9 approximates the expected performance of *Enfys*. This data represents the current component level understanding of the filter and sensor performance of the built instrument.

5 Results

5.1 Sampling MICA with Enfys

We now make an `Observation` object by sampling the MICA files `MaterialCollection` with the Enfys `Instrument`.

[117]:
```
from sptk.observation import Observation
```

[118]:
```
obs = Observation(
    material_collection=matcol,
    instrument = enfys,
    load_existing=False,
    plot_profiles=False,
    export_df=True)
```

```
Building new Observation DataFrame
for 'enfys_mica_inas_mwir'…
Sampling the Material Collection with the Instrument…
Sampling complete.
Exporting the Observation Pickle format…
Observation export complete.
```

We can plot the sampled spectra against the input high-spectra below:

[119]:
```
axes = obs.plot_profiles(scope='categories', groupby='Subgroup', stacked=True,␣
↪hires_under=True)
```

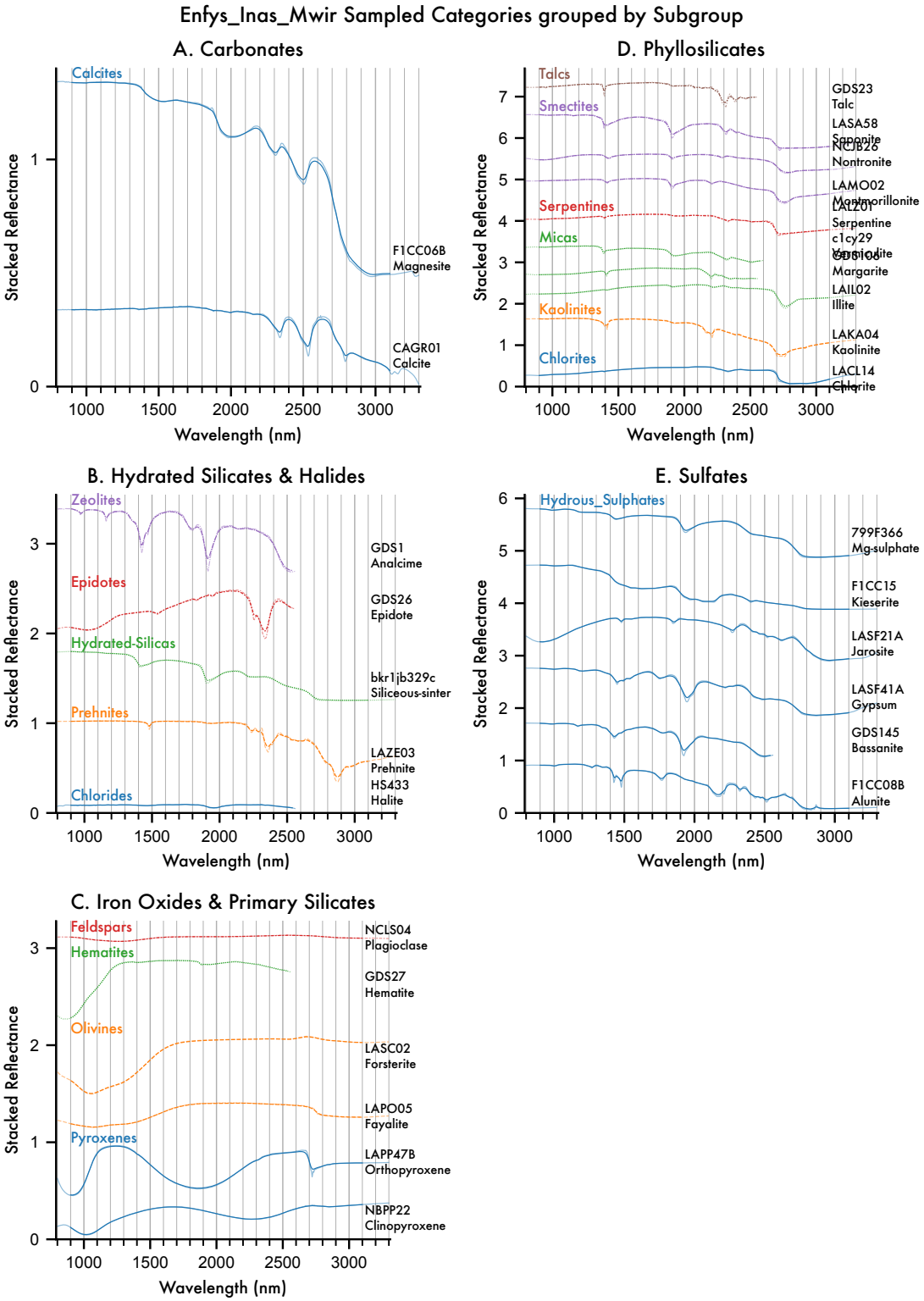


Figure 10 The MICA files as sampled by *Enfys*, with the original high-resolution spectra plotted

underneath each.

We can now see where there are deviations of the *Enfys* sampled spectra from the high-resolution source spectra.

For example, illite, plotted below for clarity.

[120]:
```
axes = obs.plot_profiles(scope='illite', groupby='Sample ID', stacked=False,␣
↪hires_under=True)
```

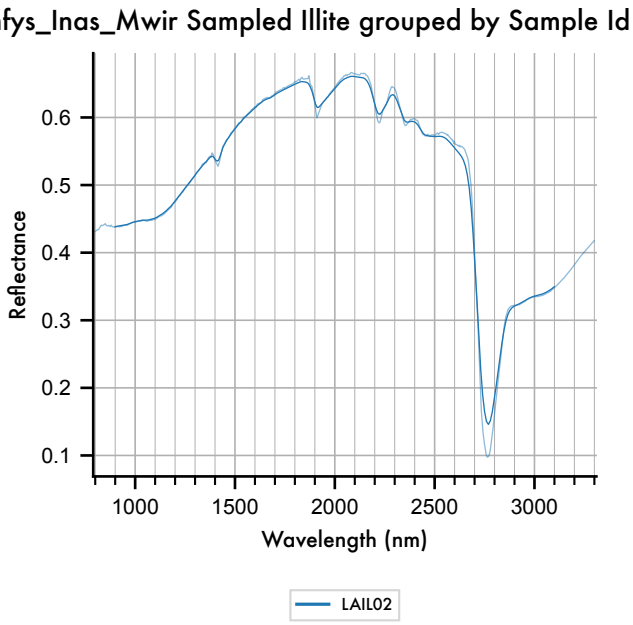


Figure 11 Deviation of the *Enfys* sampled spectrum from the high-resolution input spectrum for Illite.

We note that there is an effect by which peaks and troughs in the spectra are reduced. This is due to the long tails of the Cauchy filter profiles. Despite the reduction of absolute band depth, the features of Illite (fig. 8) are still resolvable.

We can produce a continuum-removed waterfall plot to check that the major bands are resolved.

[121]:
```
from sptk.spectral_library_analyser import SpectralLibraryAnalyser as sla

obs_cr = sla(obs).remove_continuum()
```

[122]:
```
axes = obs_cr.plot_profiles(waterfall=True, scope='all', groupby='Category')
```

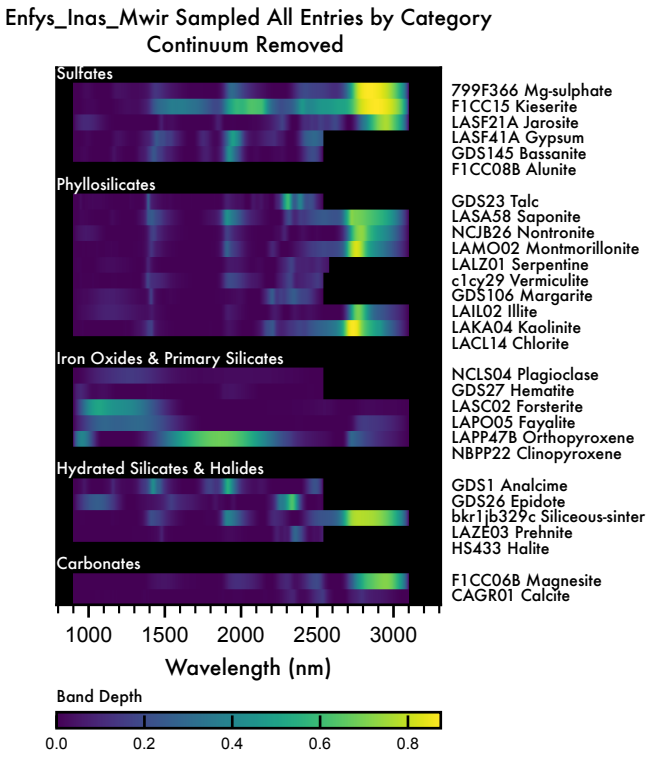


Figure 12 Continuum-Removed waterfall plot of the MICA files as sampled by *Enfys*.

Comparing Figure 12 to Figure 6, we can see that the weaker sub-2 µm bands are less well resolved.

We can save this data

[123]:
```
obs_cr.export_main_df()
```

```
Exporting the Observation Pickle format…
Observation export complete.
```

5.2 Adding Synthetic Noise

We have the capability to add synthetic noise to the dataset, by redrawing a sample from the dataset with noise added according to the signal-to-noise ratio specified for the given channel.

We can optionally redraw multiple noisy samples for each entry, to give an indication of the statistical uncertainty, or for example draw a single noisy sample, to produce a single-observation mission-realistic dataset.

First, let's draw a single noisy entry:

[124]:
```
noise_obs = obs.add_noise(1, seed=0)
```

[125]:
```
axes = obs.plot_profiles(scope='illite', groupby='Sample ID', stacked=False,␣
↪hires_under=True, ci=False)
```

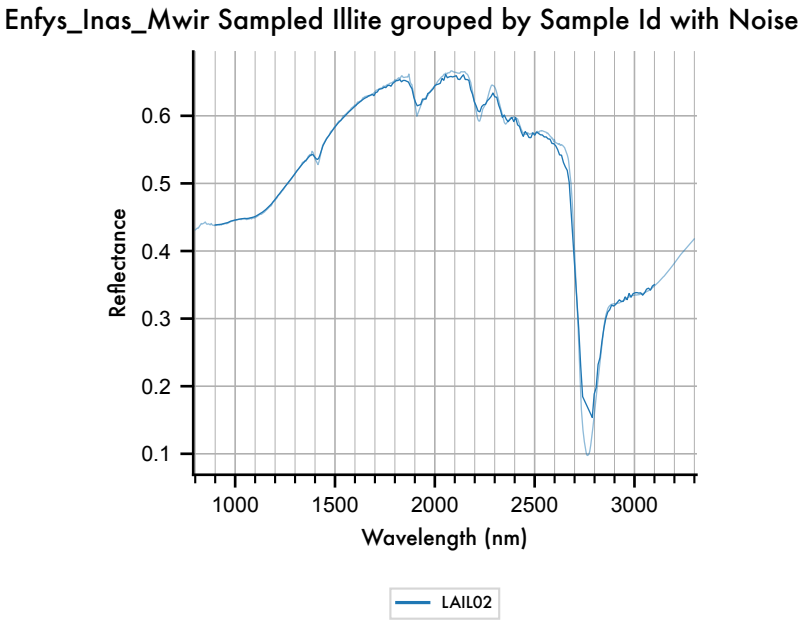
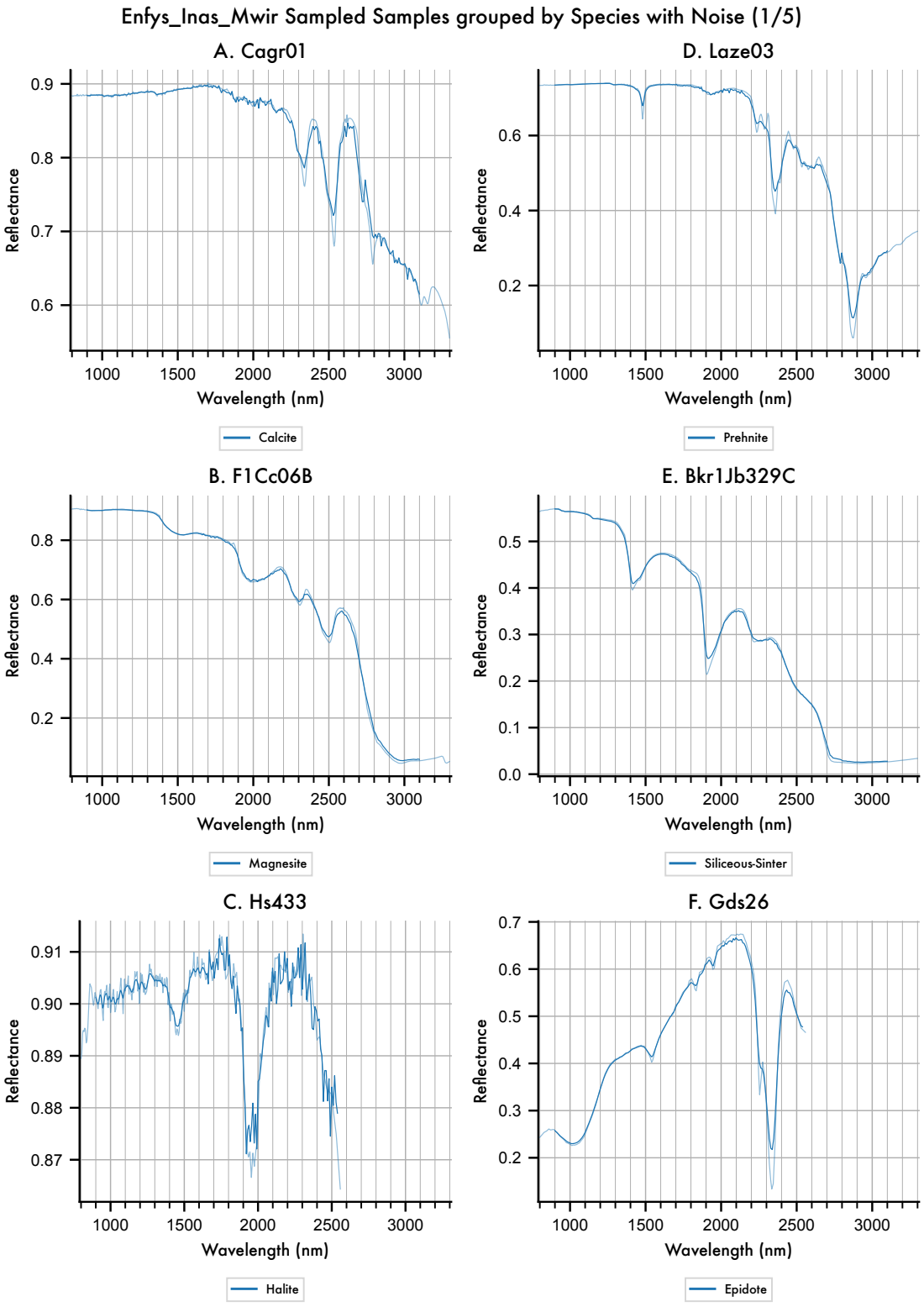
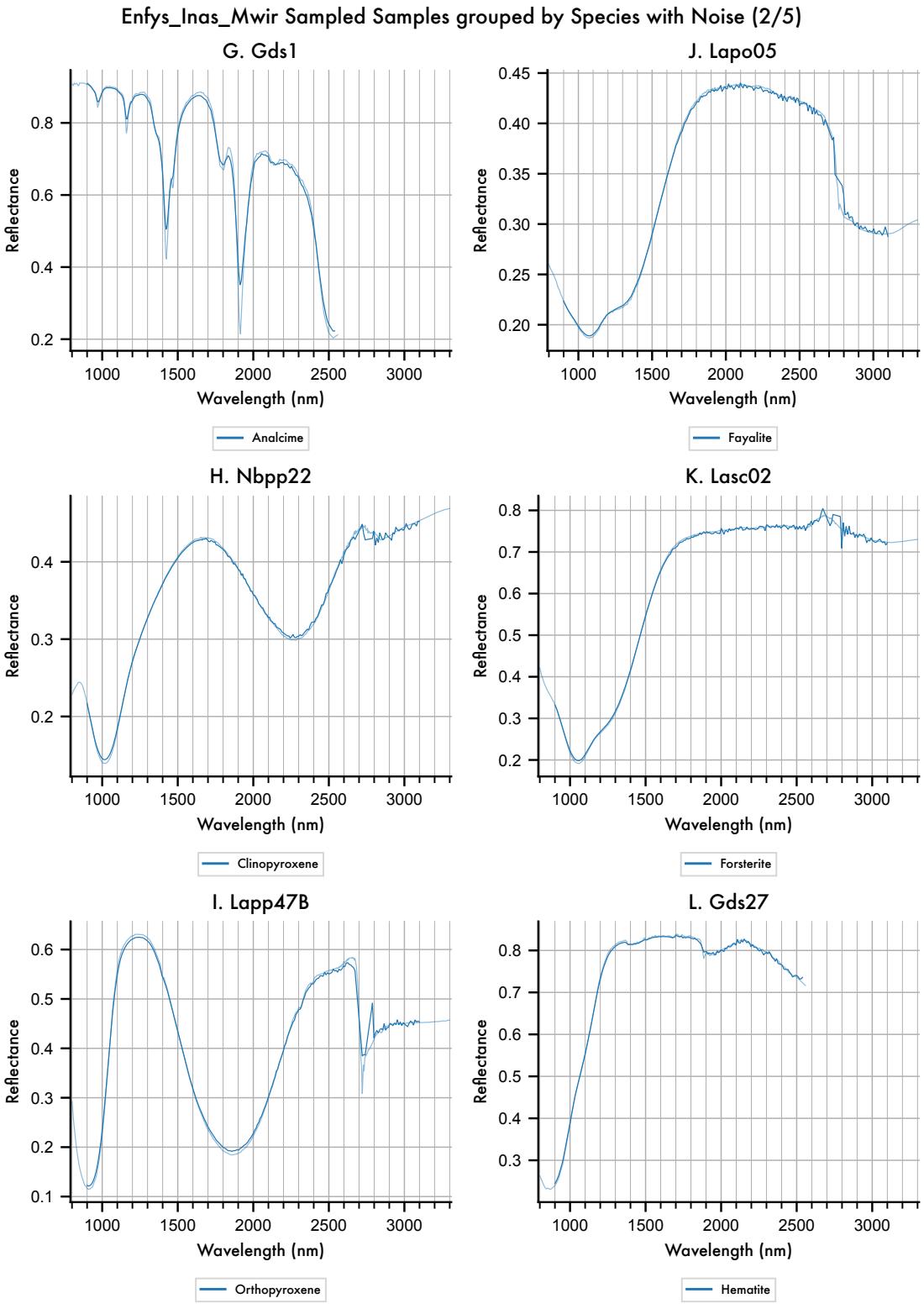


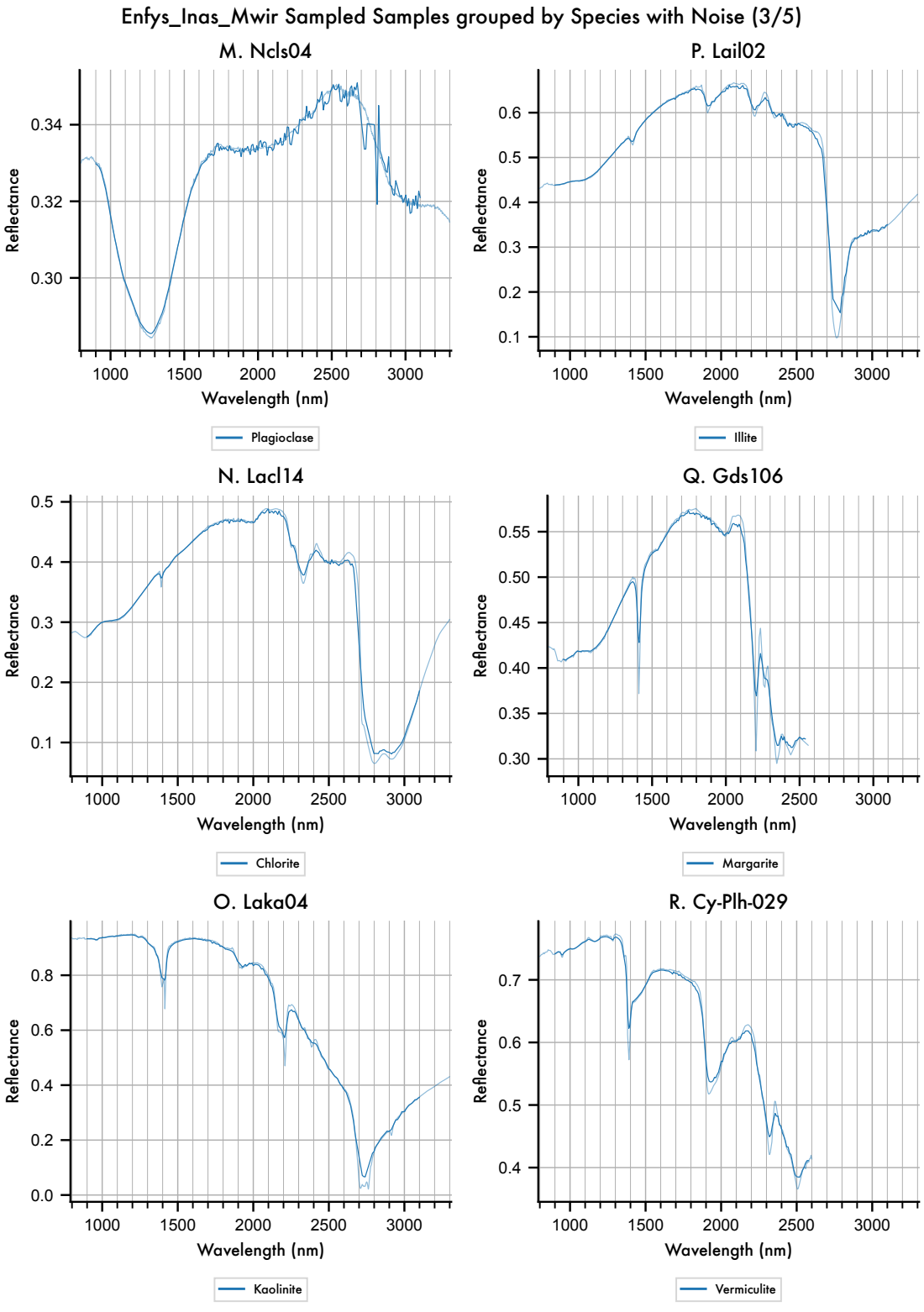
Figure 13 Noisy resampled example spectrum for Illite, showing 1 noisy sample according to the spectral signal-to-noise ratio illustrated in figure 9.B.

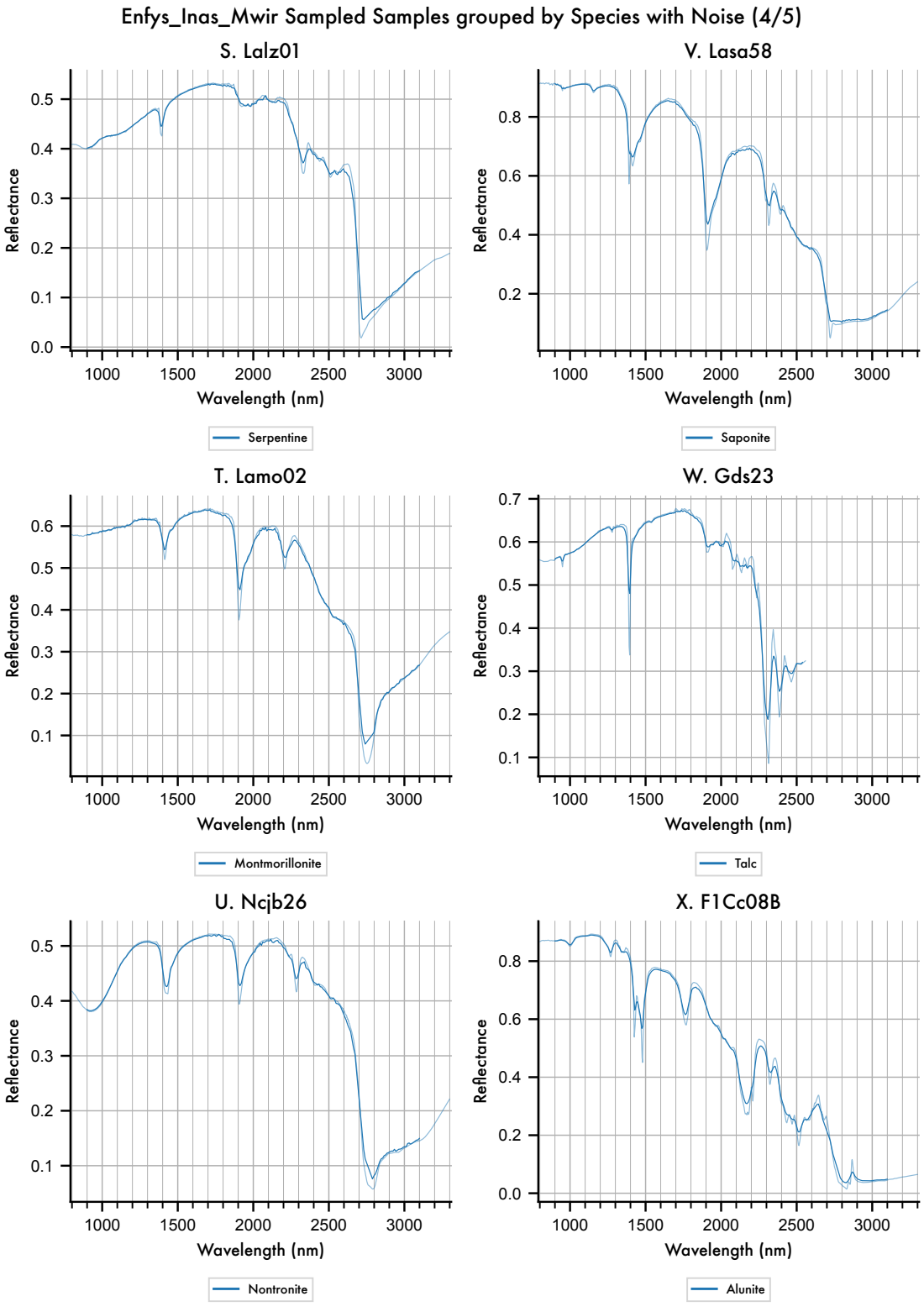
We can illustrate this in greater deal by plotting each entry on a separate figure, with the y-axis clipped to the spectral range of entry. Note that for samples with a small spectral range (i.e. samples that are spectrally flat), this stretching will appear to amplify noise, so the y-axis range must be considered when assessing the noise.

[126]:
```
axes = obs.plot_profiles(scope='samples', groupby='Species', stacked=False,␣
↪hires_under=True, ci=False)
```


Enfys_Inas_Mwir Sampled Samples grouped by Species with Noise (1/5)
A. Cagr01
D. Laze03
B. F1Cc06B
E. Bkr1Jb329C
C. Hs433
F. Gds26
Reflectance
Wavelength (nm)
Calcite
Prehnite
Magnesite
Siliceous-Sinter
Halite
Epidote


Enfys_Inas_Mwir Sampled Samples grouped by Species with Noise (2/5)
G. Gds1
J. Lapo05
H. Nbpp22
K. Lasc02
I. Lapp47B
L. Gds27
Reflectance
Wavelength (nm)
Analcime
Fayalite
Clinopyroxene
Forsterite
Orthopyroxene
Hematite


Enfys_Inas_Mwir Sampled Samples grouped by Species with Noise (3/5)
M. Ncls04
P. Lail02
N. Lacl14
Q. Gds106
O. Laka04
R. Cy-Plh-029
Reflectance
Wavelength (nm)
Plagioclase
Illite
Chlorite
Margarite
Kaolinite
Vermiculite


Enfys_Inas_Mwir Sampled Samples grouped by Species with Noise (4/5)
S. Lalz01
V. Lasa58
T. Lamo02
W. Gds23
U. Ncjb26
X. F1Cc08B
Reflectance
Wavelength (nm)
Serpentine
Saponite
Montmorillonite
Talc
Nontronite
Alunite

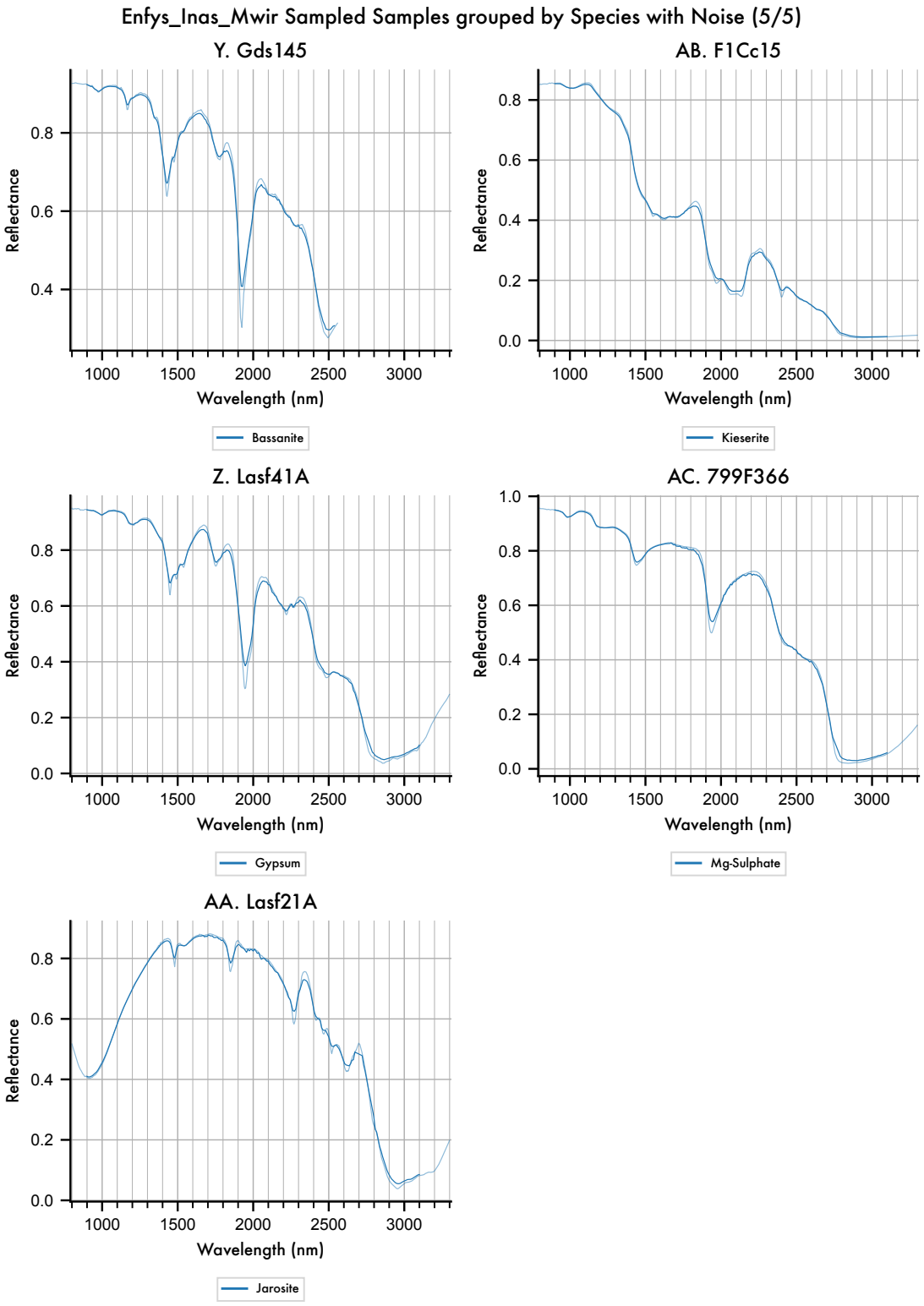


Figure 14 Detailled view of each entry in the spectral library, showing the scale of noise introduced,

relative to the features of each spectrum, and the reflectance range.

Qualitatively, these plots illustrate that *Enfys* does not significantly intrude on the diagnostic features of these type specimen of minerals identified on the Mars surface. In some cases, we can see that doublet features are lost, but this is due to the spectral resolving power, rather than the additive noise. We will assess this in the discussion section.

We can export the dataset:

[127]:
```
obs.export_main_df()
```

```
Exporting the Observation Pickle format…
Observation export complete.
```

5.3 Continuum Removal

We can apply continuum removal, even to the noisy data.

[128]:
```
from sptk.spectral_library_analyser import SpectralLibraryAnalyser as sla

cr_tool = sla(obs)
cr_obs = cr_tool.remove_continuum()
```

[129]:
```
axes = cr_obs.plot_profiles(scope='categories', groupby='Subgroup',␣
 ↪stacked=True, hires_under=False, ci=False)
```

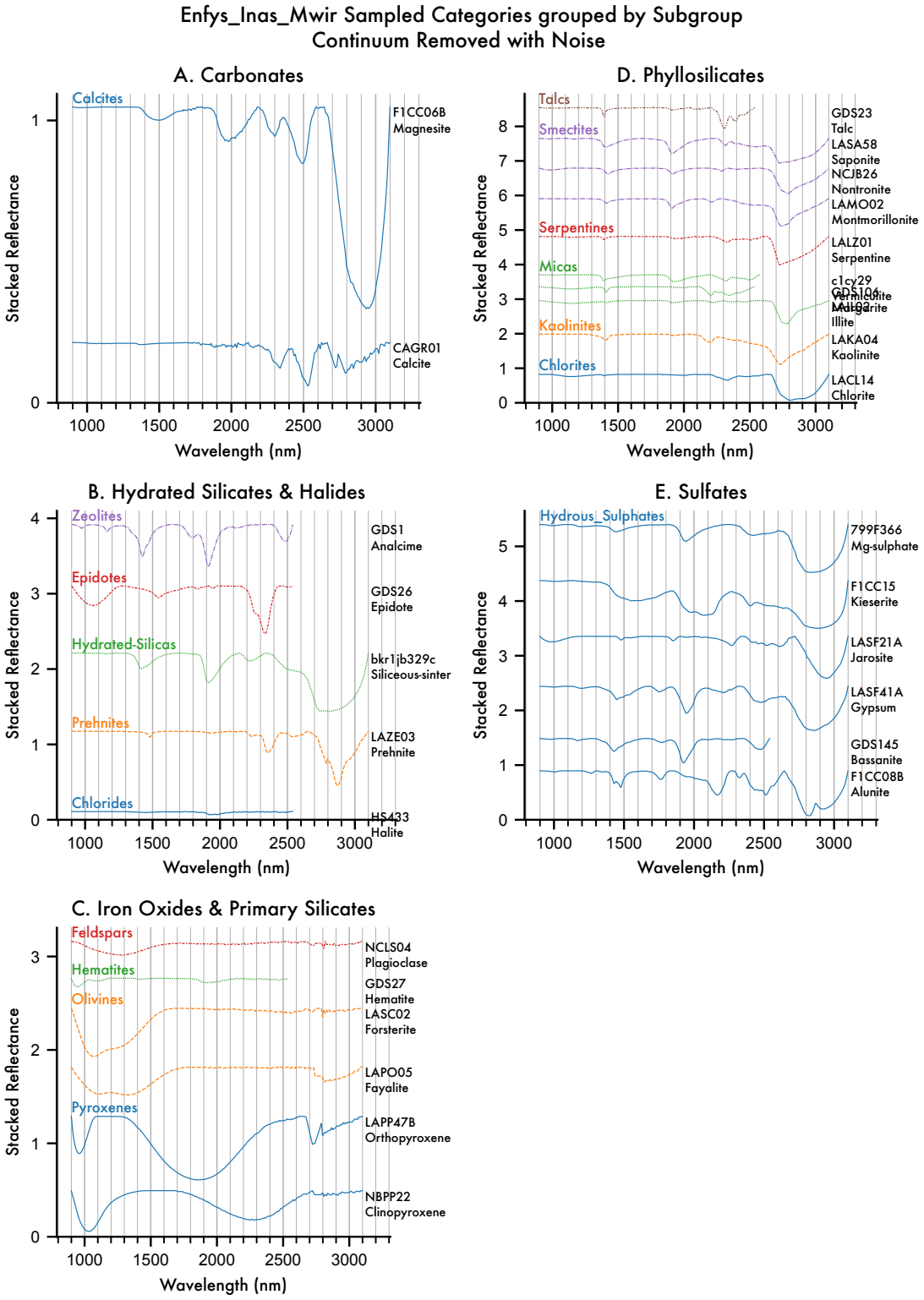


Figure 15 Continuum-Removed entries of the MICA Files, as sampled by *Enfys*, including syn-

thetic noise.

Again, we can export this:

[130]:
```
cr_obs.export_main_df()
```

```
Exporting the Observation Pickle format…
Observation export complete.
```

5.4 Repeat Noisy Samples: Std. Dev. Confidence Intervals, Continuum-Removal, and Waterfall Visualisation

5.4.1 Repeat Noisy Samples

There are several visualisations that we can produce that demonstrate the average signal over multiple repeatedly drawn noisy spectra. We can simualte this by setting the number of repeats to a larger number, e.g. 64.

[131]:
```
_ = obs.add_noise(64, seed=0)
```

Now, we can plot the average spectrum for each entry, over the repeat noisy samples, with shading indicating the standard deviation of the noisy data, where the 'ci' keywrod has been applied ('ci': confidence-interval).

[132]:
```
axes = obs.plot_profiles(scope='illite', groupby='Sample ID', stacked=False,␣
↪hires_under=True, ci=True)
```

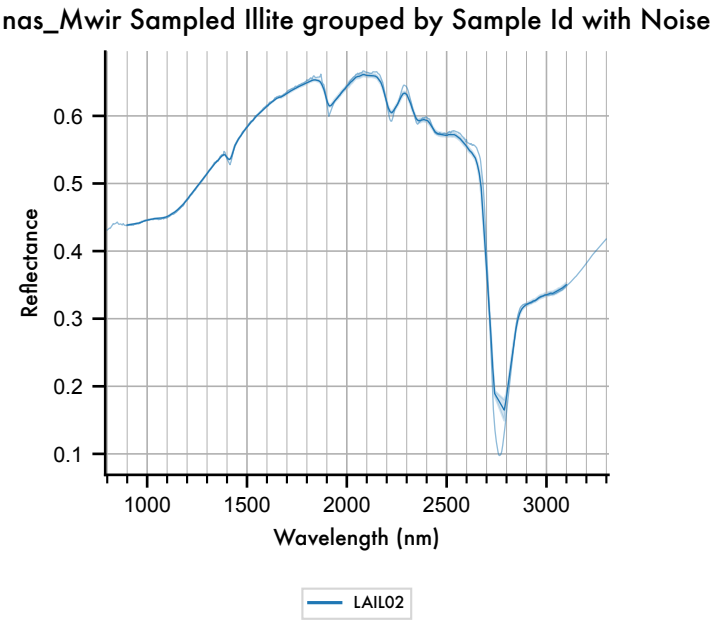


Figure 16 Mean of Noisy resampled example spectrum for Illite, showing mean and standard deviation of 64 repeat samples according to the spectral signal-to-noise ratio illustrated in figure

3.B.

Here we find that the standard deviation is almost negligible at the scale of the figure.

Plotting out all of the groups, we again see that the noise, at this scale, is indeed negligible for most entries.

[133]:
```
axes = obs.plot_profiles(scope='categories', groupby='Subgroup', stacked=True,␣
↪hires_under=True, ci=True)
```

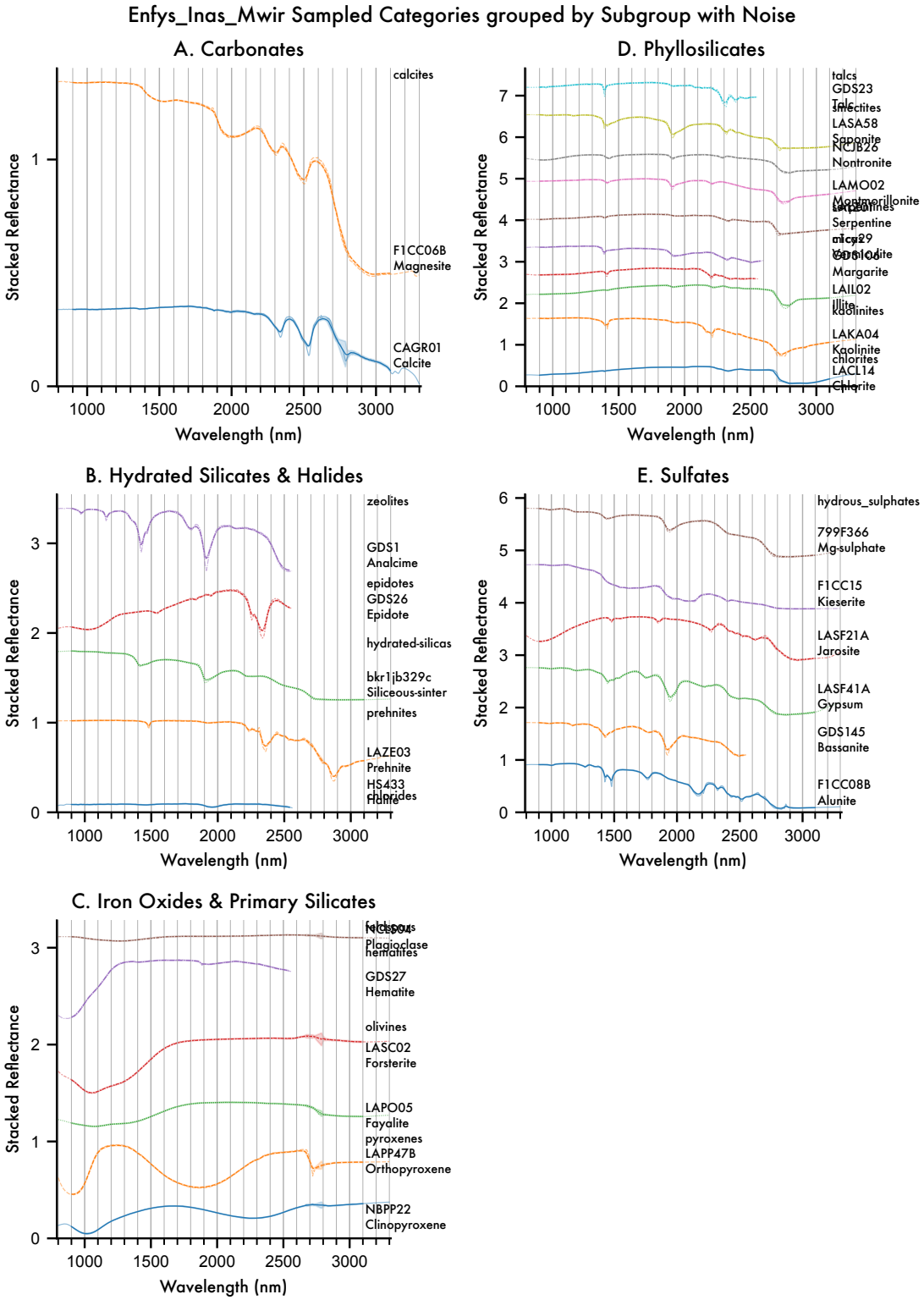


Figure 17 The MICA files as sampled by *Enfys*, with repeat samples drawn according to the ex-

pected spectral Signal-to-Noise Ratio, with the original high-resolution spectra plotted underneath each.

The key conclusion to draw from the above figure is that expected noise for *Enfys* does not significantly intrude on the diagnostic features of these type specimen of minerals identified on the Mars surface.

To visualise the impact on the resolution of absorption features, we can show the continuum removed data. We need to recompute the continuum-removed data on this noisy set.

[134]:
```
cr_tool = sla(obs)
cr_obs = cr_tool.remove_continuum()
```

Showing the data in the waterfall visualisation, each repeat noisy sample is shown as a sub-row for each entry.

[135]:
```
axes = cr_obs.plot_profiles(scope='all', groupby='Category', stacked=False,␣
↪waterfall=True, ci=False)
```

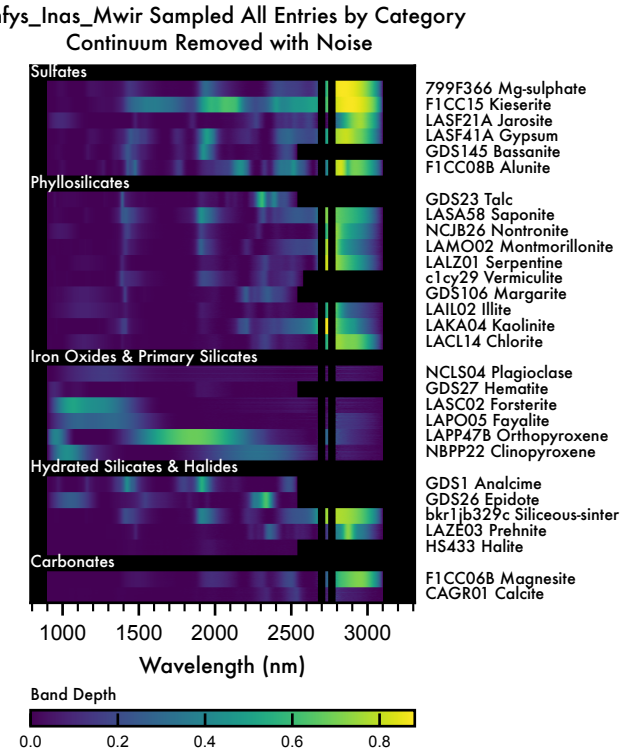
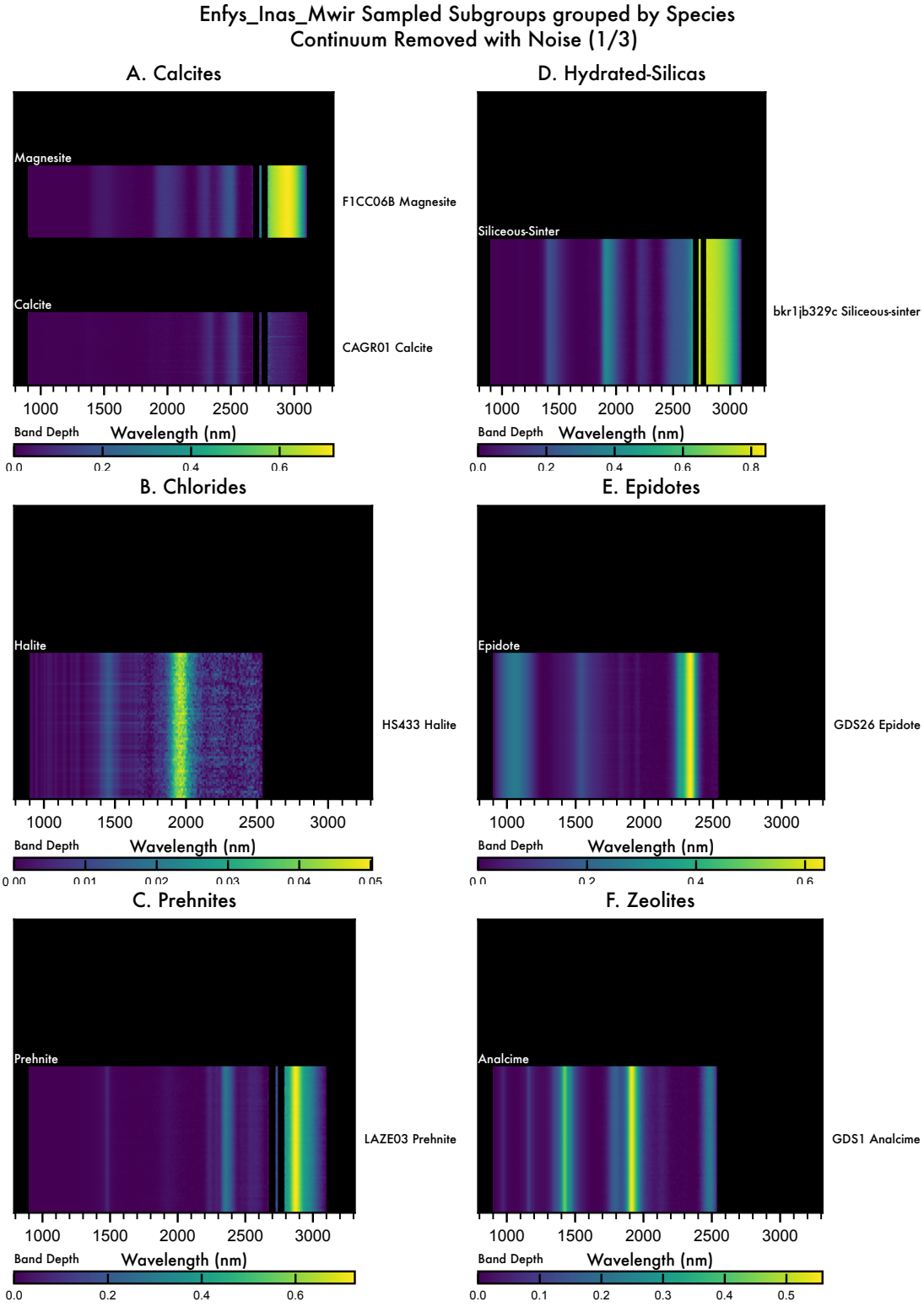
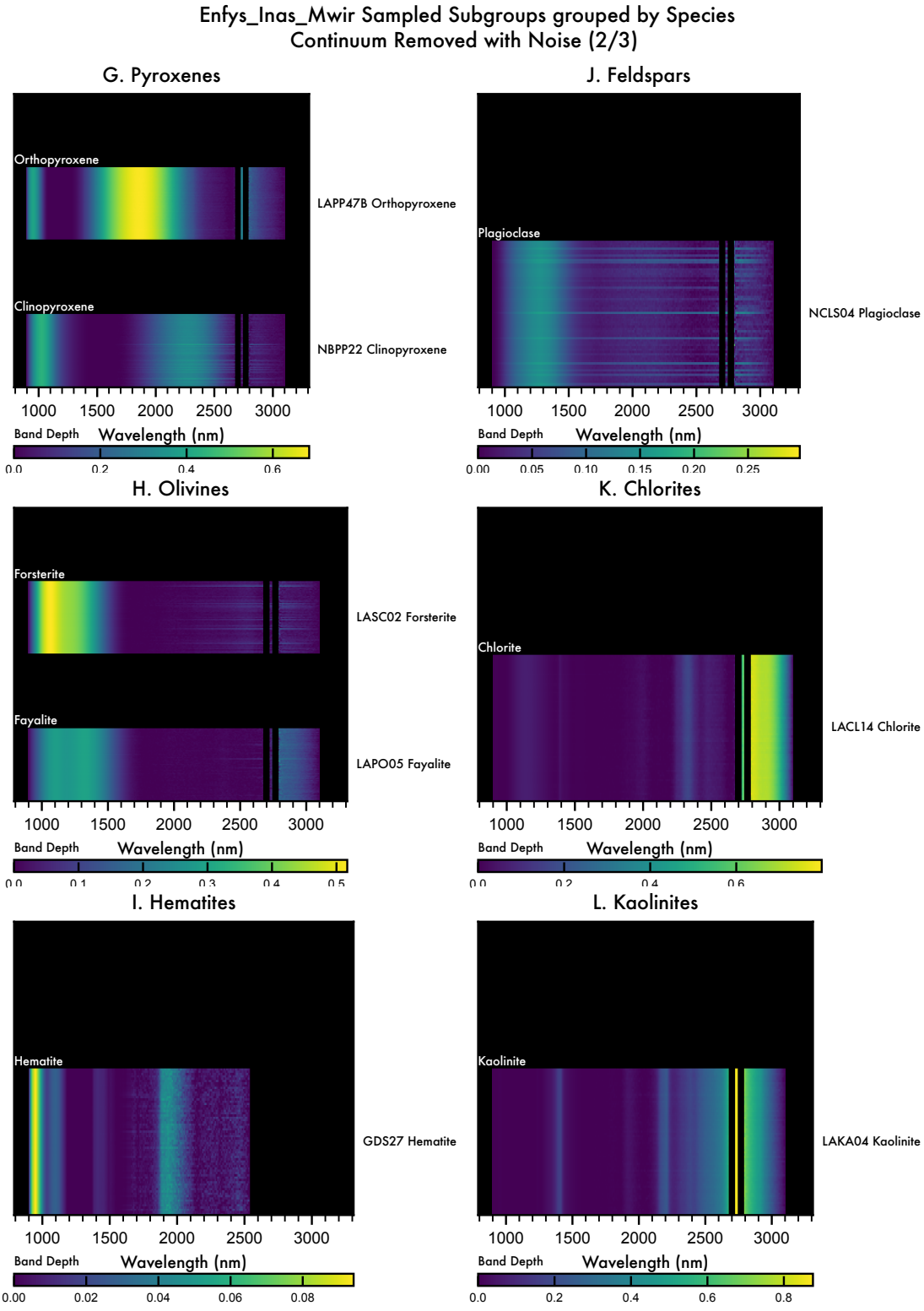


Figure 16 Continuum-Removed waterfall diagram of the entries of the MICA Files, as sampled by *Enfys*, including synthetic noise. Each entry bar is composed of n repeat noisy samples, such that each bar is composed of n rows, illustrating the noise.

We can see this clearer by plotting each entry independently.

[136]:
```
axes = cr_obs.plot_profiles(scope='subgroups', groupby='Species',␣
↪stacked=False, waterfall=True, ci=False)
```




Enfys_Inas_Mwir Sampled Subgroups grouped by Species
Continuum Removed with Noise (2/3)
G. Pyroxenes
Orthopyroxene
LAPP47B Orthopyroxene
Clinopyroxene
NBPP22 Clinopyroxene
1000 1500 2000 2500 3000
Band Depth
Wavelength (nm)
0.0 0.2 0.4 0.6
J. Feldspars
Plagioclase
NCLS04 Plagioclase
1000 1500 2000 2500 3000
Band Depth
Wavelength (nm)
0.00 0.05 0.10 0.15 0.20 0.25
H. Olivines
Forsterite
LASC02 Forsterite
Fayalite
LAPO05 Fayalite
1000 1500 2000 2500 3000
Band Depth
Wavelength (nm)
0.0 0.1 0.2 0.3 0.4 0.5
K. Chlorites
Chlorite
LACL14 Chlorite
1000 1500 2000 2500 3000
Band Depth
Wavelength (nm)
0.0 0.2 0.4 0.6
I. Hematites
Hematite
GDS27 Hematite
1000 1500 2000 2500 3000
Band Depth
Wavelength (nm)
0.00 0.02 0.04 0.06 0.08
L. Kaolinites
Kaolinite
LAKA04 Kaolinite
1000 1500 2000 2500 3000
Band Depth
Wavelength (nm)
0.0 0.2 0.4 0.6 0.8

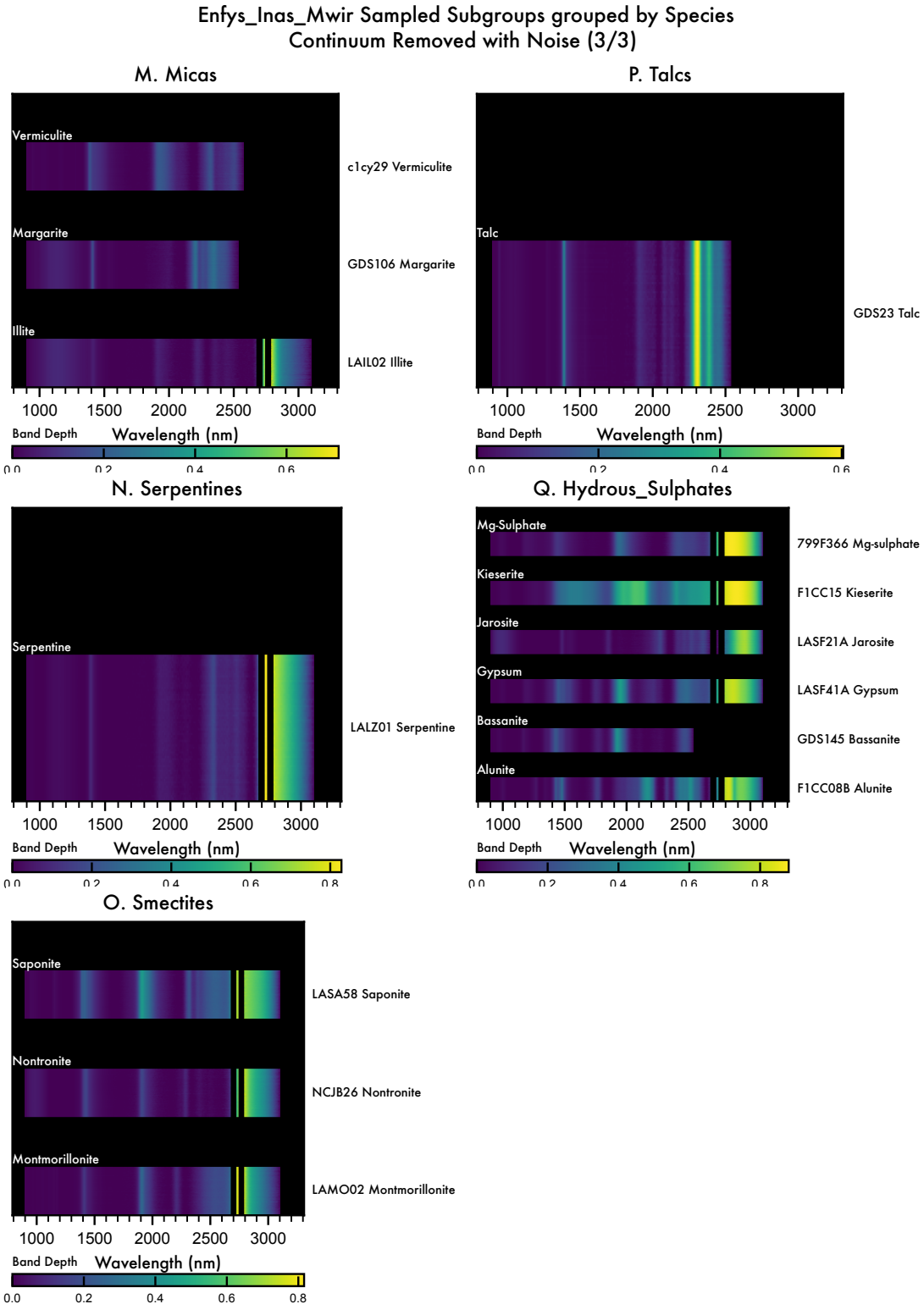

Enfys_Inas_Mwir Sampled Subgroups grouped by Species
Continuum Removed with Noise (3/3)
M. Micas
Vermiculite
c1cy29 Vermiculite
Margarite
GDS106 Margarite
Illite
LAIL02 Illite
1000 1500 2000 2500 3000
Band Depth
Wavelength (nm)
0.0 0.2 0.4 0.6
P. Talcs
Talc
GDS23 Talc
1000 1500 2000 2500 3000
Band Depth
Wavelength (nm)
0.0 0.2 0.4 0.6
N. Serpentines
Serpentine
LALZ01 Serpentine
1000 1500 2000 2500 3000
Band Depth
Wavelength (nm)
0.0 0.2 0.4 0.6 0.8
Q. Hydrous_Sulphates
Mg-Sulphate
799F366 Mg-sulphate
Kieserite
F1CC15 Kieserite
Jarosite
LASF21A Jarosite
Gypsum
LASF41A Gypsum
Bassanite
GDS145 Bassanite
Alunite
F1CC08B Alunite
1000 1500 2000 2500 3000
Band Depth
Wavelength (nm)
0.0 0.2 0.4 0.6 0.8
O. Smectites
Saponite
LASA58 Saponite
Nontronite
NCJB26 Nontronite
Montmorillonite
LAMO02 Montmorillonite
1000 1500 2000 2500 3000
Band Depth
Wavelength (nm)
0.0 0.2 0.4 0.6 0.8

Figure 17 Detailed waterfall view of each entry in the spectral library, showing the scale of noise introduced, relative to the features of each spectrum, and the reflectance range. Each entry bar is composed of n repeat noisy samples, such that each bar is composed of n rows, illustrating the noise.

This visualisations more clearly show how absorption features are resolved, and how noise impacts the different absorption features.

5.4.2 Validating the Noise

We can check that our noise is correct by aggregating the repeat samples to give the mean and standard deviation for each entry and each spectral channel (wavelength).

We can then compare this to the spectral SNR.

[137]:
```
signal = obs.main_df.groupby('Root Data ID').mean()
noise = obs.main_df.groupby('Root Data ID').std()
snr = signal/noise
```

```
/var/folders/w4/44k32f413dd82lxmtw9456w80000gp/T/ipykernel_54287/2651352844.py:1
: FutureWarning: The default value of numeric_only in DataFrameGroupBy.mean is
deprecated. In a future version, numeric_only will default to False. Either
specify numeric_only or select only columns which should be valid for the
function.
  signal = obs.main_df.groupby('Root Data ID').mean()
/var/folders/w4/44k32f413dd82lxmtw9456w80000gp/T/ipykernel_54287/2651352844.py:2
: FutureWarning: The default value of numeric_only in DataFrameGroupBy.std is
deprecated. In a future version, numeric_only will default to False. Either
specify numeric_only or select only columns which should be valid for the
function.
  noise = obs.main_df.groupby('Root Data ID').std()
```

[138]:
```
# plot the SNR
import seaborn as sns
# plot with log scale y axis
g = sns.lineplot(data=snr.mean(), label='Mean/Standard Deviation of Sampled␣
 ↪Noisy Data', markers=True)
g.set(yscale="log")


# get instrument snr
sns.lineplot(data=obs.instrument.main_df, x='cwl', y='snr', label='InGaAs +␣
 ↪InAs SNR', linestyle='--', markers=True)

g.set_xlabel('Wavelength (nm)')
g.set_ylabel('SNR')
title = g.set_title('Recovered Mean/Standard Deviation of Noisy Sampled Data vs␣
 ↪Instrument SNR')
```

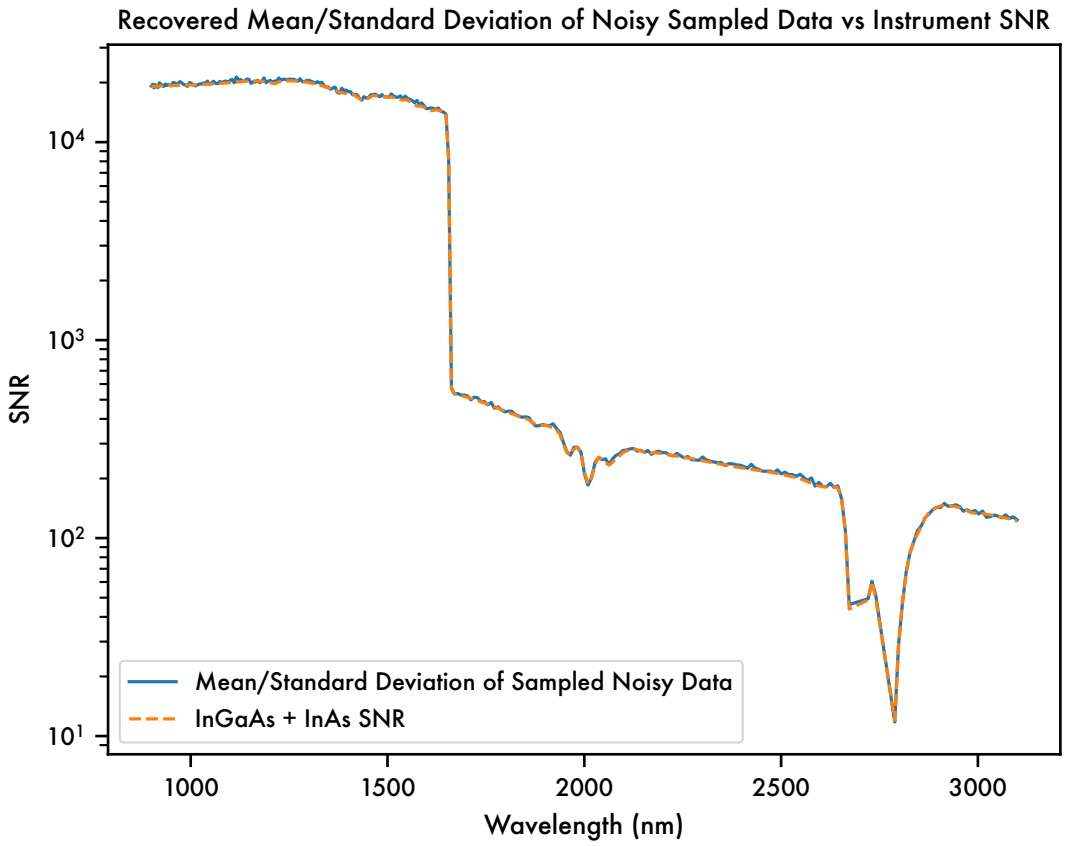


Figure 18 Recovery of the instrument specified spectral Signal-to-Noise Ratio from the averaged repeat noisy samples over the standard deviation.

We can see that our method for simulating noise by drawing additive noise from the distribution according to noise defined by the signal Reflectance and the SNR does indeed recover the input spectral Signal-to-Noise Ratio.

6 Discriminatory Evaluation with Spectral Angle Mapper

To evaluate the ability of Enfys to discriminate between the materials of the MICA files set, we will use the Spectral Angle to measure the distance between a given noisy spectrum and each clean entry of the MICA files. We expect the smallest Spectral Angle to occur for the matching material.

By running the evaluation on N-repeat noisy samples, we can estimate the propagation of the noise through the Spectral Angle. We can then compare this noise to the difference in Spectral Angle between the match and the next-nearest match, to understand the repeatability.

The Spectral Angle computation is closed-form, and can be formulated in a Linear Algebra representation that scales well with increasing sizes of spectral libraries.

6.1 Spectral Angle Evaluation

Inputs: - Test material name - Test material noisy data - MICA files clean data

Outputs: - Spectral Angle for each MICA entry, and for each repeat noisy sample, order in increasing angle - Plot of Spectral Angle results, showing score for each material, and the noisy spectrum vs the top 3 matches - Plot of mean Spectral Angle result, with errorbars, to show if final material detection is statistically significant

First, let's get a new noisy dataset with 128 repeat noisy samples for each material.

[139]:
```
noise_obs = obs.add_noise(128, seed=0)
```

Now we take a material from the data set, and get the noisy entries for that material, and the clean dataset for the entire MICA files.

[140]:
```
# getting the material entry to test
materials = obs.main_df['Root Data ID'].unique()
test_material = materials[0]
test_material
```

[140]:
```
'CAGR01 Calcite'
```

Getting the noisy dataset for this material:

[141]:
```
noisy_data = obs.get_refl_df(root_data_id=test_material).to_numpy()
noisy_data.shape
```

[141]:
```
(128, 299)
```

We need to define the 'bad bands' in the dataset, i.e. where the reflectance is NaN, and set these to 0, so that they don't contribute to the summations in the Spectral Angle.

[142]:
```
# find the Bad bands
bad_bands = ~np.isfinite(noisy_data[0,:])
noisy_data = np.where(bad_bands, 0, noisy_data) # set all nan values to 0
```

getting the clean data for all materials in the MICA set:

[143]:
```
clean_data = obs.noiseless_df[obs.wvls].to_numpy()
clean_data.shape
```

[143]:
```
(29, 299)
```

Similarly, we take the previously defined 'bad bands', from the noisy dataset, and then also set any missing values in the clean dataset (e.g., where a spectrum is only defined up to 2.5 µm), and set these values to 0 also.

[144]:
```
# get clean data
clean_data = obs.noiseless_df[obs.wvls].to_numpy()
clean_data = np.where(bad_bands, 0, clean_data) # 0 the clean data where the␣
 ↪noisy data is NaN
```

```
clean_data = np.where(~np.isfinite(clean_data), 0, clean_data) # 0 the clean␣
↪data where the clean data is NaN
```

The Spectral Angle is defined between the noisy spectrum $\tilde{R}_i[\lambda]$ of material i in the set of MICA file materials ($\mathcal{M}$) and the clean spectrum $R_j[\lambda]$, where $j \in \mathcal{M}$.

The Spectral Angle θ is:

$$\theta(i,j) = \arccos\left(\frac{\sum_\lambda \tilde{R}_i[\lambda] R_j[\lambda]}{\left(\sum_\lambda \tilde{R}_i[\lambda]^2\right)^{1/2}\left(\sum_\lambda R_j[\lambda]^2\right)^{1/2}}\right)$$

We can formulate the numerator $\sum_\lambda \tilde{R}_i[\lambda] R_j[\lambda]$ as

[145]:
```
numerator = np.inner(noisy_data, clean_data)
numerator.shape
```

[145]:
```
(128, 29)
```

and the denominator parts as:

[146]:
```
denominator = np.outer(np.sqrt(np.sum(noisy_data**2, axis=1)), np.sqrt(np.
↪sum(clean_data**2, axis=1)))
denominator.shape
```

[146]:
```
(128, 29)
```

we can then combine these to compute the Spectral Angle as:

[147]:
```
sa = np.arccos(numerator / denominator)
sa.shape
```

[147]:
```
(128, 29)
```

The Spectral Angle numpy array that this produces has dimensions of (n-repeats, m-entries).

We can package this into a DataFrame:

[148]:
```
sa_df = pd.DataFrame(sa, index=obs.get_refl_df(root_data_id=test_material).
↪index, columns=obs.noiseless_df.index)
```

To visualise, first let's take the average Spectral Angle across all of the observations, and also compute the standard deviation.

[149]:
```
# Now let's order the sa_df DataFrame columns in ascending mean value order
sa_df = sa_df.loc[:, sa_df.mean(axis=0).sort_values().index]
```

[150]:
```
sa_ave = sa_df.mean(axis=0)
sa_std = sa_df.std(axis=0)
```

Now let's order the sa_df DataFrame columns in ascending mean value order

[151]:
```
sa_ave.sort_values().index
```

[151]:
```
Index(['CAGR01 Calcite', 'NCLS04 Plagioclase', 'LAIL02 Illite',
       'LAZE03 Prehnite', 'LAMO02 Montmorillonite', 'NCJB26 Nontronite',
       'LALZ01 Serpentine', 'LACL14 Chlorite', 'LAKA04 Kaolinite',
       'F1CC06B Magnesite', 'LASF21A Jarosite', 'LASA58 Saponite',
       'LAPO05 Fayalite', 'NBPP22 Clinopyroxene', '799F366 Mg-sulphate',
       'LASF41A Gypsum', 'HS433 Halite', 'c1cy29 Vermiculite',
       'F1CC08B Alunite', 'bkr1jb329c Siliceous-sinter', 'GDS106 Margarite',
       'LAPP47B Orthopyroxene', 'GDS23 Talc', 'GDS1 Analcime',
       'GDS145 Bassanite', 'LASC02 Forsterite', 'GDS27 Hematite',
       'GDS26 Epidote', 'F1CC15 Kieserite'],
      dtype='object', name='Data ID')
```

Now collecting all of these functions in a set of callable functions, such that we can run this analysis on each entry.

Here we also compute a 'baseline' Spectral Angle, between each clean entry and each other clean entry.

[152]:
```
def spectral_angle(data_a, data_b):
    # compute spectal angle
    numerator = np.inner(data_a, data_b) #TODO replace inner with nansum
    denominator = np.outer(np.sqrt(np.sum(data_a**2, axis=1)), np.sqrt(np.
 ↪sum(data_b**2, axis=1)))
    x = numerator / denominator
    # check range of x is in [-1, 1]
    x = np.clip(x, -1, 1)
    sa = np.arccos(x)
    # convert from radians to degrees
    sa = np.degrees(sa)
    return sa

def compute_spectral_angle(obs, material_name):
    # get noisy data
    noisy_data = obs.get_refl_df(root_data_id=material_name).to_numpy()
    bad_bands = ~np.isfinite(noisy_data[0,:])
    noisy_data = np.where(bad_bands, 0, noisy_data) # set all nan values to 0

    # get clean data
    clean_data = obs.noiseless_df[obs.wvls].to_numpy()
    clean_data = np.where(bad_bands, 0, clean_data) # 0 the clean data where␣
 ↪the noisy data is NaN
    clean_data = np.where(~np.isfinite(clean_data), 0, clean_data) # 0 the␣
 ↪clean data where the clean data is NaN

    sa_noisy = spectral_angle(noisy_data, clean_data)
```

```
    # get baseline Spectral Angle
    baseline_data = obs.noiseless_df.loc[material_name][obs.wvls].to_numpy()
    baseline_data = np.expand_dims(baseline_data, axis=0).astype(np.float64)
    baseline_data = np.where(bad_bands, 0, baseline_data) # 0 the clean data␣
↪where the noisy data is NaN
    baseline_data = np.where(~np.isfinite(baseline_data), 0, baseline_data) # 0␣
↪the clean data where the clean data is NaN

    sa_baseline = spectral_angle(baseline_data, clean_data)

    # package in DF
    sa_df = pd.DataFrame(sa_noisy, index=obs.
↪get_refl_df(root_data_id=material_name).index, columns=obs.noiseless_df.
↪index)
    sa_bl = pd.Series(sa_baseline.squeeze(), index=obs.noiseless_df.index)
    # set sa_bl for index of material_name to 0
    sa_bl.loc[material_name] = 0.0

    # order DF columns in baseline value order
    sa_df = sa_df.loc[:, sa_bl.sort_values().index]
    sa_bl = sa_bl.loc[sa_bl.sort_values().index]

    return sa_df, sa_bl
```

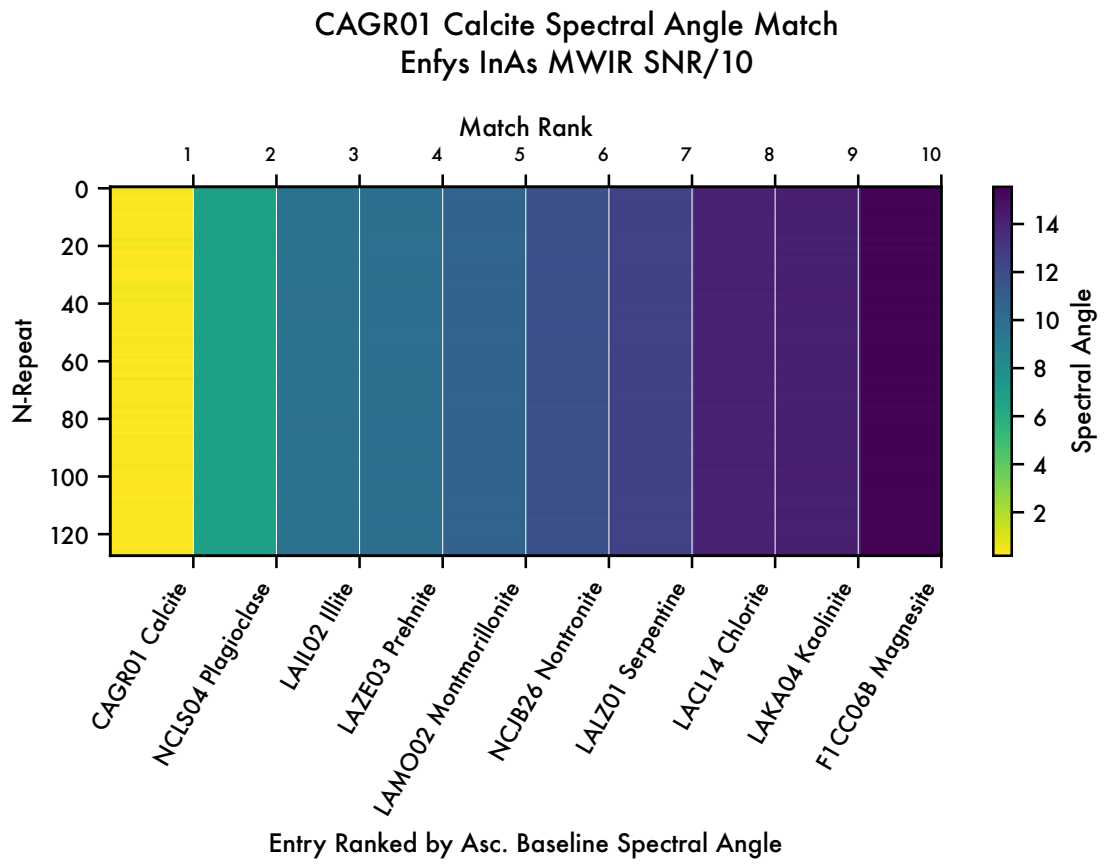
[153]:
```
sa_df, sa_bl = compute_spectral_angle(obs, test_material)
```

[154]:
```
from matplotlib import pyplot as plt
```

[155]:
```
def plot_spectral_angle_noisy_map(sa_df, material_name, top_n=10, ax: plt.
↪Axes=None):
    if ax is None:
        fig, ax = plt.subplots()
    else:
        fig = ax.figure

    top_n_sa_df = sa_df.iloc[:,:top_n]

    im = ax.imshow(top_n_sa_df.to_numpy(),
                cmap='viridis_r',
                interpolation='none', resample=False,
                # vmin=0, vmax=1,
                aspect='auto')
    # add color bar called 'Spectral Angle'
    ax.set_xlabel('Entry Ranked by Asc. Baseline Spectral Angle')
    ax.set_ylabel('N-Repeat')
    cbar = ax.figure.colorbar(im, ax=ax, location='right')
    cbar.ax.set_ylabel('Spectral Angle')
```

```
    # use index for axis labels
    # set every tickmark to the corresponding index
    xticks=ax.set_xticks(np.arange(len(top_n_sa_df.columns))+0.5)
    xlbls=ax.set_xticklabels(top_n_sa_df.columns.to_list(), rotation=60,␣
↪horizontalalignment='right', verticalalignment='top')

    # add vertical gridlines
    ax.grid(which='major', axis='x', color='white', linestyle='-', linewidth=0.
↪2)

    # add a top y-axis called Match Rank, with ticks aligned to the right
    ax2 = ax.twiny()
    ax2.set_xlim(ax.get_xlim())
    ax2.set_xticks(np.arange(len(top_n_sa_df.columns))+0.5)
    ax2.set_xticklabels(np.arange(1, len(top_n_sa_df.columns)+1),␣
↪horizontalalignment='right', verticalalignment='bottom', fontsize=7)
    ax2.set_xlabel('Match Rank')

    fig.suptitle(f'{material_name} Spectral Angle Match \n Enfys InAs MWIR SNR/
↪10')
    fig.tight_layout()

    return fig, ax
```

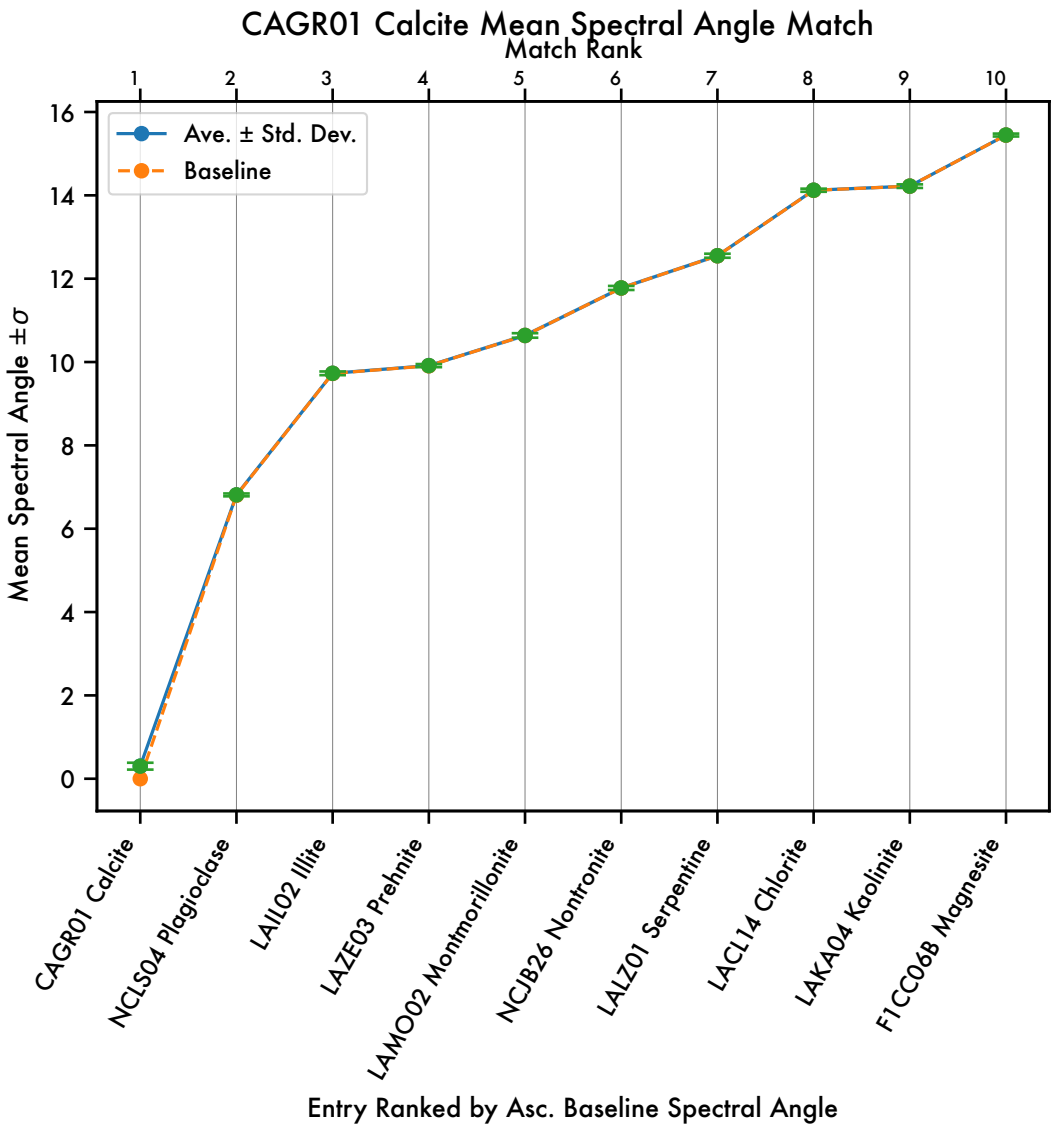
[156]:
```
fig, ax = plot_spectral_angle_noisy_map(sa_df, test_material)
```



[157]:
```
# alternatively, we can plot the Spectal Angle score against the entry
def plot_mean_spectral_angle(sa_df, sa_bl, material_name, top_n: int=10, ax:␣
↪plt.Axes=None):
    if ax is None:
        fig, ax = plt.subplots()
    else:
        fig = ax.figure

    top_n_sa_df = sa_df.iloc[:,:top_n]

    ax.plot(top_n_sa_df.mean(axis=0), marker='o', label='Ave. ± Std. Dev.')
    ax.plot(sa_bl[:top_n], marker='o', label='Baseline', linestyle='--')
    ax.legend(loc='upper left')
    # set ylimits
    # ax.set_ylim(0, 1)
    # add error bars
    ax.errorbar(np.arange(len(top_n_sa_df.columns)), top_n_sa_df.mean(axis=0),␣
↪yerr=top_n_sa_df.std(axis=0), fmt='o', capsize=5)
```

```
    ax.fill_between(np.arange(len(top_n_sa_df.columns)), top_n_sa_df.
↪mean(axis=0) - top_n_sa_df.std(axis=0), top_n_sa_df.mean(axis=0) +␣
↪top_n_sa_df.std(axis=0), alpha=0.2)
    ax.set_ylabel('Mean Spectral Angle $\pm \sigma$')
    ax.set_xlabel('Entry Ranked by Asc. Baseline Spectral Angle')
    ax.set_xticks(top_n_sa_df.columns.to_list())
    xlbls=ax.set_xticklabels(top_n_sa_df.columns.to_list(), rotation=60,␣
↪horizontalalignment='right', verticalalignment='top')

    # add vertical gridlines
    ax.grid(which='major', axis='x', color='gray', linestyle='-', linewidth=0.2)

    # add a top x-axis called Match Rank, with ticks aligned to the right
    ax2 = ax.twiny()
    ax2.set_xlim(ax.get_xlim())
    ax2.set_xticks(np.arange(len(top_n_sa_df.columns)))
    ax2.set_xticklabels(np.arange(1, len(top_n_sa_df.columns)+1),␣
↪horizontalalignment='right', verticalalignment='center', fontsize=7)
    x2lbls=ax2.set_xlabel('Match Rank')

    title = f'{material_name} Mean Spectral Angle Match'
    fig.suptitle(title)

    return fig, ax
```

[158]:
```
fig, ax = plot_mean_spectral_angle(sa_df, sa_bl, test_material)
```



Finally, let's plot the top 3 results in a spectral library analyser plot.

[159]:
```
top3 = sa_df.mean(axis=0).sort_values().index[:3].to_list()
top3
```

[159]:
```
['CAGR01 Calcite', 'NCLS04 Plagioclase', 'LAIL02 Illite']
```

It will be more convenient to show single-drawn noisy samples here, so we will make another observation object, that only draws 1 repeat sample.

[160]:
```
obs_1 = Observation(
    material_collection=matcol,
    instrument = enfys,
```

```
    load_existing=False,
    plot_profiles=False,
    export_df=False)
```

```
Building new Observation DataFrame
for 'enfys_mica_inas_mwir'…
Sampling the Material Collection with the Instrument…
Sampling complete.
```

[161]:
```
_ = obs_1.add_noise(1, seed=0)
```

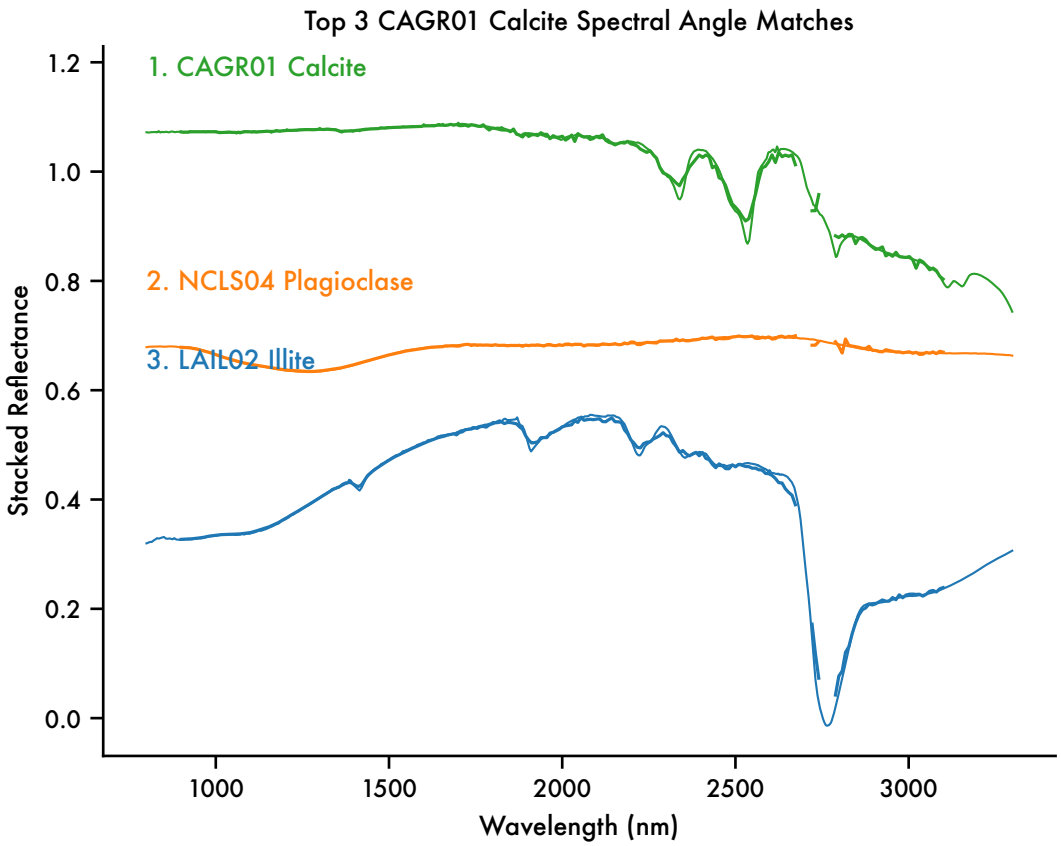
[162]:
```
top3
```

[162]:
```
['CAGR01 Calcite', 'NCLS04 Plagioclase', 'LAIL02 Illite']
```

[163]:
```
def plot_top3_matches(obs, matcol, material_name, top3, ax: plt.Axes=None):
    if ax is None:
        fig, ax = plt.subplots()
    else:
        fig = ax.figure

    top3_hires_df = matcol.main_df.loc[top3]
    refl_df_top3_hires = top3_hires_df[matcol.wvls]

    top3_obs_df = obs.main_df[obs.main_df['Root Data ID'].isin(top3)]
    # set Root Data ID as index
    top3_obs_df = top3_obs_df.set_index('Root Data ID')
    refl_df_top3 = top3_obs_df[obs.wvls]

    # get reverse order of top3
    top3_r = top3[::-1]

    refl_df_top3 = refl_df_top3.loc[top3_r]
    refl_df_top3_hires = refl_df_top3_hires.loc[top3_r]

    minima = refl_df_top3.min(axis=1)
    maxima = refl_df_top3.max(axis=1)
    spec_range = maxima - minima
    max_range = (spec_range).max()

    # offset the reflectance
    padding =  max_range * 1/6
    spec_range[spec_range < padding] = padding
    offsets = - minima + (spec_range+padding).cumsum().shift(periods=1,␣
↪fill_value = 0) + padding/2

    # # apply offsets to each entry
    # offsets = refl_df_top3.max(axis=1).cumsum()
```

```
    refl_df_top3 = refl_df_top3.add(offsets, axis=0)
    refl_df_top3_hires = refl_df_top3_hires.add(offsets, axis=0)

    maxima = refl_df_top3_hires.max(axis=1)

    cmap = plt.get_cmap('tab10_r')

    refl_df_top3_hires.T.plot(ax=ax, linewidth=0.8)
    l = ax.get_lines()
    refl_df_top3.T.plot(ax=ax, color=[i.get_color() for i in l])

    # add padding to y lim
    # get ylim
    ylim = ax.get_ylim()
    ax.set_ylim(ylim[0], ylim[1]+padding)

    # despine
    sns.despine(ax=ax)

    # xlabel
    ax.set_xlabel('Wavelength (nm)')
    # ylabel
    ax.set_ylabel('Stacked Reflectance')

    # no legend
    ax.legend().set_visible(False)

    # annotate with the top3 at the end of each plot line
    for i, line in enumerate(l):
        ax.annotate(f'{3-i}. {top3_r[i]}', xy=(line.get_xdata()[0],␣
↪maxima[top3_r[i]]+padding), color=line.get_color())

    ax.set_title(f'Top 3 {material_name} Spectral Angle Matches')

    return fig, ax
```

[164]:
```
for material in materials[0:1]:
    sa_df, sa_bl = compute_spectral_angle(obs, material)
    top3 = sa_bl.index[:3].to_list()
    fig, ax = plot_top3_matches(obs_1, matcol, material, top3)
```



6.1.1 Summary Statistics

For each material, we can summarise the ability to resolve the matching material using the Fisher Ratio, as a measure of separability between the spectral angles of the 2 top matches, that incorporates the uncertainty on each spectral angle.

For the top match, the spectral angle is $\theta_1 \pm \sigma_{\theta_1}$, and for the next match, the spectral angle is $\theta_2 \pm \sigma_{\theta_2}$.

The Fisher Ratio (FR) gives the separation between the Spectral Angles, normalised to the total variance.

$$FR = \frac{|\theta_1 - \theta_2|^2}{|\sigma_{\theta_1}^2 + \sigma_{\theta_1}^2|}$$

[165]:
```
analysis_dir = obs_1.object_dir / 'analysis'
sa_dir = analysis_dir / 'spectral_angle'
sa_dir.mkdir(parents=True, exist_ok=True)
```

[166]:
```
fr_list = []
sa_list = []

fig_ax_dict = {}

for material in materials:
    print(f'Evaluating Spectral Angle for {material}')
    sa_df, sa_bl = compute_spectral_angle(obs, material)

    fig, axes = plt.subplots(1,3, figsize=(12,6))

    fig, ax = plot_spectral_angle_noisy_map(sa_df, material, ax=axes[0])
    # fig.savefig(f'../reports/figures/
 ↪{PROJECT_NAME}_{material}_spectral_angle_map.png', dpi=300,␣
 ↪bbox_inches='tight')
    # plt.close(fig)

    fig, ax = plot_mean_spectral_angle(sa_df, sa_bl, material, ax=axes[1])
    # fig.savefig(f'../reports/figures/
 ↪{PROJECT_NAME}_{material}_mean_spectral_angle.png', dpi=300,␣
 ↪bbox_inches='tight')
    # plt.close(fig)

    top3 = sa_bl.index[:3].to_list()
    fig, ax = plot_top3_matches(obs_1, matcol, material, top3, ax=axes[2])

    # log the mean spectral angle of the top match, and the uncertainty
    top_match = top3[0]
    ave1_sa = sa_df.mean().loc[top_match]
    std1_sa = sa_df.std().loc[top_match]
    sa_list.append((ave1_sa, std1_sa))

    # compute the Fisher Ratio between the 1st and 2nd matches
    second_match = top3[1]
    ave2_sa = sa_df.mean().loc[second_match]
    std2_sa = sa_df.std().loc[second_match]
    fisher_ratio = (ave1_sa - ave2_sa)**2 / (std1_sa**2 + std2_sa**2)
    fr_list.append(fisher_ratio)
    fig.tight_layout()
    fig_ax_dict[material] = fig, ax

    # save the figure
    fig.savefig(sa_dir / f'{material}_spectral_angle.pdf', bbox_inches='tight')
    fig.savefig(sa_dir / f'{material}_spectral_angle.svg', bbox_inches='tight')
    # plt.close(fig)

# format the top spectral angle results and Fisher Ratios
```

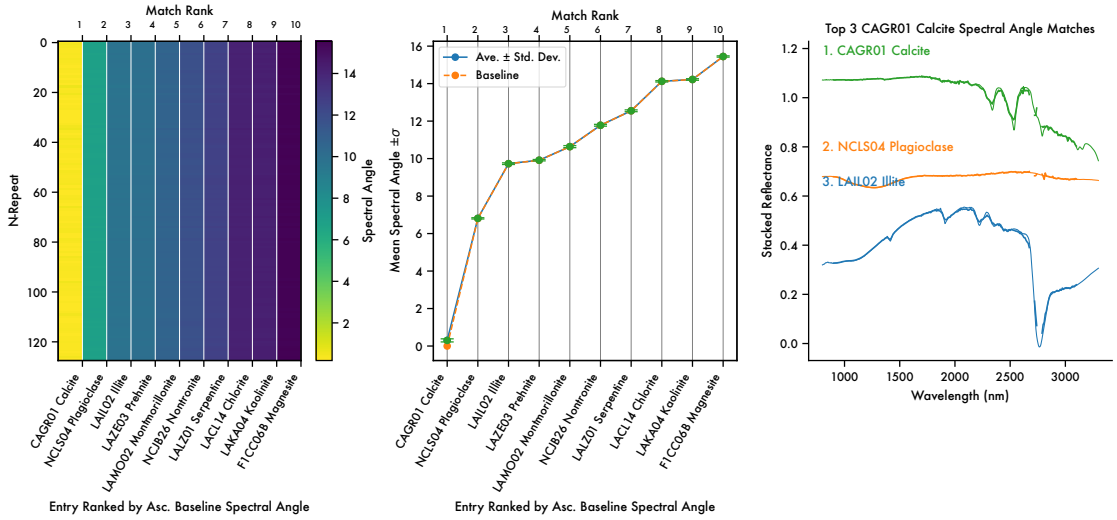
```
fr_series = pd.Series(fr_list, index=materials)
fr_series.sort_values(ascending=False, inplace=True)
sa_series = pd.Series(sa_list, index=materials)
sa_top_df = pd.DataFrame(sa_series.values.tolist(), index=sa_series.index,␣
 ↪columns=['ave', 'std'])
# sort by average
sa_top_df.sort_values(by='ave', ascending=True, inplace=True)

# save the top spectral angle results and Fisher Ratios
sa_top_df.to_csv(sa_dir / 'top_spectral_angles.csv')
fr_series.to_csv(sa_dir / 'fisher_ratios.csv')
```

```
Evaluating Spectral Angle for CAGR01 Calcite
Evaluating Spectral Angle for F1CC06B Magnesite
Evaluating Spectral Angle for HS433 Halite
Evaluating Spectral Angle for LAZE03 Prehnite
Evaluating Spectral Angle for bkr1jb329c Siliceous-sinter
Evaluating Spectral Angle for GDS26 Epidote
Evaluating Spectral Angle for GDS1 Analcime
Evaluating Spectral Angle for NBPP22 Clinopyroxene
Evaluating Spectral Angle for LAPP47B Orthopyroxene
Evaluating Spectral Angle for LAPO05 Fayalite
Evaluating Spectral Angle for LASC02 Forsterite
Evaluating Spectral Angle for GDS27 Hematite
Evaluating Spectral Angle for NCLS04 Plagioclase
Evaluating Spectral Angle for LACL14 Chlorite
Evaluating Spectral Angle for LAKA04 Kaolinite
Evaluating Spectral Angle for LAIL02 Illite
Evaluating Spectral Angle for GDS106 Margarite
Evaluating Spectral Angle for c1cy29 Vermiculite
Evaluating Spectral Angle for LALZ01 Serpentine
Evaluating Spectral Angle for LAMO02 Montmorillonite
Evaluating Spectral Angle for NCJB26 Nontronite
```

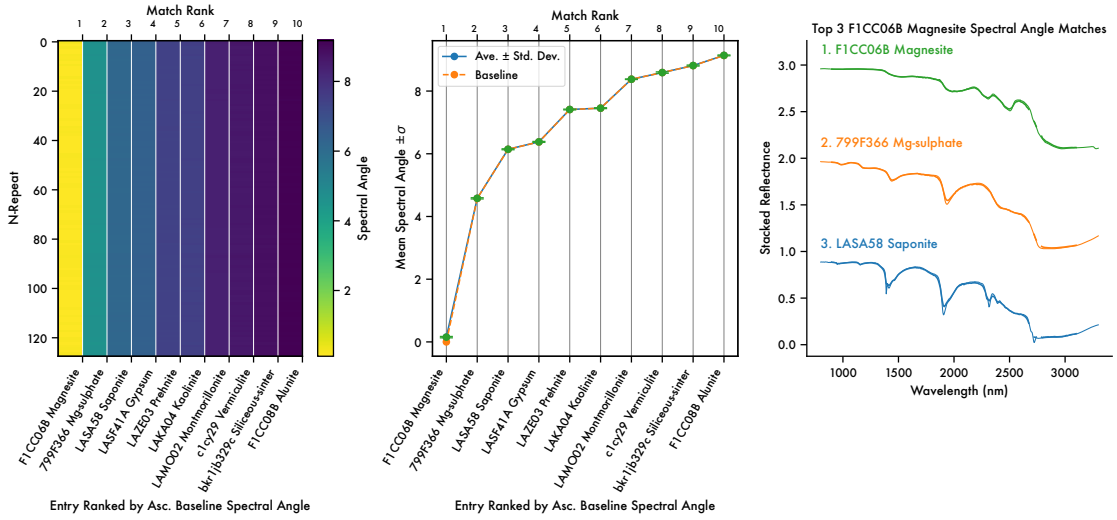
```
/var/folders/w4/44k32f413dd82lxmtw9456w80000gp/T/ipykernel_54287/181728151.py:10
: RuntimeWarning: More than 20 figures have been opened. Figures created through
the pyplot interface (`matplotlib.pyplot.figure`) are retained until explicitly
closed and may consume too much memory. (To control this warning, see the
rcParam `figure.max_open_warning`). Consider using `matplotlib.pyplot.close()`.
  fig, axes = plt.subplots(1,3, figsize=(12,6))
```

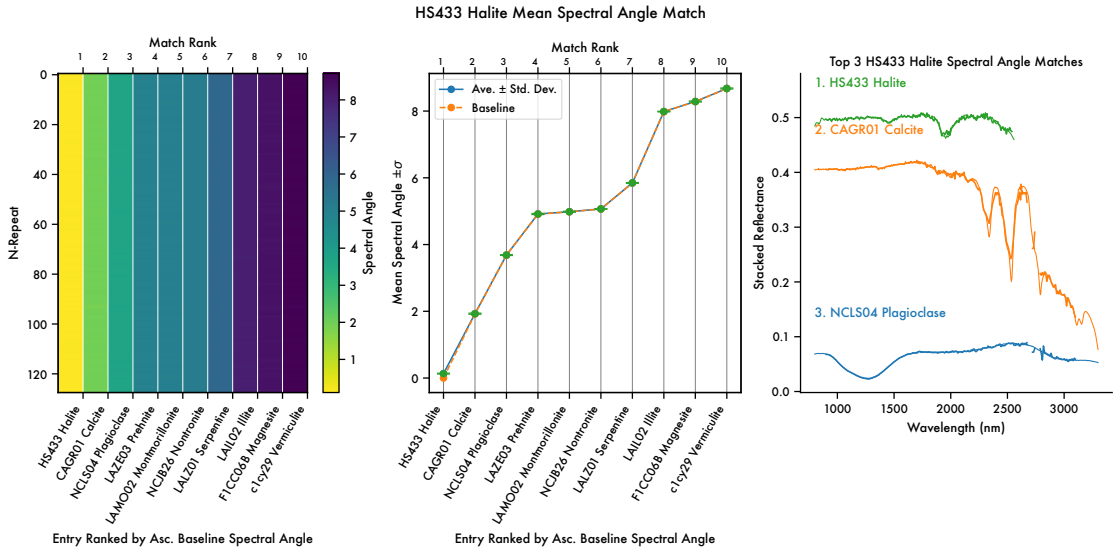
```
Evaluating Spectral Angle for LASA58 Saponite
Evaluating Spectral Angle for GDS23 Talc
Evaluating Spectral Angle for F1CC08B Alunite
Evaluating Spectral Angle for GDS145 Bassanite
Evaluating Spectral Angle for LASF41A Gypsum
Evaluating Spectral Angle for LASF21A Jarosite
Evaluating Spectral Angle for F1CC15 Kieserite
Evaluating Spectral Angle for 799F366 Mg-sulphate
```

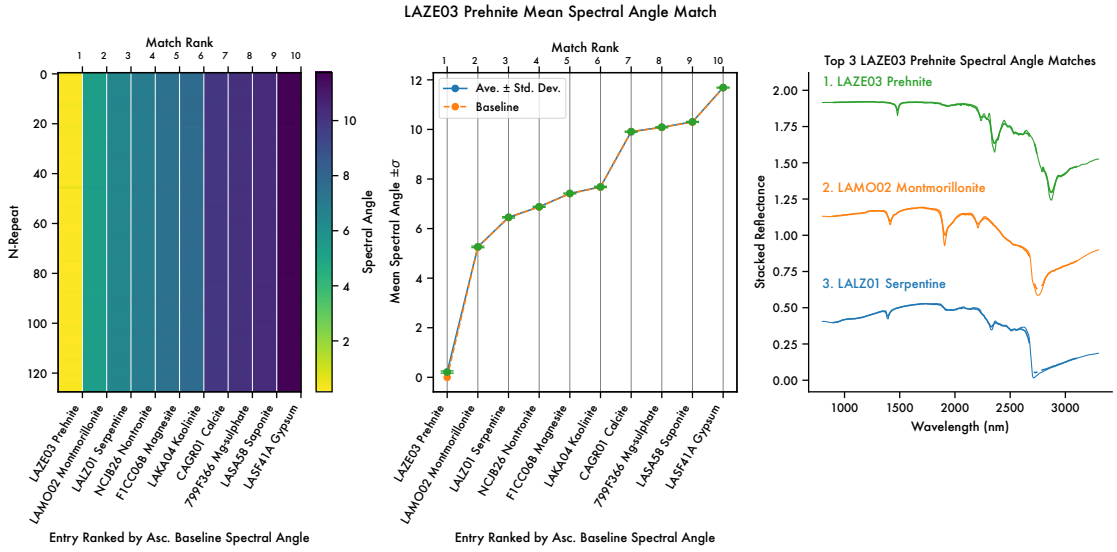
CAGR01 Calcite Mean Spectral Angle Match

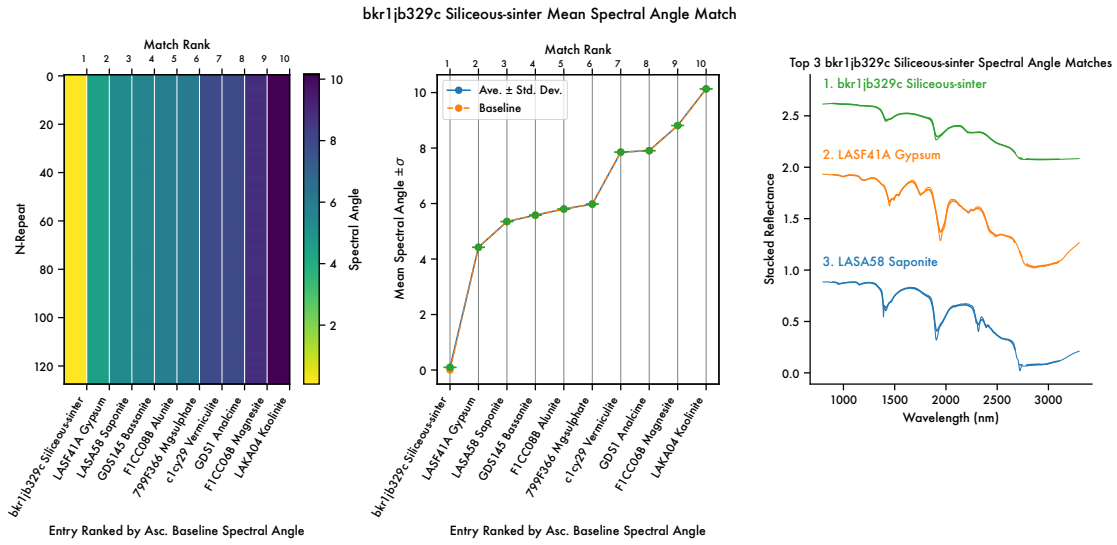


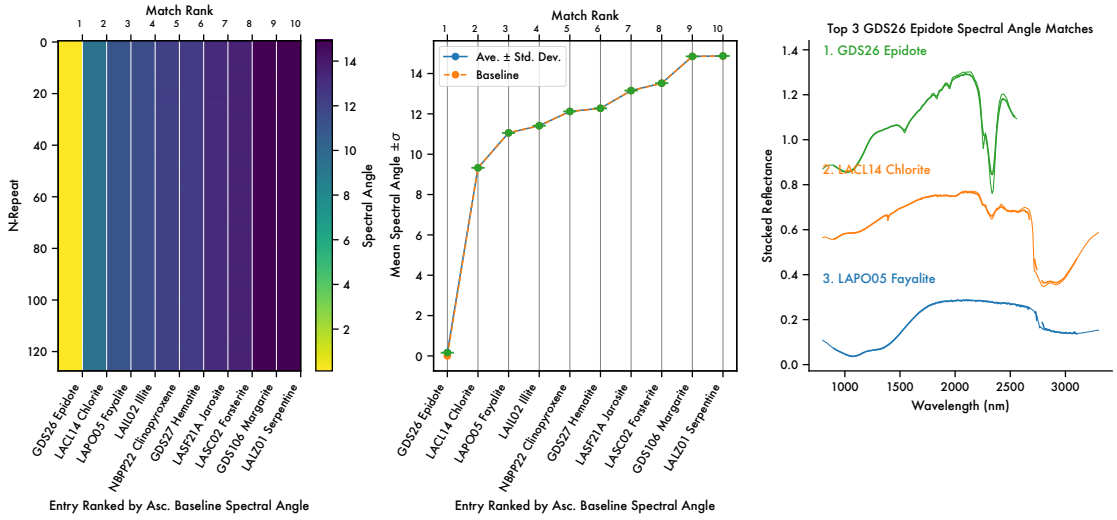
F1CC06B Magnesite Mean Spectral Angle Match

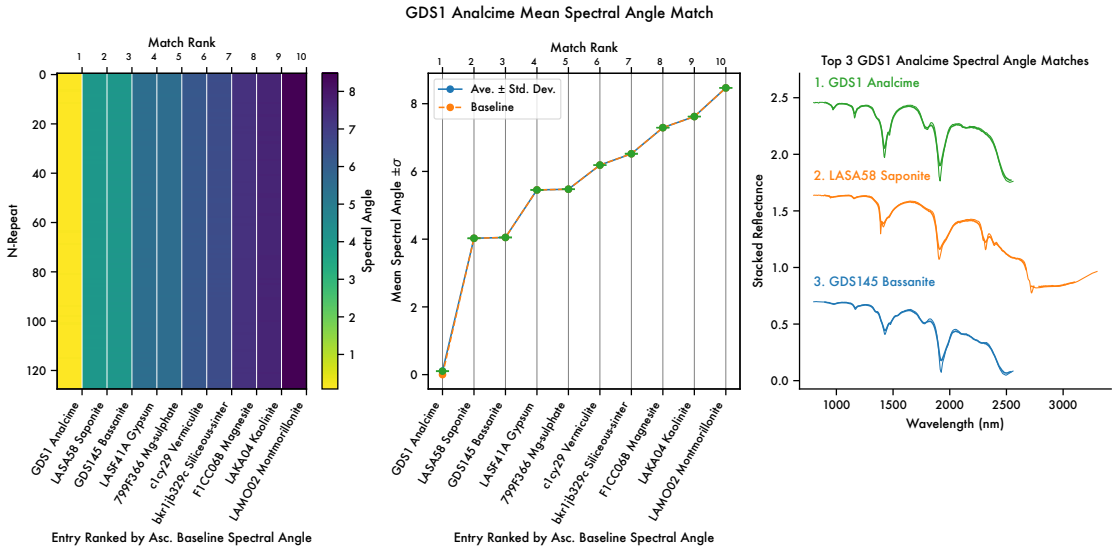


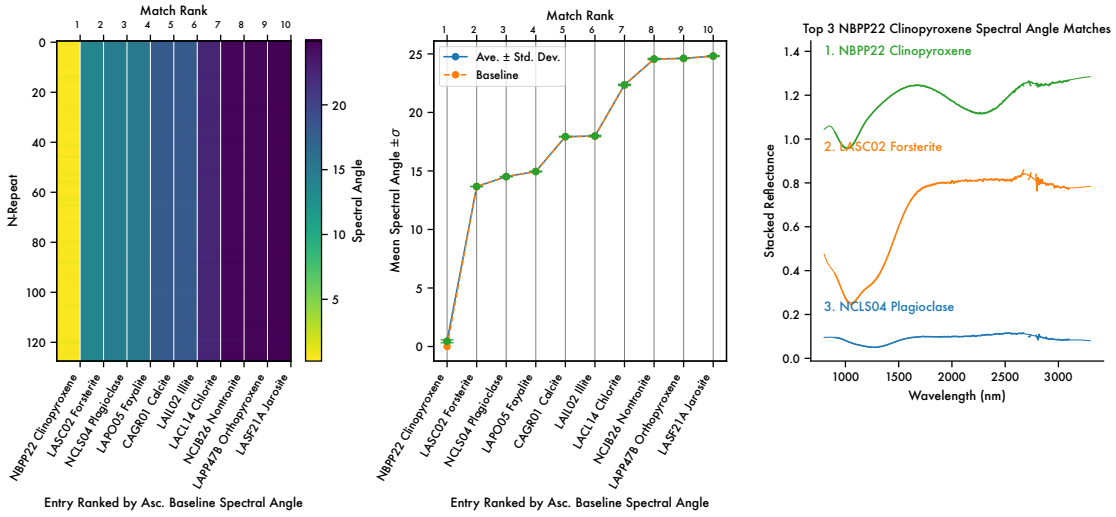

HS433 Halite Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Top 3 HS433 Halite Spectral Angle Matches
1. HS433 Halite
2. CAGR01 Calcite
3. NCLS04 Plagioclase
Stacked Reflectance
Wavelength (nm)
HS433 Halite
CAGR01 Calcite
NCLS04 Plagioclase
LAZE03 Prehnite
LAMO02 Montmorillonite
NCIB26 Nontronite
LALZ01 Serpentine
LAIL02 Illite
F1CC06B Magnesite
c1c92p Vermiculite
Entry Ranked by Asc. Baseline Spectral Angle


LAZE03 Prehnite Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Top 3 LAZE03 Prehnite Spectral Angle Matches
1. LAZE03 Prehnite
2. LAMO02 Montmorillonite
3. LALZ01 Serpentine
Stacked Reflectance
Wavelength (nm)
LAZE03 Prehnite
LAMO02 Montmorillonite
LALZ01 Serpentine
NCIB26 Nontronite
F1CC06B Magnesite
LAKA04 Kaolinite
CAGR01 Calcite
799F566 Mgsulphate
LASA58 Saponite
LASF41A Gypsum
Entry Ranked by Asc. Baseline Spectral Angle

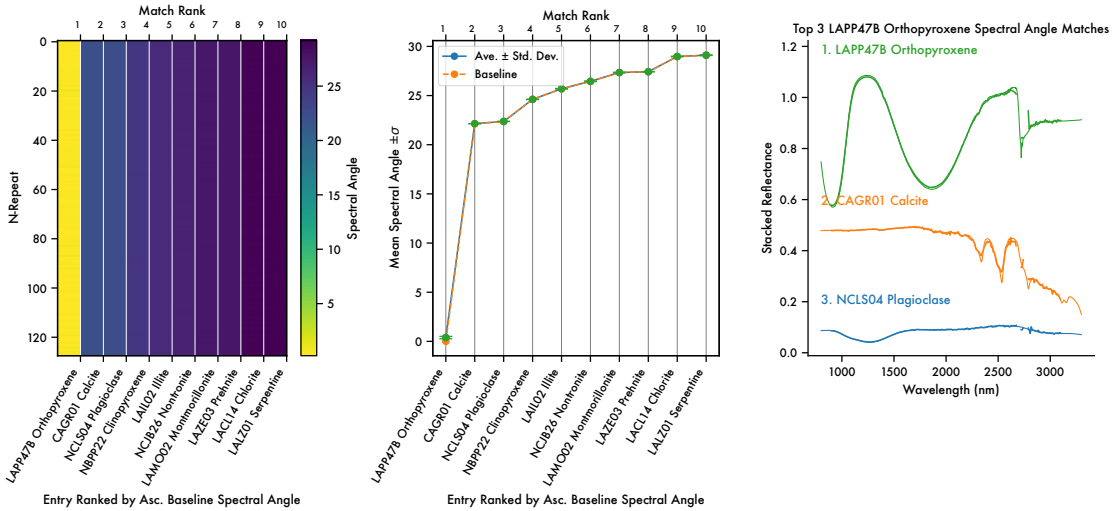

bkr1jb329c Siliceous-sinter Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Top 3 bkr1jb329c Siliceous-sinter Spectral Angle Matches
1. bkr1jb329c Siliceous-sinter
2. LASF41A Gypsum
3. LASA58 Saponite
Stacked Reflectance
Wavelength (nm)
bkr1jb329c Siliceous-sinter
LASF41A Gypsum
LASA58 Saponite
GDS145 Bassanite
F1CC08B Alunite
799F366 Mgsulphate
c1cy29 Vermiculite
GDS1 Analcime
F1CC06B Magnesite
LAKA04 Kaolinite
Entry Ranked by Asc. Baseline Spectral Angle


GDS26 Epidote Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Top 3 GDS26 Epidote Spectral Angle Matches
1. GDS26 Epidote
2. LACL14 Chlorite
3. LAPO05 Fayalite
Stacked Reflectance
Wavelength (nm)
GDS26 Epidote
LACL14 Chlorite
LAPO05 Fayalite
LAIL02 Illite
NBPP22 Clinopyroxene
GDS27 Hematite
LASF21A Jarosite
LASC02 Forsterite
GDS106 Margarite
LALZ01 Serpentine
Entry Ranked by Asc. Baseline Spectral Angle

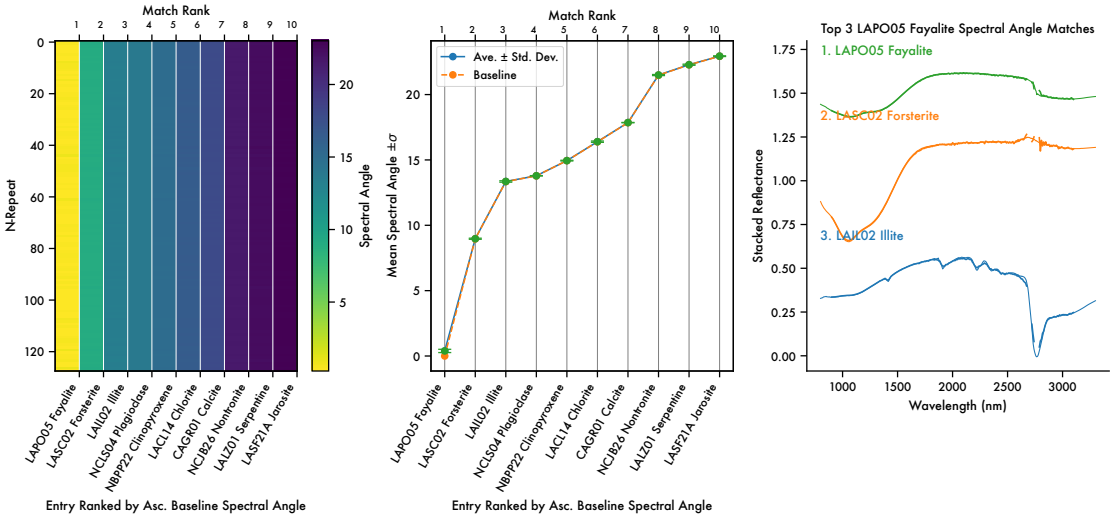

GDS1 Analcime Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Entry Ranked by Asc. Baseline Spectral Angle
Top 3 GDS1 Analcime Spectral Angle Matches
1. GDS1 Analcime
2. LASA58 Saponite
3. GDS145 Bassanite
Stacked Reflectance
Wavelength (nm)

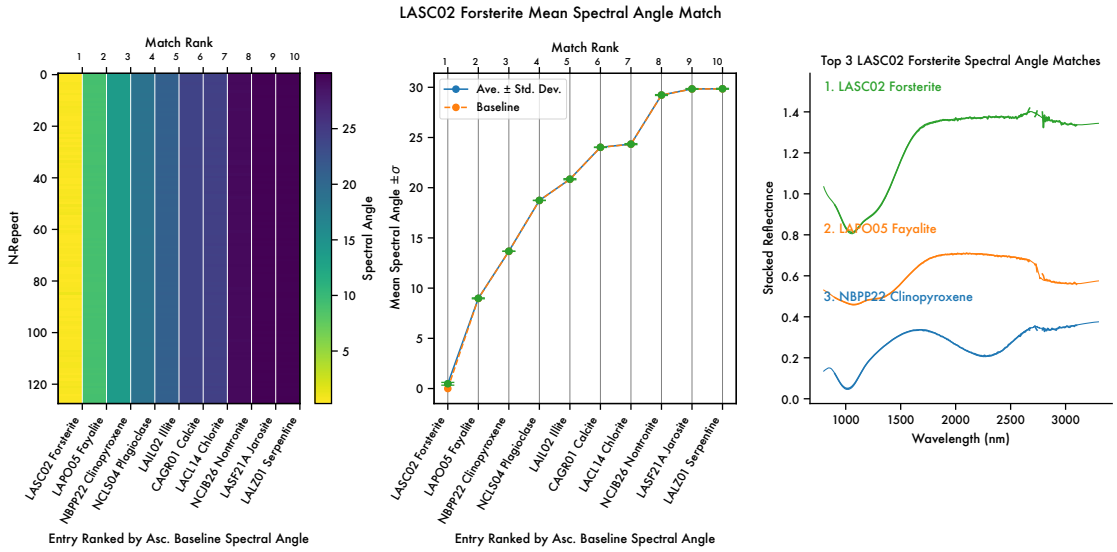

NBPP22 Clinopyroxene Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Entry Ranked by Asc. Baseline Spectral Angle
Top 3 NBPP22 Clinopyroxene Spectral Angle Matches
1. NBPP22 Clinopyroxene
2. LASC02 Forsterite
3. NCLS04 Plagioclase
Stacked Reflectance
Wavelength (nm)

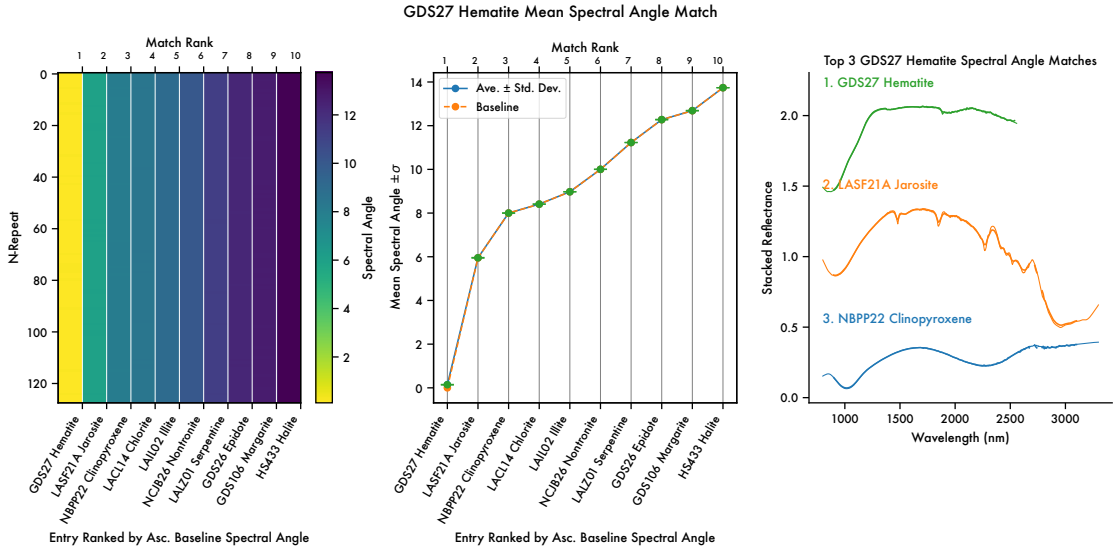
LAPP47B Orthopyroxene Mean Spectral Angle Match

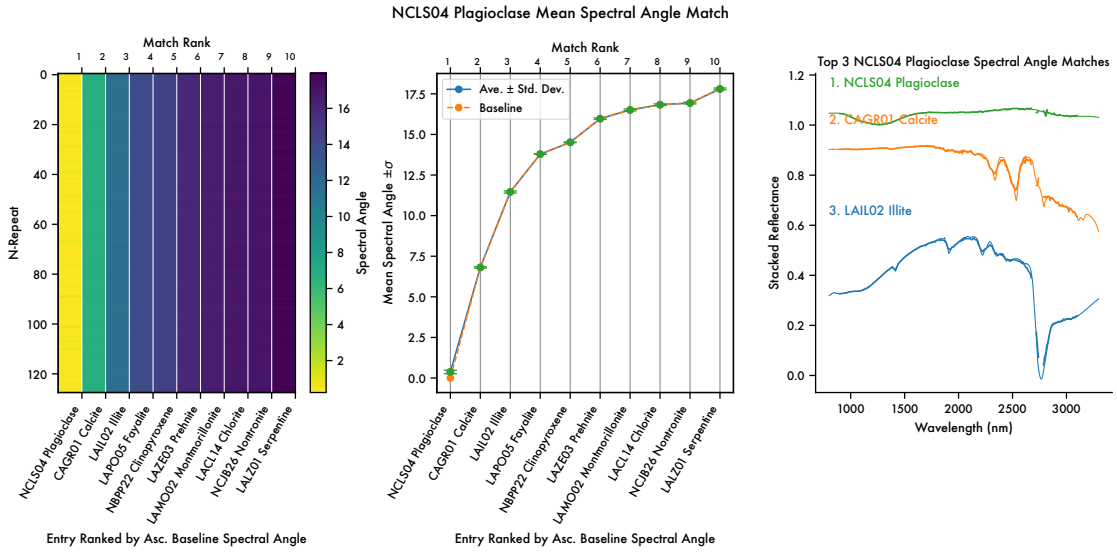


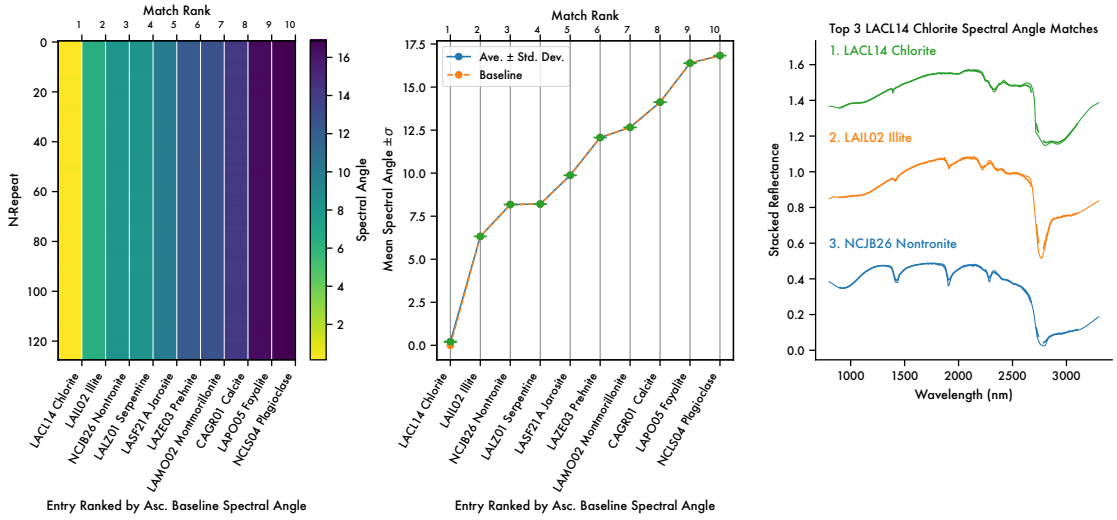
LAPO05 Fayalite Mean Spectral Angle Match

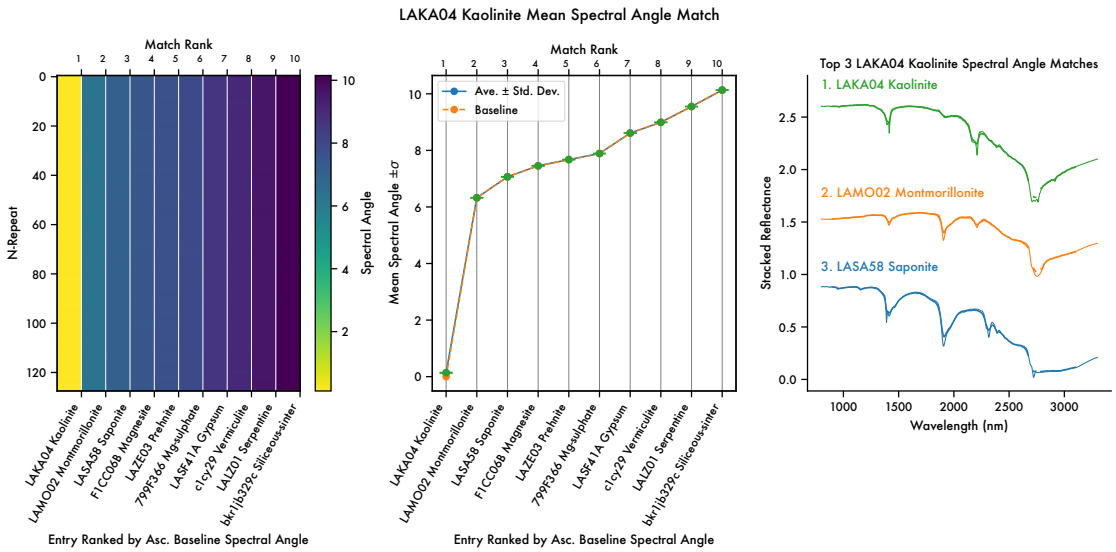


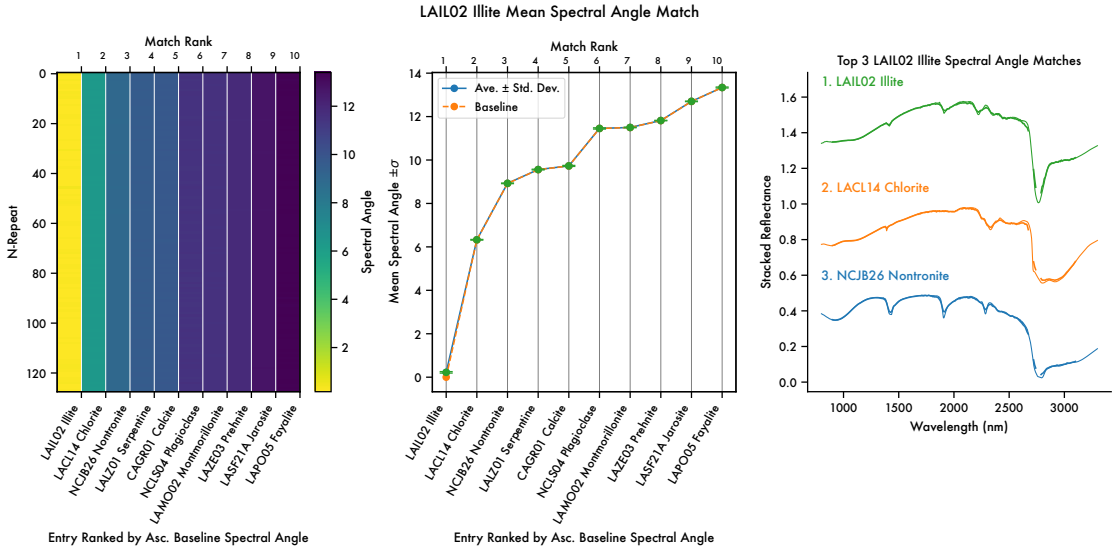

LASC02 Forsterite Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Top 3 LASC02 Forsterite Spectral Angle Matches
1. LASC02 Forsterite
2. LAPO05 Fayalite
3. NBPP22 Clinopyroxene
Stacked Reflectance
Wavelength (nm)
LASC02 Forsterite
LAPO05 Fayalite
NBPP22 Clinopyroxene
NCLS04 Plagioclase
LAIL02 Illite
CAGR01 Calcite
LACL14 Chlorite
NCIB26 Nontronite
LASF21A Jarosite
LALZ01 Serpentine
Entry Ranked by Asc. Baseline Spectral Angle

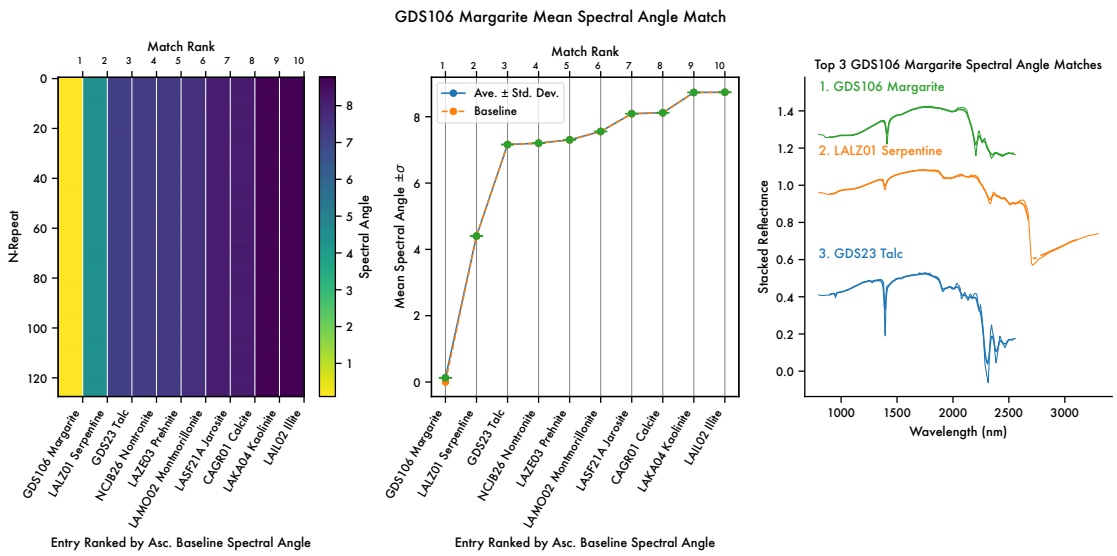

GDS27 Hematite Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Top 3 GDS27 Hematite Spectral Angle Matches
1. GDS27 Hematite
2. LASF21A Jarosite
3. NBPP22 Clinopyroxene
Stacked Reflectance
Wavelength (nm)
GDS27 Hematite
LASF21A Jarosite
NBPP22 Clinopyroxene
LACL14 Chlorite
LAIL02 Illite
NCIB26 Nontronite
LALZ01 Serpentine
GDS26 Epidote
GDS106 Margarite
HS433 Hollite
Entry Ranked by Asc. Baseline Spectral Angle

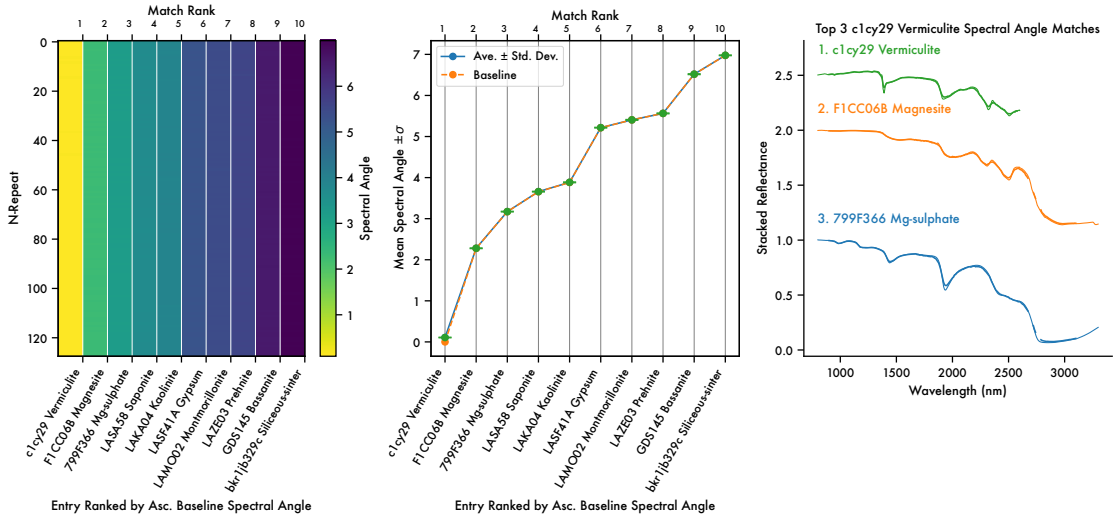

NCLS04 Plagioclase Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Top 3 NCLS04 Plagioclase Spectral Angle Matches
1. NCLS04 Plagioclase
2. CAGR01 Calcite
3. LAIL02 Illite
Stacked Reflectance
Wavelength (nm)
NCLS04 Plagioclase
CAGR01 Calcite
LAIL02 Illite
LAPO05 Fayalite
NBPP22 Clinopyroxene
LAZE03 Prehnite
LAMO02 Montmorillonite
LACL14 Chlorite
NCJB26 Nontronite
LALZ01 Serpentine
Entry Ranked by Asc. Baseline Spectral Angle

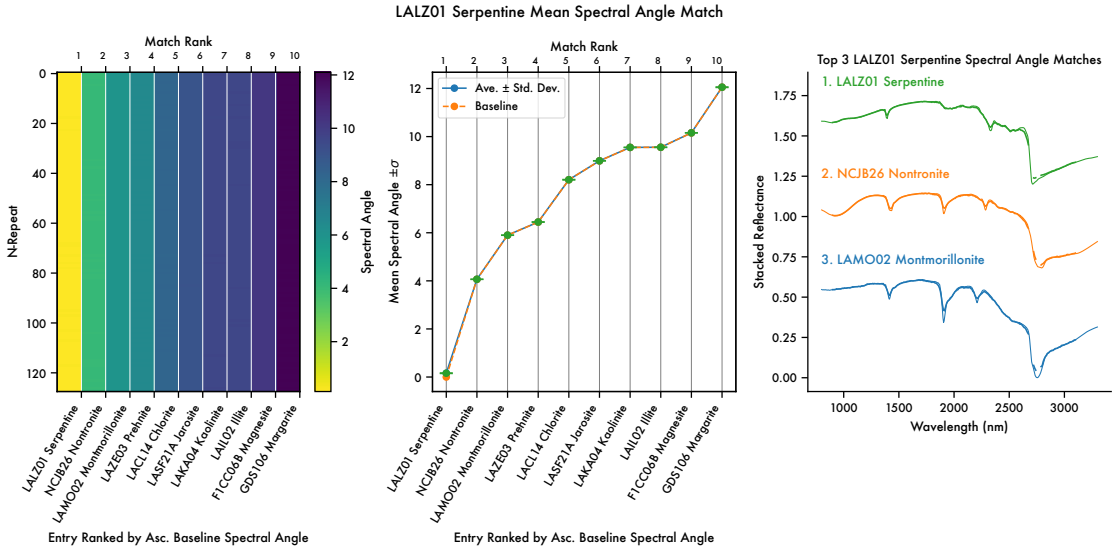

LACL14 Chlorite Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Top 3 LACL14 Chlorite Spectral Angle Matches
1. LACL14 Chlorite
2. LAIL02 Illite
3. NCJB26 Nontronite
Stacked Reflectance
Wavelength (nm)
LACL14 Chlorite
LAIL02 Illite
NCJB26 Nontronite
LALZ01 Serpentine
LASF21A Jarosite
LAZE03 Prehnite
LAMO02 Montmorillonite
CAGR01 Calcite
LAPO05 Fayalite
NCLS04 Plagioclase
Entry Ranked by Asc. Baseline Spectral Angle

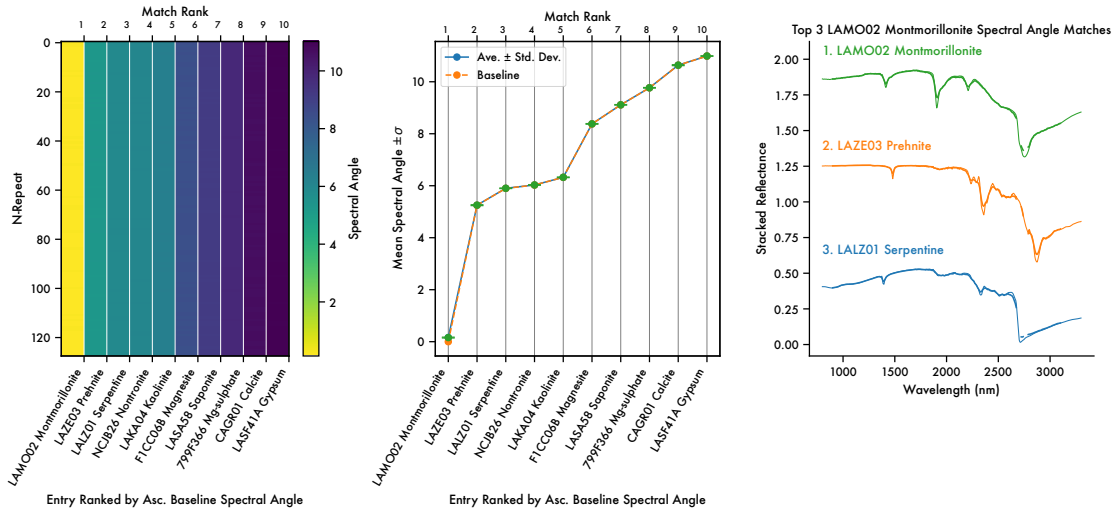

LAKA04 Kaolinite Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Stacked Reflectance
Wavelength (nm)
Top 3 LAKA04 Kaolinite Spectral Angle Matches
1. LAKA04 Kaolinite
2. LAMO02 Montmorillonite
3. LASA58 Saponite
LAKA04 Kaolinite
LAMO02 Montmorillonite
LASA58 Saponite
F1CC06B Magnesite
LAZE03 Prehnite
799F366 Mgsulphate
LASF41A Gypsum
c1cy29 Vermiculite
LALZ01 Serpentine
bkr1jb329c Siliceous-sinter
Entry Ranked by Asc. Baseline Spectral Angle

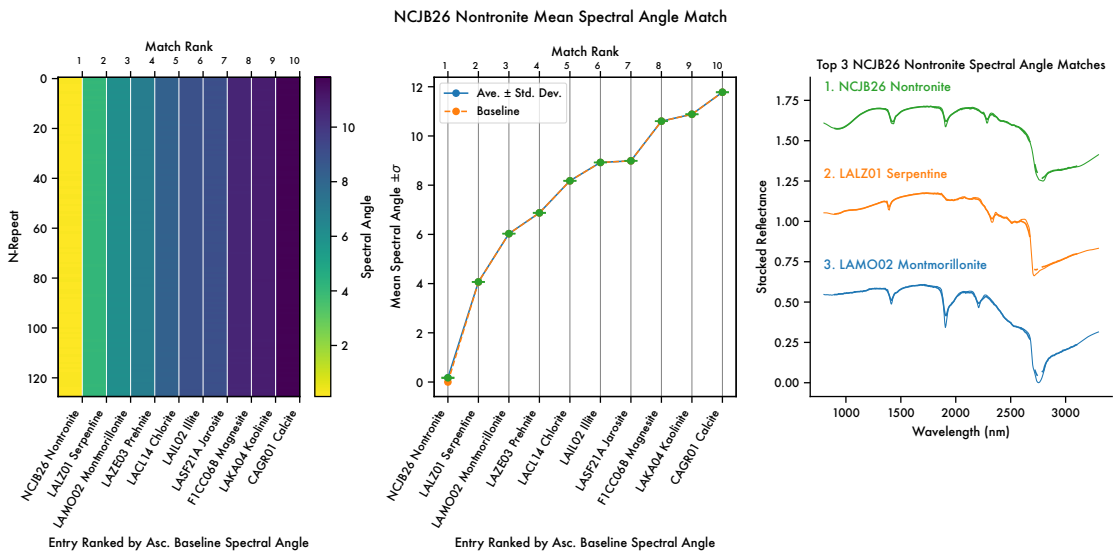

LAIL02 Illite Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Stacked Reflectance
Wavelength (nm)
Top 3 LAIL02 Illite Spectral Angle Matches
1. LAIL02 Illite
2. LACL14 Chlorite
3. NCJB26 Nontronite
LAIL02 Illite
LACL14 Chlorite
NCJB26 Nontronite
LALZ01 Serpentine
CAGR01 Calcite
NCLS04 Plagioclase
LAMO02 Montmorillonite
LAZE03 Prehnite
LASF21A Jarosite
LAPO05 Foyalite
Entry Ranked by Asc. Baseline Spectral Angle

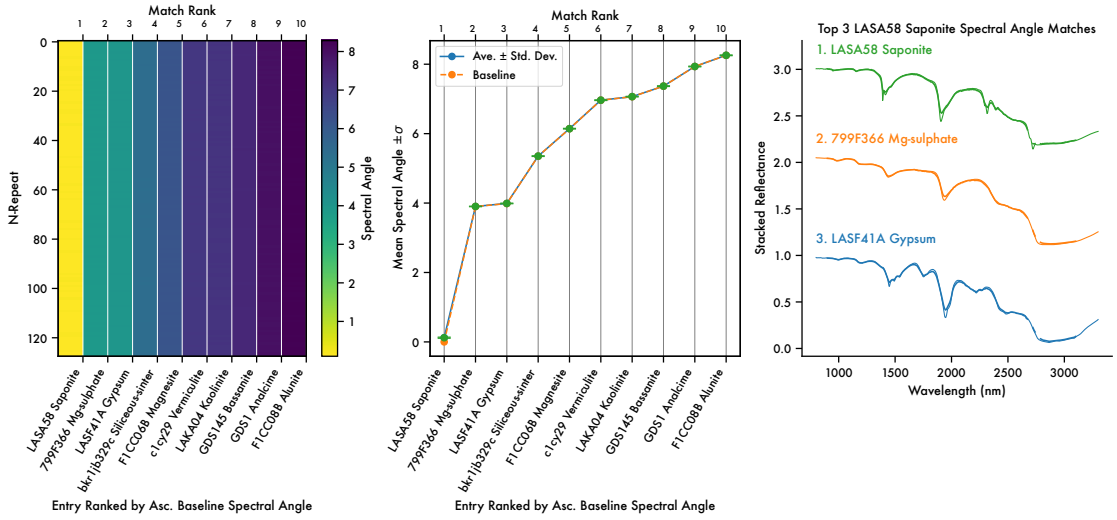

GDS106 Margarite Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Top 3 GDS106 Margarite Spectral Angle Matches
1. GDS106 Margarite
2. LALZ01 Serpentine
3. GDS23 Talc
Stacked Reflectance
Wavelength (nm)
GDS106 Margarite
LALZ01 Serpentine
GDS23 Talc
NCIB26 Montronite
LAZE03 Prehnite
LAMO02 Montmorillonite
LASF21A Jarosite
CAGR01 Calcite
LAKA04 Kaolinite
LAIL02 Illite
Entry Ranked by Asc. Baseline Spectral Angle

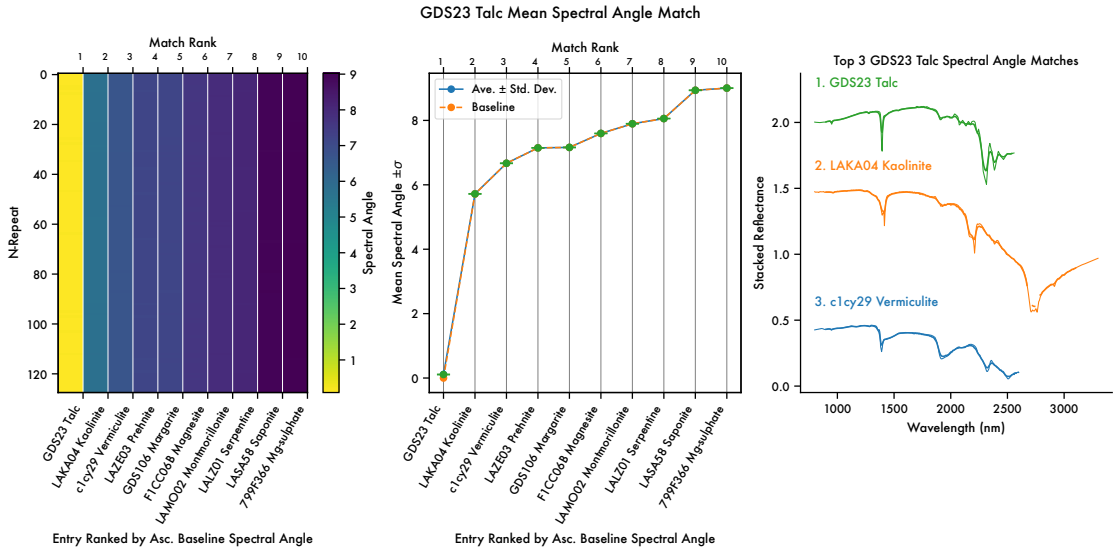

c1cy29 Vermiculite Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Top 3 c1cy29 Vermiculite Spectral Angle Matches
1. c1cy29 Vermiculite
2. F1CC06B Magnesite
3. 799F366 Mg-sulphate
Stacked Reflectance
Wavelength (nm)
c1cy29 Vermiculite
F1CC06B Magnesite
799F366 Mg-sulphate
LASA58 Saponite
LAKA04 Kaolinite
LASF41A Gypsum
LAMO02 Montmorillonite
LAZE03 Prehnite
GDS145 Bassanite
bkr1jb329c Siliceous-sinter
Entry Ranked by Asc. Baseline Spectral Angle

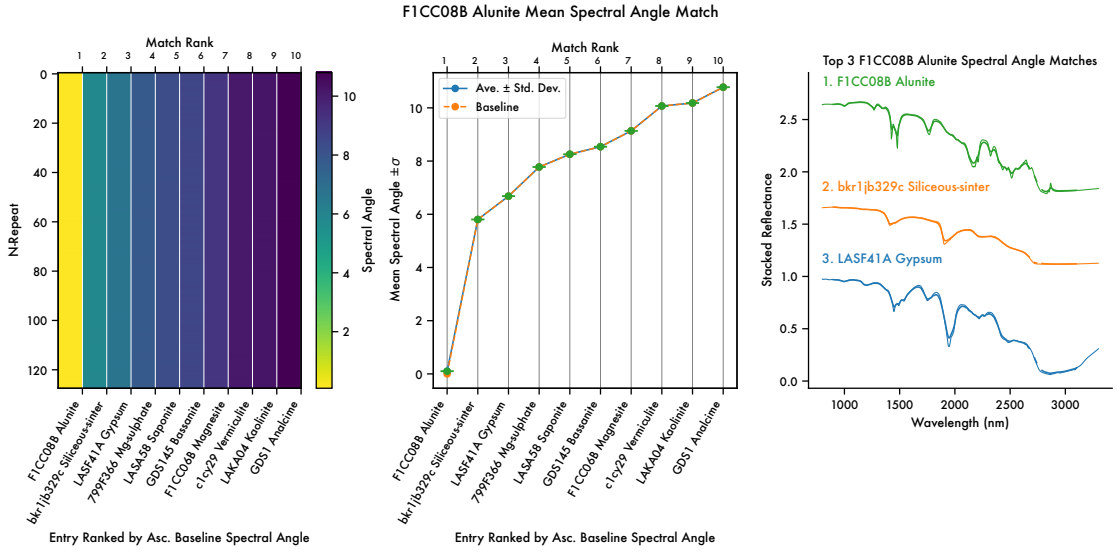

LALZ01 Serpentine Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Top 3 LALZ01 Serpentine Spectral Angle Matches
1. LALZ01 Serpentine
2. NCJB26 Nontronite
3. LAMO02 Montmorillonite
Stacked Reflectance
Wavelength (nm)
LALZ01 Serpentine
NCJB26 Nontronite
LAMO02 Montmorillonite
LAZE03 Prehnite
LACL14 Chlorite
LASF21A Jarosite
LAKA04 Kaolinite
LALL02 Illite
F1CC068 Magnesite
GDS106 Margarite
Entry Ranked by Asc. Baseline Spectral Angle

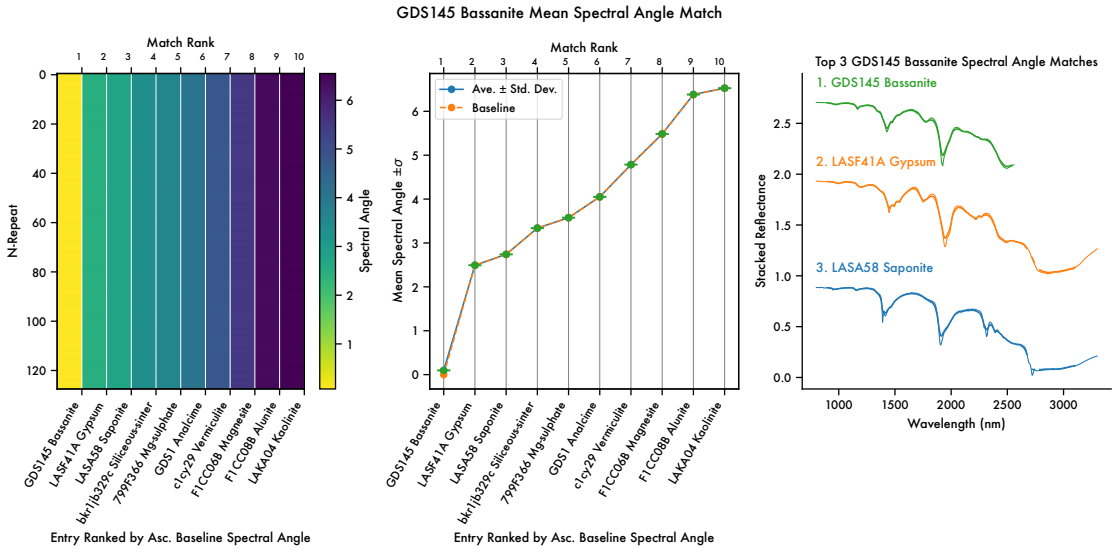

LAMO02 Montmorillonite Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Top 3 LAMO02 Montmorillonite Spectral Angle Matches
1. LAMO02 Montmorillonite
2. LAZE03 Prehnite
3. LALZ01 Serpentine
Stacked Reflectance
Wavelength (nm)
LAMO02 Montmorillonite
LAZE03 Prehnite
LALZ01 Serpentine
NCJB26 Nontronite
LAKA04 Kaolinite
F1CC068 Magnesite
LASA58 Saponite
79PF366 Magnolphate
CAGR01 Calcite
LASF41A Gypsum
Entry Ranked by Asc. Baseline Spectral Angle

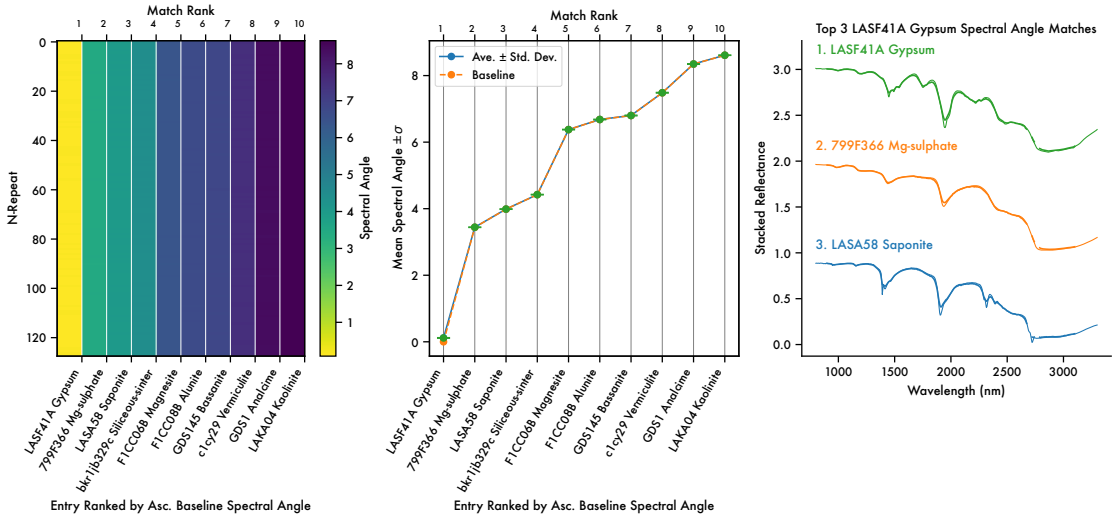

NCJB26 Nontronite Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Stacked Reflectance
Wavelength (nm)
Top 3 NCJB26 Nontronite Spectral Angle Matches
1. NCJB26 Nontronite
2. LALZ01 Serpentine
3. LAMO02 Montmorillonite
NCJB26 Nontronite
LALZ01 Serpentine
LAMO02 Montmorillonite
LAZE03 Prehnite
LACL14 Chlorite
LALI02 Illite
LASF21A Jarosite
F1CC06B Magnesite
LAKA04 Kaolinite
CAGR01 Calcite
Entry Ranked by Asc. Baseline Spectral Angle


LASA58 Saponite Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Stacked Reflectance
Wavelength (nm)
Top 3 LASA58 Saponite Spectral Angle Matches
1. LASA58 Saponite
2. 799F366 Mg-sulphate
3. LASF41A Gypsum
LASA58 Saponite
799F366 Mg-sulphate
LASF41A Gypsum
bkr1jb329c Siliceous-sinter
F1CC06B Magnesite
c1cy29 Vermiculite
LAKA04 Kaolinite
GDS145 Bassanite
GDS1 Analcime
F1CC08B Alunite
Entry Ranked by Asc. Baseline Spectral Angle

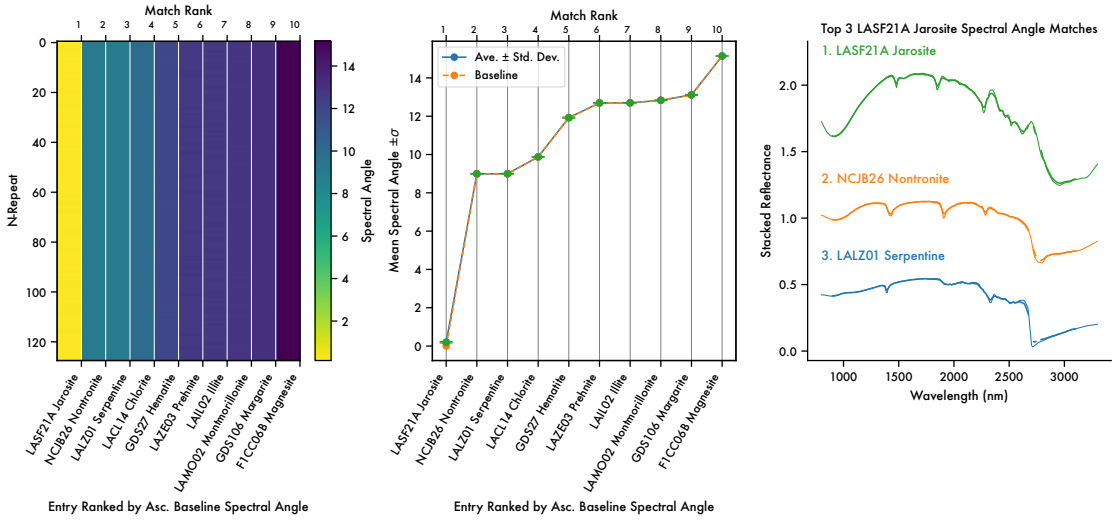

GDS23 Talc Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Top 3 GDS23 Talc Spectral Angle Matches
1. GDS23 Talc
2. LAKA04 Kaolinite
3. c1cy29 Vermiculite
Stacked Reflectance
Wavelength (nm)
GDS23 Talc
LAKA04 Kaolinite
c1cy29 Vermiculite
LAZE03 Prehnite
GDS106 Margarite
F1CC08B Magnesite
LAMO02 Montmorillonite
LALZ01 Serpentine
LASA58 Saponite
799F366 Mg-sulphate
Entry Ranked by Asc. Baseline Spectral Angle


F1CC08B Alunite Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Top 3 F1CC08B Alunite Spectral Angle Matches
1. F1CC08B Alunite
2. bkr1jb329c Siliceous-sinter
3. LASF41A Gypsum
Stacked Reflectance
Wavelength (nm)
F1CC08B Alunite
bkr1jb329c Siliceous-sinter
LASF41A Gypsum
799F366 Mg-sulphate
LASA58 Saponite
GDS145 Bassanite
F1CC068 Magnesite
c1cy29 Vermiculite
LAKA04 Kaolinite
GDS1 Analcime
Entry Ranked by Asc. Baseline Spectral Angle

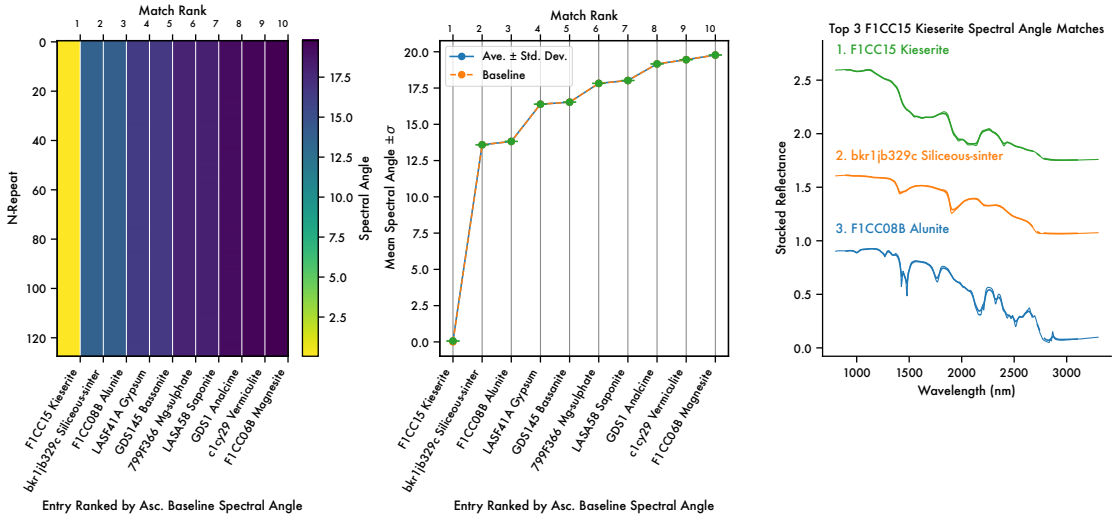

GDS145 Bassanite Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Top 3 GDS145 Bassanite Spectral Angle Matches
1. GDS145 Bassanite
2. LASF41A Gypsum
3. LASA58 Saponite
Stacked Reflectance
Wavelength (nm)
GDS145 Bassanite
LASF41A Gypsum
LASA58 Saponite
bkr1jb329c Siliceous-sinter
799F366 Mg-sulphate
GDS1 Analcime
c1cy29 Vermiculite
F1CC08 Magnesite
F1CC08B Alunite
LAKA04 Kaolinite
Entry Ranked by Asc. Baseline Spectral Angle

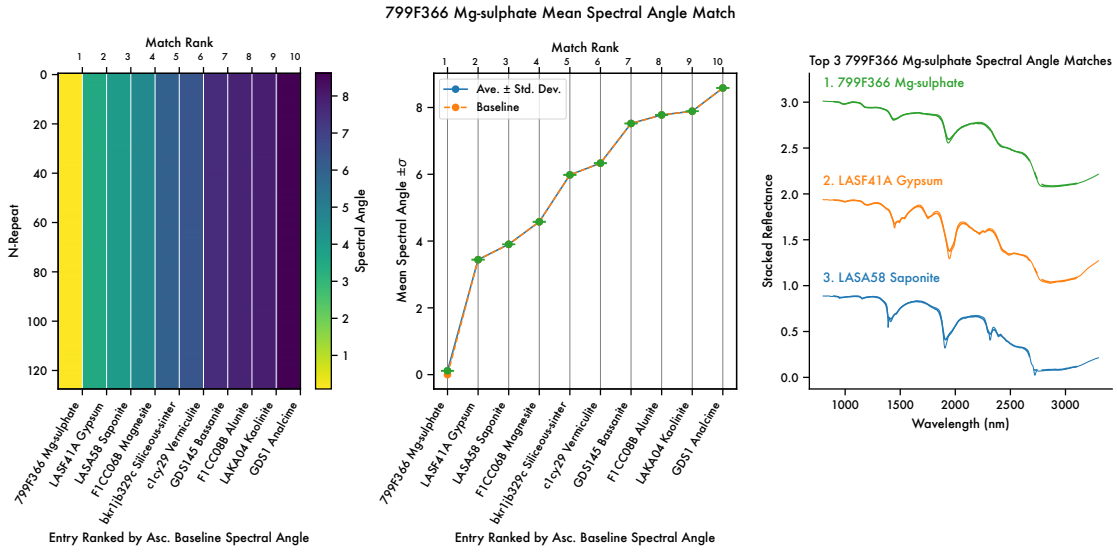

LASF41A Gypsum Mean Spectral Angle Match
Match Rank
N-Repeat
Spectral Angle
Mean Spectral Angle ±σ
Ave. ± Std. Dev.
Baseline
Top 3 LASF41A Gypsum Spectral Angle Matches
1. LASF41A Gypsum
2. 799F366 Mg-sulphate
3. LASA58 Saponite
Stacked Reflectance
Wavelength (nm)
LASF41A Gypsum
799F366 Mg-sulphate
LASA58 Saponite
bkr1jb329c Siliceous-sinter
F1CC08 Magnesite
F1CC08B Alunite
GDS145 Bassanite
c1cy29 Vermiculite
GDS1 Analcime
LAKA04 Kaolinite
Entry Ranked by Asc. Baseline Spectral Angle

LASF21A Jarosite Mean Spectral Angle Match



F1CC15 Kieserite Mean Spectral Angle Match





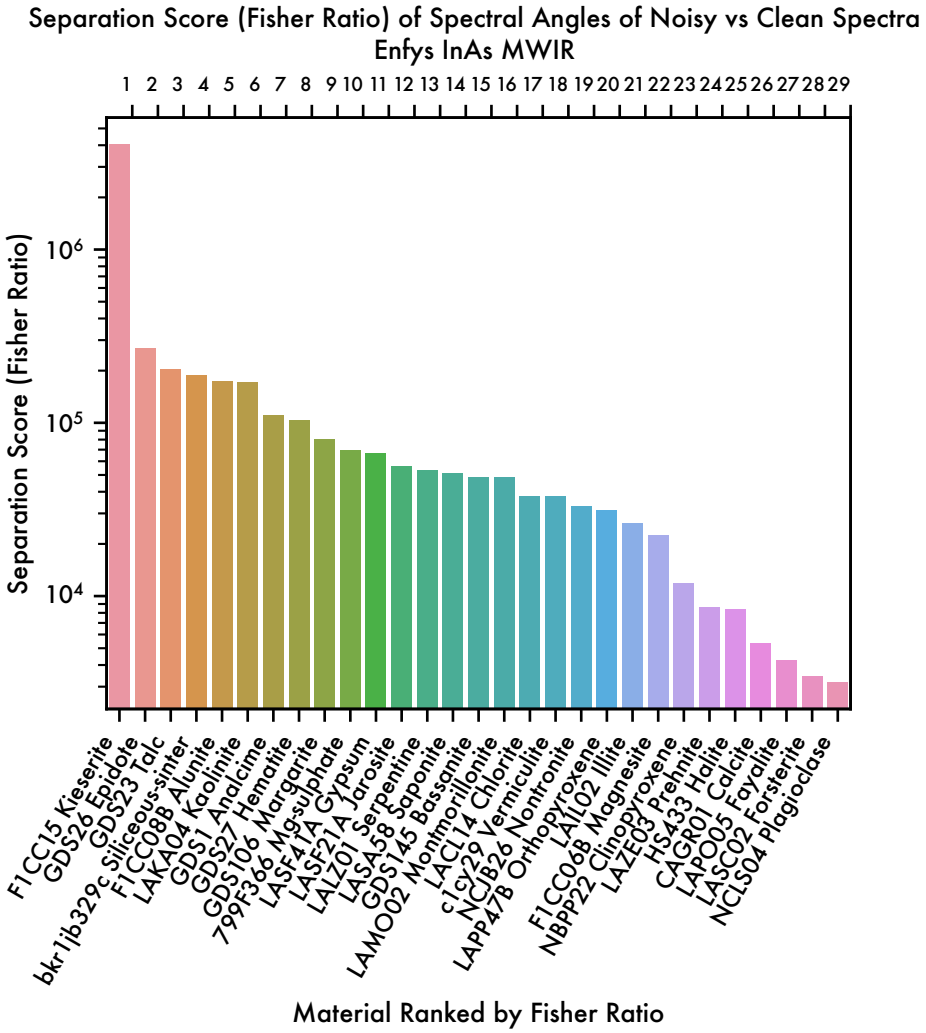
6.2 Summarizing the Spectral Angle Results

To collect the results of performing Spectral Angle mapper analysis we need to summarise this information across the instrument in one or two figures.

We summarise these results in the Fisher Ratio plot. We have ordered the Fisher Ratio results in decreasing order, such that lower entries on the x-axis have the best separation, and higher entries have worse separation.

[167]:
```
fig, ax = plt.subplots(figsize=(5,4))

g = sns.barplot(x=fr_series.index, y=fr_series.values, ax=ax)

# get the bar colours
bar_colours = [p.get_facecolor() for p in ax.patches]

xlbls=g.set_xticklabels(g.get_xticklabels(), rotation=60,␣
 ↪horizontalalignment='right', verticalalignment='top')
# set y axis to logarithm
g.set_yscale('log')
g.set_title('Separation Score (Fisher Ratio) of Spectral Angles of Noisy vs␣
 ↪Clean Spectra \n Enfys InAs MWIR')
g.set_xlabel('Material Ranked by Fisher Ratio')
g.set_ylabel('Separation Score (Fisher Ratio)')

# add a top y-axis called Match Rank, with ticks aligned to the right
ax2 = ax.twiny()
ax2.set_xlim(ax.get_xlim())
ax2.set_xticks(np.arange(len(fr_series.index))+0.5)
```

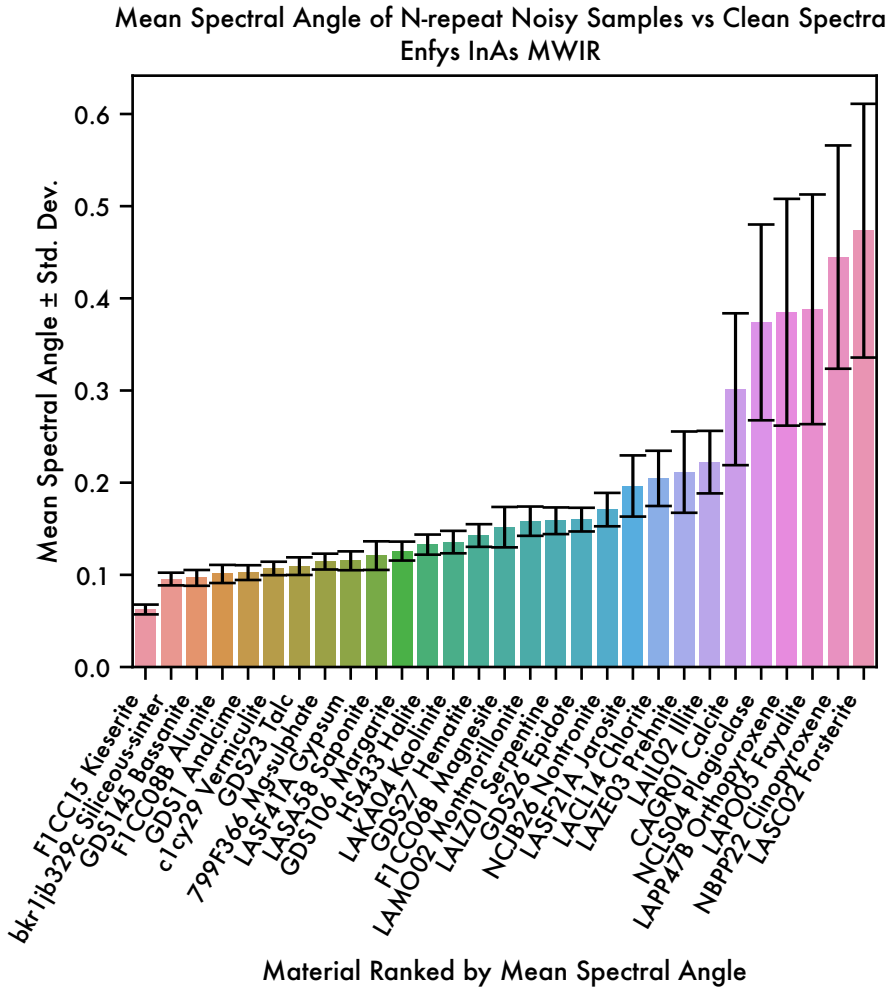
```
x2lbls=ax2.set_xticklabels(np.arange(1, len(fr_series.index)+1),␣
 ↪horizontalalignment='right', verticalalignment='bottom', fontsize=7)

fig.savefig(sa_dir / f'fisher_ratio_plot.pdf', bbox_inches='tight')
fig.savefig(sa_dir / f'fisher_ratio_plot.svg', bbox_inches='tight')
```



We can also plot the mean Spectral Angle, along with errorbars.

[168]:
```
fig, ax = plt.subplots(figsize=(5,4))

ax.bar(sa_top_df.index, sa_top_df['ave'], yerr=sa_top_df['std'], capsize=5)
g = sns.barplot(x=sa_top_df.index, y=sa_top_df['ave'], ax=ax)

g.set_title('Mean Spectral Angle of N-repeat Noisy Samples vs Clean Spectra \n␣
 ↪Enfys InAs MWIR')
```

```
g.set_xlabel('Material Ranked by Mean Spectral Angle')
g.set_ylabel('Mean Spectral Angle ± Std. Dev.')

xlbls = ax.set_xticklabels(ax.get_xticklabels(), rotation=60,␣
 ↪horizontalalignment='right', verticalalignment='top')

fig.savefig(sa_dir / f'spectral_angle_plot.pdf', bbox_inches='tight')
fig.savefig(sa_dir / f'spectral_angle_plot.svg', bbox_inches='tight')
```



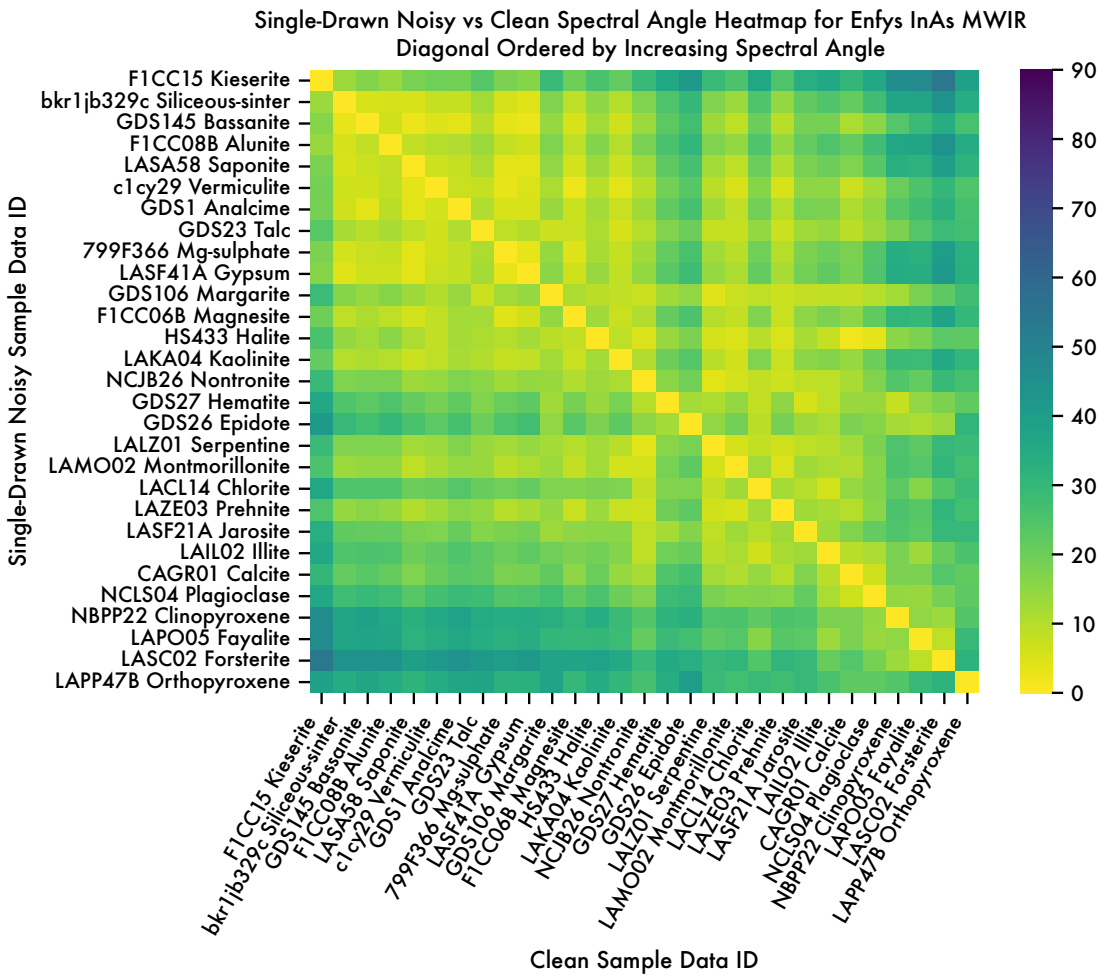
First, we evaluate the Spectral Angle between the single-sample noisy spectra and the clean spectra, and concatenate the results into an M x M array.

[169]:
```
sa_df_list = []
for material in materials:
    sa_df, sa_bl = compute_spectral_angle(obs_1, material)
    # standardise the ordering
```

```
    sa_df = sa_df.reindex(sorted(sa_df.columns), axis=1)
    sa_bl = sa_bl.reindex(sorted(sa_bl.index))
    sa_df_list.append(sa_df)
obs_1_sa_df = pd.concat(sa_df_list)
obs_1_sa_df = obs_1_sa_df.reindex(sorted(obs_1_sa_df.index), axis=0)
# zip the columns and index into a dictionary
zip_dict = dict(zip(obs_1_sa_df.index, obs_1_sa_df.columns))
# rename the index using this dictionary
obs_1_sa_df = obs_1_sa_df.rename(index=zip_dict)

# we can order the columns by increasing value of the diagonal entries of the␣
 ↪dataframe
sorted_by_sa = pd.Series(np.diag(obs_1_sa_df), index=obs_1_sa_df.columns).
 ↪sort_values(ascending=True)

# now applying the ordering to the dataframe, to the columns,
obs_1_sa_df = obs_1_sa_df[sorted_by_sa.index]
# and the rows
obs_1_sa_df = obs_1_sa_df.reindex(index=sorted_by_sa.index)
```

We can display the results as a heatmap.

[170]:
```
# displaying the dataframe as a heatmap
g = sns.heatmap(obs_1_sa_df, cmap='viridis_r', vmin=0, vmax=90)
g.set_title("Single-Drawn Noisy vs Clean Spectral Angle Heatmap for Enfys InAs␣
 ↪MWIR \n Diagonal Ordered by Increasing Spectral Angle")
g.set_ylabel("Single-Drawn Noisy Sample Data ID")
g.set_xlabel("Clean Sample Data ID")
# rotate x-axis labels
xlbls=g.set_xticklabels(g.get_xticklabels(), rotation=60,␣
 ↪horizontalalignment='right', verticalalignment='top')

fig.savefig(sa_dir / f'spectral_angle_heatmap.pdf', bbox_inches='tight')
fig.savefig(sa_dir / f'spectral_angle_heatmap.svg', bbox_inches='tight')
```



7 Discussion

7.1 Fisher Ratio Separation Score

We begin our discussion by considering the Fisher Ratio bar plot.

This shows that all minerals can be separated with a separation-to-variance ratio of >2000.

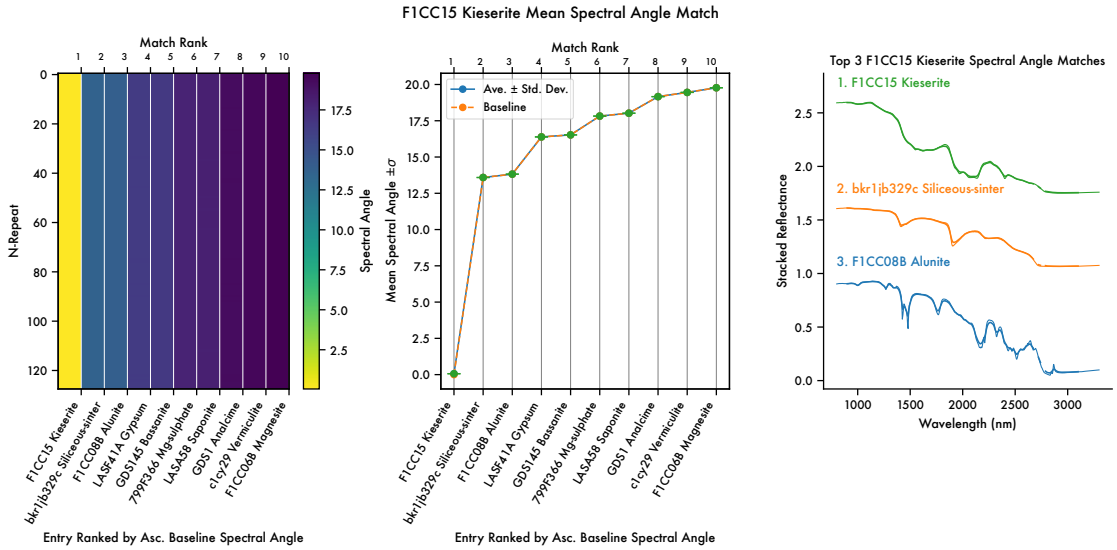
Let's consider the top 3 and bottom 3 scoring materials.

7.2 Top 3 Fisher Ratio Scores

The top Fisher Ratio scoring material is Kieserite.

[171]:
```
fig_ax_dict['F1CC15 Kieserite'][0]
```

[171]:

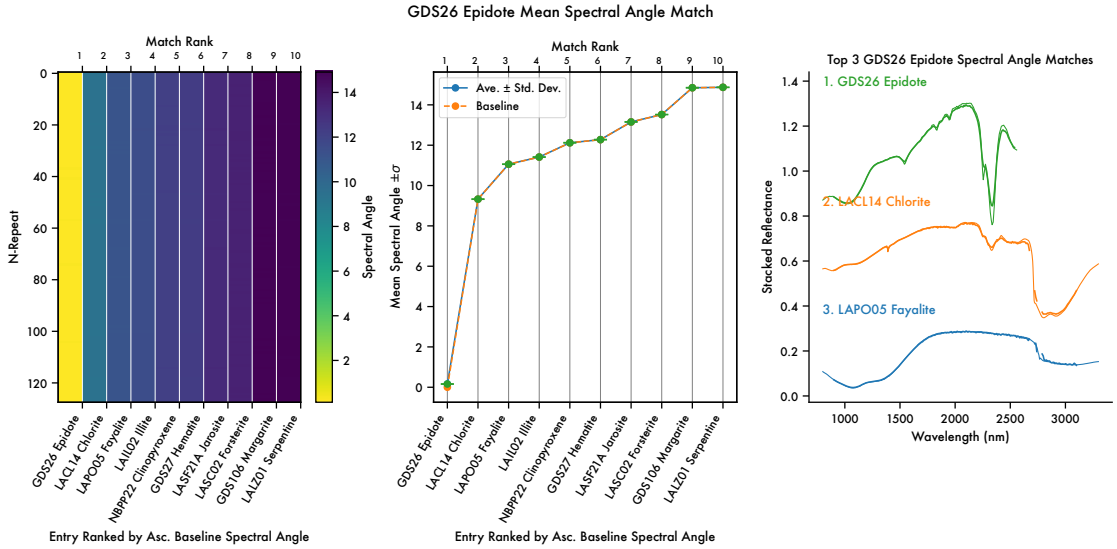


We can see here that there is indeed a large difference in spectral angle between the closest match, Kieserite, and the next closest match, Siliceous-Sinter (hydrated sillica). We can see fom the plot that the key difference is the breadth of the Kieserite absorption features, partciularly in comparison to those of the hydrated silica. This feature of Kieserite makes it easy to distinguish.

The next highest score is Epidote.

[172]:
```
fig_ax_dict['GDS26 Epidote'][0]
```

[172]:



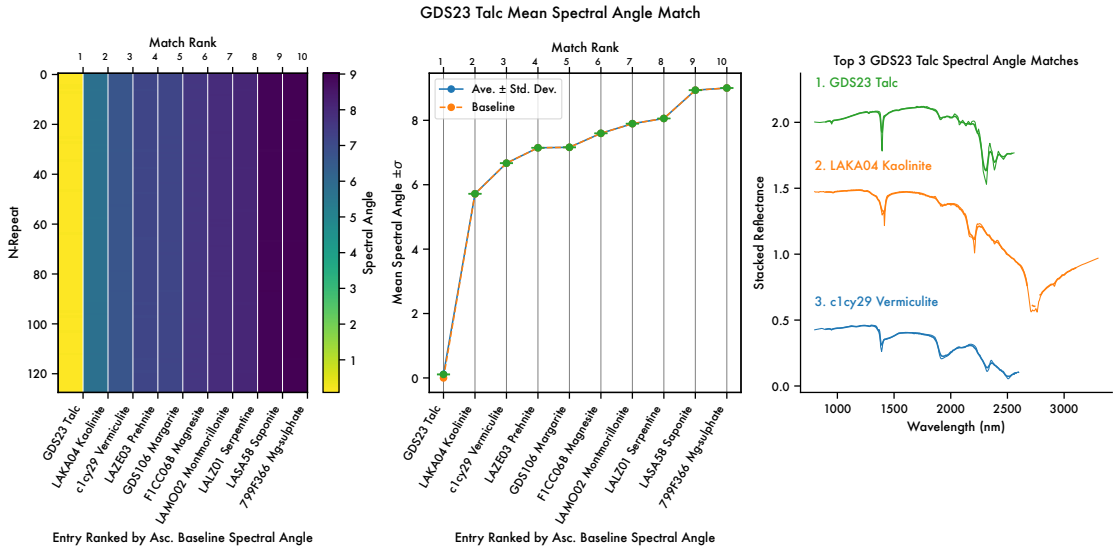
We can see that Epidote has a uniquely strong absorption band at ~2.35 µm, that also has a subtle doublet feature that is just resolved. This is distinct from the much shallower feature that Chlorite

exhibits.

The 3rd top scoring Fisher Ratio is Talc.

[173]:
```
fig_ax_dict['GDS23 Talc'][0]
```

[173]:



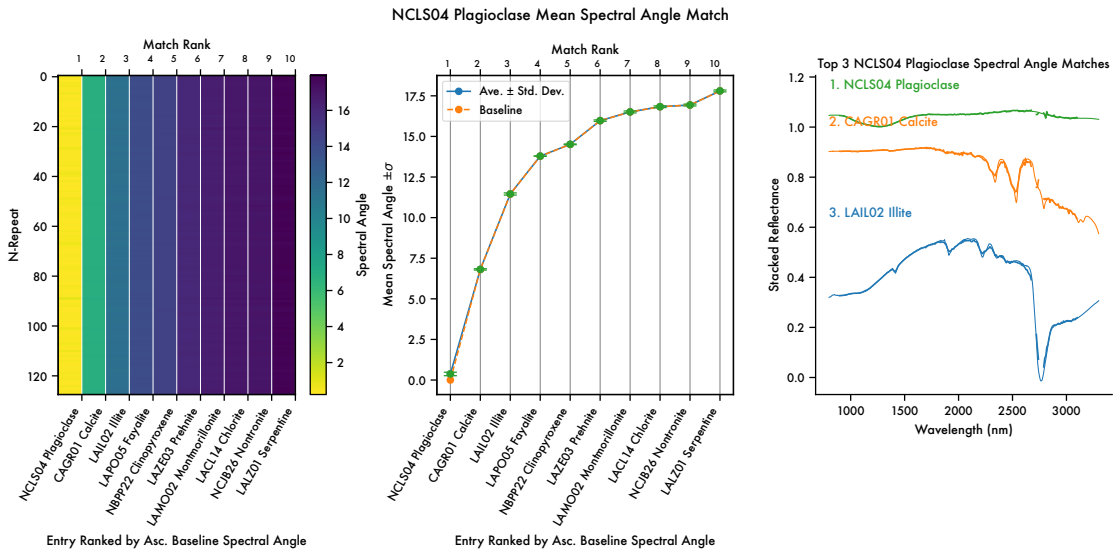
We can see that Talc has in common with next nearest Spectral Angle matches Kaolinite and Vermiculite a ~1.4 µm narrow absorption feature, and a shallow and broad ~1.8 µm feature, and that it is the ~2.25 µm feature that is the strongest distinguishing feature, that is indeed resolved here.

7.3 Bottom 3 Fisher Ratio Scores

The lowest scoring Fisher Ratio is for Plagioclase.

[174]:
```
fig_ax_dict['NCLS04 Plagioclase'][0]
```

[174]:

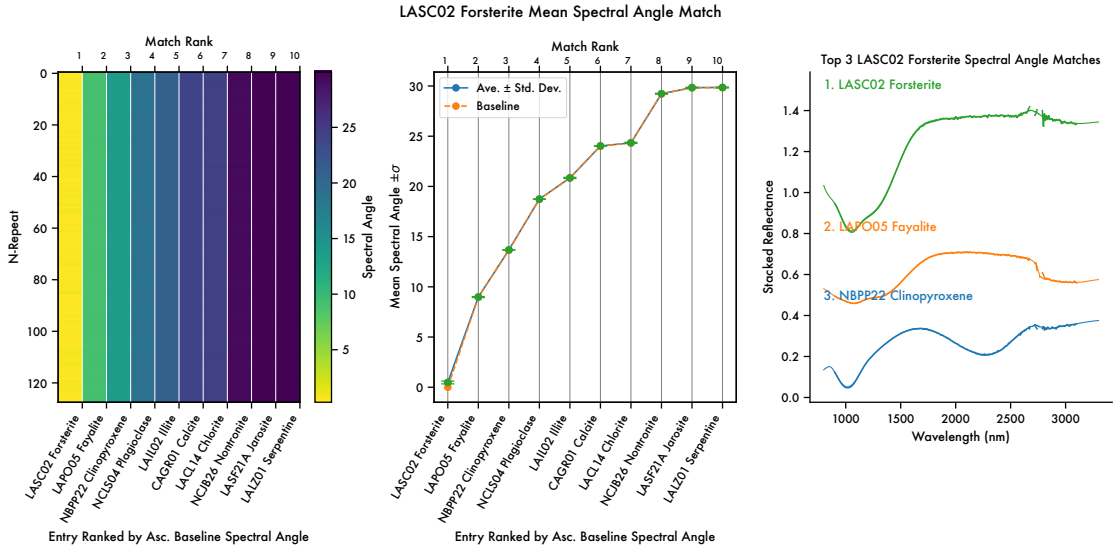


Plagioclase has low spectral variability, but a broad absorption feature at ~1.25 µm distinguishes it from calcite. In this example we can see that the spectrum for plagioclase suffers from the high noise in the CO2 ~2.6-2.7 µm gap, that is likely the cause of the high spectral angle value.

The next lowest scoring Fisher Ratio occurs for Forsterite.

[175]:
```
fig_ax_dict['LASC02 Forsterite'][0]
```

[175]:

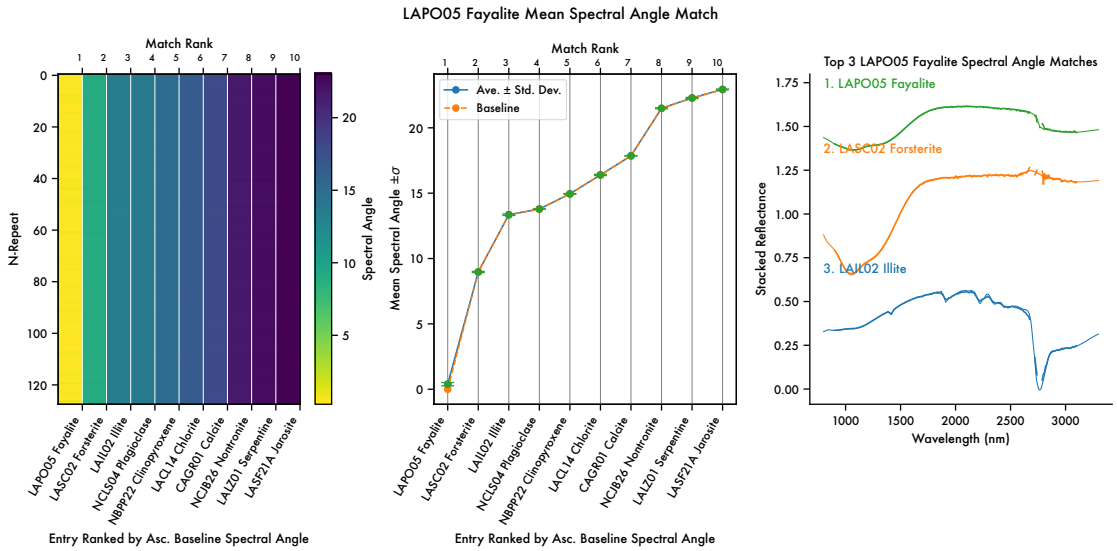


We can see that Forsterite and Fayalite, two different species of Olivine, have similar spectra, with a broad doublet feature of Fayalite contributing to the large difference in spectral angle that gives the relatively low, but still strongly statistically significant, Fisher Ratio score.

The thrid lowest Fisher Ratio score is for Fayalite.

[176]:
```
fig_ax_dict['LAPO05 Fayalite'][0]
```

[176]:



As discussed above, Fayalite and Forsterite have similar features, as both are Olivines, but despite their low Fisher Ratio relative to the MICA group, the spectral angle separation is still statistically significant.

8 Conclusions

In conclusion, the InAs MWIR sensor of Enfys performing at the nominal expected SNR is capable of resolving and distinguishing all minerals of the MICA files.

9 References